



UNIVERSITATEA DIN
BUCUREȘTI
FACULTY OF
MATHEMATICS AND
COMPUTER SCIENCE



NATURAL LANGUAGE PROCESSING

Master's Thesis

On the Use of In-Game LLMs for Dynamic Non-Playable Character Behavior

Graduate

Ciobanu Radu-Florin

Coordinator

Lect. Nisioi Sergiu

Bucharest, September 2026

Abstract

Our goal is to leverage the capabilities of modern Large Language Models (LLMs) along the axis of natural interaction in video games and identify where the tradeoff lies between output variety and computational requirements. Current systems designed for non-playable characters (NPCs) are still frequently limited to pre-scripted, short and repetitive dialogue, scaling poorly with the size of the world while also partially affecting the replayability of the game.

This work comprises a native inference implementation of state-of-the-art LLMs in the Unity game engine free of any external dependencies, runned concurrently with Text-to-Speech (TTS) under custom optimizations in accordance with the constraints imposed by real-time graphics rendering.

We show that the latest sub-billion models ([Qwen 3.5 0.8B](#), [pocket-tts](#)) bring significant benefits to game immersion, while being deployable even on old, low-budget consumer-grade GPUs through our dedicated shader kernels, without having to drastically compromise on frame rates or real-time interaction. We publish our source code, post-trained models and synthetic datasets within our DeepUnity framework for the community to build upon.

Contents

1	Introduction	3
2	Preliminaries	5
2.1	Feed Forward Network	5
2.1.1	Gated Linear Units	6
2.1.2	Mixture of Experts	7
2.2	Attention Mechanism	8
2.2.1	Self-Attention mechanism	8
2.2.2	Causal Attention, KV Cache Growth and Transformer++	10
2.2.3	Multi-Query Attention (MQA)	11
2.2.4	Grouped Query Attention (GQA)	12
2.2.5	Attention Module Optimizations	13
2.2.6	Linear Attention	15
2.3	Linear Recurrent Models	17
2.3.1	Structured State Space Models (S4)	17
2.3.2	Selective State Space Models (S6)	19
2.3.3	Parallel Associative Scan	22
2.3.4	Gated DeltaNet	24
2.4	Positional Embeddings	27
2.4.1	Absolute Positional Embeddings	27
2.4.2	Relative Position Embeddings	28
2.4.3	Rotary Positional Embeddings	29
2.5	Inference Optimizations	31
2.5.1	Speculative Decoding	31
2.5.2	Multi-Token Prediction (MTP)	33
2.5.3	Quantization	35
2.6	Finetuning Methods	36
2.7	DeepUnity Framework	41
2.7.1	Deep Learning Library	42
2.7.2	Reinforcement Learning Library	42

3	Implementation	43
3.1	Implementation Details	43
3.1.1	Tokenization	43
3.1.2	Compute Kernels and Quantization	47
3.2	Interface	48
3.2.1	Designing the NPC Interface	49
3.2.2	Interface Features	49
3.2.3	The Memory System	50
3.3	Optimizations	51
3.3.1	Loading Across Frames	51
3.3.2	The Dialogue Pipeline	51
3.3.3	Selecting the Performance Tradeoff	52
3.4	Finetuning for Roleplay	53
3.4.1	Synthetic Dataset	54
3.4.2	Tool Use and the Gating Law	56
3.4.3	Training Procedure	56
4	Evaluation	57
4.1	Prefill and Decoding Throughput	58
4.2	Comparison Against Reference Engines	61
4.3	Latency Distribution Across the Network	62
5	Conclusion and Future Work	63
	Bibliography	65
	Appendix: Notation	73
	Appendix	74

Chapter 1

Introduction

In 2024, we released DeepUnity¹, an open-source framework for training RL Agents in Unity environments. The tools provided by Unity create an incredible framework for designing RL environments and rewarding systems, making it easy for game developers to train their AI systems right in the engine, without the need of prior PyTorch experience or libraries with a steep learning curve like MuJoCo to then adapt back to their actual game. Since the library has zero dependencies, it also acts as a Deep Learning library, useful for common tasks using neural networks.

In this paper we are taking the next step in game’s AI integration: Player to NPC interactions. We ask whether convincing, real-time AI NPCs are worth sharing a fraction of the GPU budget, or whether dialogue should take priority over graphics, so to speak. With the rise of small language models, capable of running directly on device, and with relatively good performance in acting like chatbots even at sub-1B parameters, it is their moment to be part of the modern game narrative design. Over the course of years, even since the early days of games, the player interaction with the world has progressed very little, with similar NPC mechanics and tree-based dialogue systems that sometimes just barely mimic the importance of player input. While the story side is easily neglected with such omission, a system objectively passing the Turing test can expand the dimensionality of the gameplay, expanding it on the axis of dialogue and art of negotiation.

The *Nemesis* system (Hoge 2018; Plater et al. 2021) introduced in **Shadow of Mor-dor**, is a novel storyline building mechanic that creates unique and personalized interactions between the player and the game’s NPCs. It was designed with several goals: it defines *orc* NPCs with different levels of strengths, vulnerabilities, personalities, fears etc., it builds hierarchical and non-hierarchical relationships between them (rivalries, friendships, etc.), implements the persistence of memory in the player’s interactions and many others, so that the player experiences a better but still shallow story progression through his own interactions.

¹<https://github.com/smtmRadu/DeepUnity>

A prevalent demo that showcases the strengths of AI-driven NPCs was NVIDIA ACE (Avatar Cloud Engine, NVIDIA 2023), combining the power of Automatic Speech Recognition (ASR), Text-To-Speech (TTS), Language and Audio-To-Face (A2F) models to spark life, though it remains extremely compute intensive and since then it was not adopted to real products. Besides compute, the only other major consumer of GPU memory is graphics; other parallelizable game workloads such as physics, pathfinding, or particle systems conventionally run on the CPU, so they never contend for it. This leaves our inference to share the GPU with rendering alone, and it is precisely this narrow resource budget that we design around, aiming to assemble a convincing player experience within indie-game-like 3D environments across our demos.

Throughout this paper, we cover the mechanics of modern LLMs, their recent architecture optimizations and serving improvements, along with their integration in the Unity Game Engine. We also discuss the post-training techniques to design roleplaying agents and draw cross-model and cross-GPU performance metrics. More specifically, our contributions consist of:

1. an on-device, in-engine inference framework for LLM and TTS models with dedicated optimized kernels;
2. a production-ready integration interface for developers to create AI-driven NPCs;
3. multiple example demos and various benchmark studies.

The following goes through the required background on state-of-the-art architectures in order to lay the mathematical foundations, focusing on modern optimizations over the standard Transformer++ and revising latest post-training recipes that overcomes resources budgets and classic degradation pitfalls such as catastrophic forgetting. The following chapter describes the model adjustments we make to create a believable roleplaying agent, enhancing it with custom tool calling, vocal bursts (for TTS) and chat summarization capabilities. The remaining chapters synthesize the specific implementation designs and tradeoffs we apply for Unity and our goals, ending with evaluation reports and recommendations.

A summary of notation and conventions used throughout this paper can be found at the end, in the Appendix.

Chapter 2

Preliminaries

Over the years, the original Transformer decoder received numerous small improvements that formed till this day a standard baseline in benchmarking new architectures. Layer Normalization (Ba, Kiros, and G. E. Hinton 2016, Zhang and Sennrich 2019), Rotary Positional Embeddings and Gated MLPs are now the performance standard modules that form the so called Transformer++ model (Gu and Dao 2023). While scaling (Kaplan et al. 2020) defined a guaranteed recipe for improvement, the mainstream path of research has been to optimize the architecture itself.

This chapter introduces the main concepts used in recent research to permit scaling while maintaining performance. We briefly review the fundamental components of transformer-based language models, outline several architectural and efficiency improvements, and discuss the practical techniques that make modern LLM deployment feasible.

2.1 Feed Forward Network

Inside any architecture, the feed forward network (FFN) does the heavy lifting, since the majority of the parameters belongs to it. It is responsible for shaping and memorizing the concept space (Geva et al. 2021). We define a standard (2 layered) Feed Forward Network as follows:

$$\text{FFN}(\mathbf{u}) = \sigma(\mathbf{u}\mathbf{W}_{\uparrow}^{\top} + \mathbf{b}_1)\mathbf{W}_{\downarrow}^{\top} + \mathbf{b}_2 \quad (2.1)$$

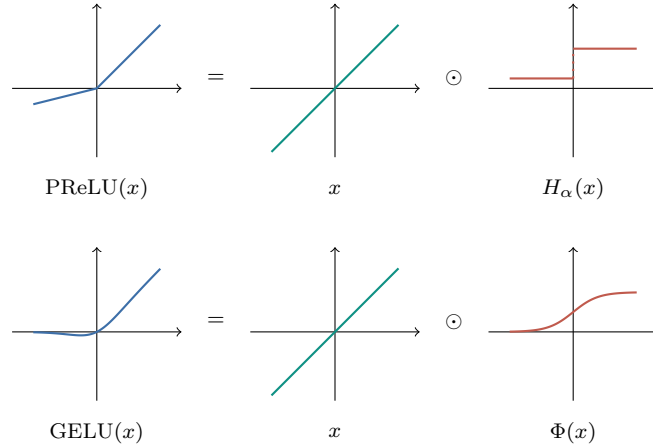
where $\mathbf{u} \in \mathbb{R}^d$, $\mathbf{W}_{\uparrow} \in \mathbb{R}^{\epsilon d \times d}$, $\mathbf{W}_{\downarrow} \in \mathbb{R}^{d \times \epsilon d}$, $\mathbf{b}_1 \in \mathbb{R}^{\epsilon d}$, $\mathbf{b}_2 \in \mathbb{R}^d$ ($\epsilon \geq 1, \epsilon d \in \mathbb{N}$) and σ is any nonlinear function.

In a simple manner of understanding the FFN, each hidden unit (a neuron) can be described as being responsible for one or more concepts (Elhage et al. 2022). When the concept triggers (i.e., the inner product between the hidden representation and the neuron’s weights is high), the neuron’s activation is high; otherwise, with a ReLU activation, it is zeroed out. In other words, on the channel axis, FFNs act as key-value memories

(Geva et al. 2021), where $\mathbf{u}$ acts as the query, the rows of $\mathbf{W}_\uparrow$ act as a set of keys, and the columns of $\mathbf{W}_\downarrow$ represent the values; updating the residual stream towards predicting the next token. Early layers capture general concepts, while later layers specialize in deeper, more specific ones.

2.1.1 Gated Linear Units

Self-gated activations can be viewed as a composition of two components: the content and the gate. For example:



where H_α is the step function lifted to α over negative inputs and Φ is the cumulative distribution function of the standard Gaussian. PReLU’s gate is discontinuous and makes sudden high jumps in gradients when the input crosses zero, leading to slower optimization. On the other hand, GELU’s gate is smooth, while also being the definition of the mathematical output value of a neuron in the context of dropout (Hendrycks and Gimpel 2016). Dauphin et al. (2017) parameterizes the gate with its own projection matrix, yielding the Gated Linear Unit (GLU):

$$\text{FFN}(\mathbf{u}) = (\mathbf{u}\mathbf{W}_\uparrow^\top \odot \sigma(\mathbf{u}\mathbf{W}_{gate}^\top))\mathbf{W}_\downarrow^\top \quad (2.2)$$

where $\mathbf{u} \in \mathbb{R}^d$, $\mathbf{W}_\uparrow, \mathbf{W}_{gate} \in \mathbb{R}^{\epsilon d \times d}$, $\mathbf{W}_\downarrow \in \mathbb{R}^{d \times \epsilon d}$, ($\epsilon \geq 1, \epsilon d \in \mathbb{N}$) and σ is the non-linear activation.¹ The unbounded multiplicative operations grow in a squared manner comparing to gate, and that becomes sensitive in low precision on the positive end. Kimi Team, Moonshot AI 2026 propose *capping* the input linear projections ($\mathbf{u}\mathbf{W}_\uparrow^\top$ and $\mathbf{u}\mathbf{W}_{gate}^\top$) by passing them through a parametrized Tanh activation in order to avoid large value accumulations.

¹Modern architectures skip the bias terms for computational efficiency and use normalization layers instead for training stability

2.1.2 Mixture of Experts

The training and inference costs increase linearly as the number of parameters in the model gets larger. Currently, the best way to address this is to have a sparse activation of the model, more precisely to use only a subset of parameters to process a given input.

Firstly, we define a Mixture of Experts layer as a set of E distinct feed forward networks. We will consider a subset of $S \ll E$ such networks as shared experts across all possible input spaces (i.e. they are always active), and the rest N experts as routed experts (i.e. a set of sparsely activated networks, selected based on given input $\mathbf{u} \in \mathbb{R}^d$), such that $S + N = E$. The empirical results are somewhat mixed (Muennighoff et al. 2025) when it comes to the actual need of a shared expert, but the next formulation will generalize.

We will define a routing system as a function that projects the input vector to a map of all possible routed experts.

$$\phi(\mathbf{u}) = \sigma(\mathbf{u}\mathbf{W}_\phi^\top) \quad (2.3)$$

where $\mathbf{W}_\phi^\top \in \mathbb{R}^{N \times d}$ represents the router parameters and σ is the sigmoid activation function.

Based on the router result ($\phi(\mathbf{u}) \in \mathbb{R}^R$), we select the top- k scores ($1 \leq k \ll R$ and define a sparse gate $\mathbf{g} \in \mathbb{R}^N$ that is used to mask the non-routed experts and weight the vector importance:

$$g_i = \begin{cases} \phi_i(\mathbf{u}), & \phi_i(\mathbf{u}) \in \text{TopK}(\phi_{\mathbf{u}}, k) \\ 0, & \text{otherwise} \end{cases} \quad (2.4)$$

MoE models, in this routing formulation, suffer from unbalanced loading (i.e. the gate opens only for certain experts during training) eventually leading to routing collapse. This issue was addressed using a penalty loss on the router to encourage the router to distribute the input vectors more evenly during the early stage of pretraining, while DeepSeek-AI 2025 proposes adding a bias $\mathbf{b} \in \mathbb{R}^N$ when doing the top- k selection. At each update step, each of the N bias terms is individually modified by an update parameter γ (e.g. for a highly loaded expert j in the batch, $\mathbf{b}_j = \mathbf{b}_j - \gamma$ and viceversa). Note that the same authors adopt the use of a larger number of smaller experts (for DeepSeekMoE $h = 7168$, $S = 1$, $N = 256$, $k = 8$, $h^{\text{expert}} = 2048$ routed experts of which 8 are active, each with a hidden dimension of 2048) over a small number of larger ones. Thus, the routing is redefined as:

$$g_i = \begin{cases} \phi_i(\mathbf{u}), & \phi_i(\mathbf{u}) + \mathbf{b}_i \in \text{TopK}(\phi(\mathbf{u}) + \mathbf{b}, k) \\ 0, & \text{otherwise} \end{cases} \quad (2.5)$$

Knowing all its components, we define a Mixture of Experts layer as such:

$$\text{MoE}(\mathbf{u}) = \sum_{i=1}^S \text{FFN}_i^{\text{shared}}(\mathbf{u}) + \sum_{i=1}^N \frac{\mathbf{g}_i}{\sum \mathbf{g}} \text{FFN}_i^{\text{routed}}(\mathbf{u}) \quad (2.6)$$

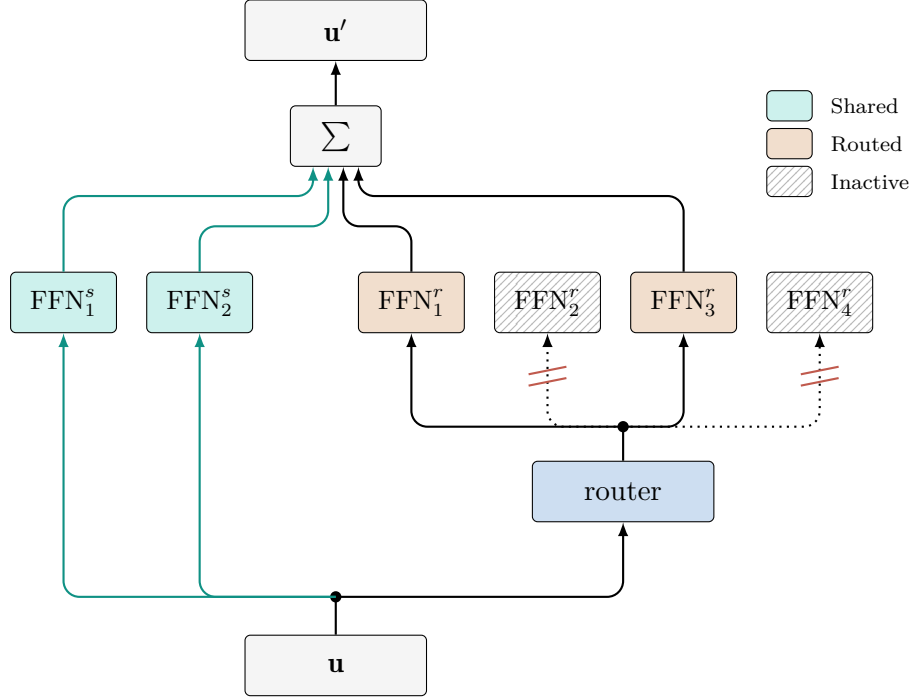


Figure 2.1: An MoE layer. All tokens $\mathbf{u}$ pass through the shared experts, while a router computes scores for routed experts and passes the current token selectively to a subset of k smaller FFNs. The final output sums the shared expert outputs and the normalized gated outputs of the selected routed experts.

2.2 Attention Mechanism

While the feed forward is handling channel mixing (interactions between the features of a token), a module that handles interactions between the tokens is necessary (time mixing). Before the introduction of the Transformer architecture Vaswani et al. 2017, the RNNs (Elman 1990, Hochreiter and Schmidhuber 1997, Cho et al. 2014) were dominating the time sequence processing tasks along with their scaling bottleneck of sequential computation. Linear variants (Gu and Dao 2023) with time-parallel computation still lag behind in terms of performance of the attention module, the core ingredient of the Transformer; and that is simply defined by the *memory* mechanism that differ from one another.

2.2.1 Self-Attention Mechanism

In the decoder-only architecture, we recall the self-attention definition as follows:

$$\begin{aligned}\text{SelfAttention}(\mathbf{u}) &= \text{softmax}\left(\frac{\mathbf{Q}^\top \mathbf{K}}{\sqrt{d}}\right) \mathbf{V} \\ \mathbf{Q} &= \mathbf{u} \mathbf{W}_Q^\top \quad \mathbf{K} = \mathbf{u} \mathbf{W}_K^\top \quad \mathbf{V} = \mathbf{u} \mathbf{W}_V^\top\end{aligned}\tag{2.7}$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{d \times d\epsilon}$ are linear projections of the input $\mathbf{u} \in \mathbb{R}^{T \times d}$, T is the sequence length, d the dimension of the model, ϵ the expansion size for the attention layer (for simplicity we can consider $\epsilon = 1$) and the projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d\epsilon}$.

The attention mechanism can be interpreted as an information sharing between the vector representations of the tokens. The projection learn to highlight features for the input tokens that matter when computing the inner product between two of their vector representations. So, if the linear projection of the i -th token $\mathbf{q}_i$ is aligned with the linear projection of the j -th token $\mathbf{k}_j$, then $\mathbf{q}_i^\top \mathbf{k}_j$ will be high (the value of this operation is known as score). This relevance weights are normalized by the softmax function to sum up to 1, and this attention vector will act as an information retrieval gate for a given token that will be applied over the *value* representations $\mathbf{v}_k$ and $\mathbf{v}_j$ respectively. Basically in a full self-attention operation, any token k will end up as a weighted sum between all the vectors in the sequence (including itself, with a caveat²).

To enhance the diversity of feature focus, running multiple self-attention heads in parallel at the same layer in the network is a standard. The operation is done separately on $\mathbf{q}, \mathbf{k}$ and $\mathbf{v}$ vectors subspaces and concatenated at the end. A final linear projection is responsible to mix the information across the heads and reduce the vectors to the model space (if the expansion size $\epsilon > 1$).

$$\text{MHA}(\mathbf{u}) = [\mathbf{h}_1; \mathbf{h}_2; \dots; \mathbf{h}_H] \mathbf{W}_O^\top \tag{2.8}$$

$$\mathbf{h}_i = \text{softmax}\left(\frac{\mathbf{Q}_i^\top \mathbf{K}_i}{\sqrt{d_h}}\right) \mathbf{V}_i \tag{2.9}$$

where $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i \in \mathbb{R}^{T \times d_h}$ are the head-wise splits of the query, key and value projections for each head $i = 1, 2, \dots, H$, $d_h = d/H$ is the head dimension and $\mathbf{W}_O \in \mathbb{R}^{d \times d\epsilon}$ is the output projection.

$$\begin{aligned}\mathbf{Q} &= \mathbf{u} \mathbf{W}_Q^\top = [\mathbf{Q}_1; \mathbf{Q}_2; \dots; \mathbf{Q}_H] \\ \mathbf{K} &= \mathbf{u} \mathbf{W}_K^\top = [\mathbf{K}_1; \mathbf{K}_2; \dots; \mathbf{K}_H] \quad \mathbf{V} = \mathbf{u} \mathbf{W}_V^\top = [\mathbf{V}_1; \mathbf{V}_2; \dots; \mathbf{V}_H]\end{aligned}\tag{2.10}$$

As a side note, the role of score scaling by $\sqrt{d} \in \mathbb{R}$ is to decrease their magnitude so at the early stages of the training, the attention weight distribution to be close to even (tokens have a chance to attend to all tokens from the sequence and this will surpass a

²Attending over itself acts as a form of channel mixing, competing directly with the role handled by the FFN. As described in Zhai 2026, the *value* projections naturally have high cosine similarity with their embeddings, wasting attention capacity that could otherwise be spent on time mixing.

probable local minima).

All tasks that imply time series are dealing with the necessity of tokens to receive positional information: vector representations should be enhanced with positional information before the attention operation, because the word order matters. This is described in detail at 2.4.

2.2.2 Causal Attention, KV Cache Growth and Transformer++

For generative Large Language Models, the self-attention operation is never used at its full power, and a tradeoff is made between the performance and generation speed. While language foundation models like BERT Devlin et al. 2018 can be used to decode the next token in the sequence by appending a masked token to the input, it will require the entire self-attention recomputation to be done, which has a complexity of $\mathcal{O}(T^2)$.

In causal attention, a mask enforces that each token may only attend to itself and all preceding tokens. A common optimization is sliding window attention, where each token attends only to the w most recent positions (including itself), reducing the per-token cost from $\mathcal{O}(T)$ to $\mathcal{O}(w)$ while still allowing information to propagate across the full context through successive layers.

For auto-regressive models, a trade-off is made between generation speed and model accuracy. While bidirectional attention is richer in feature extraction and accuracy, it keeps the quadratic computation and memory complexity even at inference time. The causal attention instead runs in linear complexity at inference at the cost of limiting the power of the network. At inference, the KV-Cache increases linearly with the context size. We can compute its size as follows:

$$\text{size}(\text{KV}_{\text{Cache}}) = H \times d_h \times T \times L \times b \times 2 \quad (2.11)$$

where d is the dimension of the model, ϵ is the attention expansion size, T is the context size, L is the number of layers in the decoder and b is the precision (the number of bytes allocated per parameter).

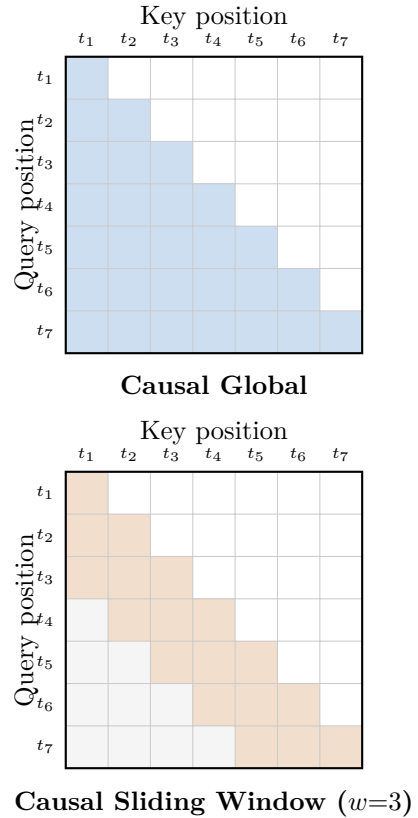


Figure 2.2: In causal attention (top), each token attends to all preceding tokens. In sliding window causal attention (bottom), each token attends only to the w most recent positions.

Considering a small transformer with 4B parameters (typically $d = 2560$, $\epsilon = 1.6$ and $L = 36$), of which size would be roughly $\approx 8GB$ in `float16` format, the dimension of the KV for a context size of 4096 tokens (still in `float16`) would be $2560 \times 1.6 \times 4096 \times 36 \times 2 \approx 1.2B$ values, or $\approx 2.4GB$. And that’s an enormous amount of memory usage given that it must stay ready in the HBM for computation and cannot be retrieved selectively from slow persistent storages. The memory holds as the main bottleneck and progress was made in reducing the KV cache size to enable longer context sizes that we will uncover in the following section.

Beside that, this form of causal self-attention (for time-mixing, with rotary embeddings over its queries and keys, see Section 2.4.3) and SwiGLU feed-forward (for channel-mixing) combined represent the default modules that form an LLM block in architecture baselines, named as *Transformer++* (Gu and Dao 2023). With that, we will define a standard LLM backbone as a stack of L layers as in Algorithm 1.

Algorithm 1 Transformer++ Block. Combines Causal Self-Attention (with RoPE) and SwiGLU, both prenormalized by RMSNorm (Zhang and Sennrich 2019) and with residual connections (He et al. 2016) for depth. No QK norm used or MoE.

Input: $\mathbf{u} \in \mathbb{R}^{B \times T \times d}$, causal mask $\mathbf{M} \in \mathbb{R}^{T \times T}$, $M_{ij} = 0$ for $j \leq i$ and $-\infty$ otherwise

Output: $\mathbf{u}' \in \mathbb{R}^{B \times T \times d}$

Require: $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O \in \mathbb{R}^{d \times d}$

Require: $\mathbf{W}_\uparrow, \mathbf{W}_{gate} \in \mathbb{R}^{\epsilon d \times d}$, $\mathbf{W}_\downarrow \in \mathbb{R}^{d \times \epsilon d}$

Require: $\mathbf{w}_1, \mathbf{w}_2 \in \mathbb{R}^d$, the two RMSNorm gains

- 1: $\mathbf{r} \leftarrow \mathbf{w}_1 \odot \mathbf{u} / \sqrt{\frac{1}{d} \sum_{i=1}^d u_i^2}$ ▷ pre-norm
 - 2: $\mathbf{Q}, \mathbf{K}, \mathbf{V} \leftarrow \mathbf{r} \mathbf{W}_Q^\top, \mathbf{r} \mathbf{W}_K^\top, \mathbf{r} \mathbf{W}_V^\top$
 - 3: $\mathbf{Q}, \mathbf{K} \leftarrow \text{RoPE}(\mathbf{Q}), \text{RoPE}(\mathbf{K})$ ▷ cached across steps
 - 4: $\mathbf{u} \leftarrow \mathbf{u} + \text{softmax}(\mathbf{Q} \mathbf{K}^\top / \sqrt{d} + \mathbf{M}) \mathbf{V} \mathbf{W}_O^\top$
 - 5: $\mathbf{r} \leftarrow \mathbf{w}_2 \odot \mathbf{u} / \sqrt{\frac{1}{d} \sum_{i=1}^d u_i^2}$
 - 6: $\mathbf{u}' \leftarrow \mathbf{u} + (\mathbf{r} \mathbf{W}_\uparrow^\top \odot \sigma(\mathbf{r} \mathbf{W}_{gate}^\top)) \mathbf{W}_\downarrow^\top$ ▷ SwiGLU
 - 7: **return** $\mathbf{u}'$
-

2.2.3 Multi-Query Attention (MQA)

Generating a single key and value vector and share it across all heads Shazeer 2019 is one efficient method to scale down the KV Cache dimension by the number of heads in the attention layer. While also reducing the number of parameters and computation required, this trick was adopted especially for the KV caching problem at the cost of losing some performance (in their tests).

If H is the number of attention heads, then the KV Cache is reduced approximately by a factor of H :

$$\text{size}(\text{KV}_{\text{Cache}}^{\text{MQA}}) = \frac{1}{H} \text{size}(\text{KV}_{\text{Cache}}^{\text{MHA}}) \quad (2.12)$$

This improves decoding throughput and allows longer contexts under the same memory budget. The main drawback is that all heads must share the same key-value representation, which reduces the expressiveness of the attention layer and can lead to a small quality drop compared to standard multi-head attention.

The per-head computation becomes:

$$\mathbf{h}_i = \text{softmax}\left(\frac{\mathbf{Q}_i^\top \mathbf{K}}{\sqrt{d_h}}\right) \mathbf{V} \quad (2.13)$$

where $\mathbf{Q}_i \in \mathbb{R}^{T \times d_h}$ is the i -th head query split (as in MHA), while $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times d_h}$ are shared across all heads:

$$\begin{aligned} \mathbf{Q} &= \mathbf{u} \mathbf{W}_Q^\top = [\mathbf{Q}_1; \mathbf{Q}_2; \dots; \mathbf{Q}_H] \\ \mathbf{K} &= \mathbf{u} \mathbf{W}_K^\top \quad \mathbf{V} = \mathbf{u} \mathbf{W}_V^\top \end{aligned} \quad (2.14)$$

where $\mathbf{W}_Q \in \mathbb{R}^{d \times d_e}$ and $\mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d_h}$.

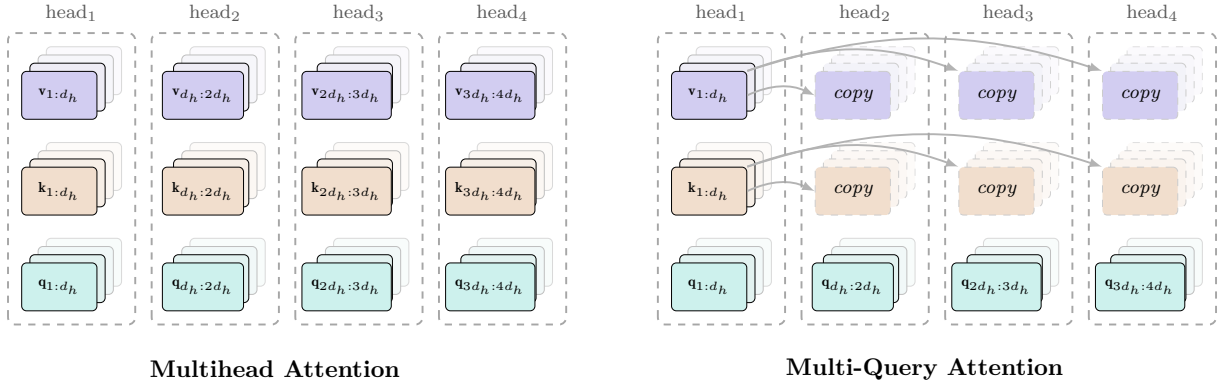


Figure 2.3: Comparison of standard multi-head attention (left) and multi-query attention (right). Each head operates on d_h dimensions. In MHA, each head has unique projections. In MQA, a single key-value projection is shared across all heads, while queries remain independent. The depth axis represents the sequence (time) dimension.

2.2.4 Grouped Query Attention (GQA)

As an interpolation between MHA and MQA, Grouped Query Attention (GQA) (Ainslie et al. 2023) is a general formulation for multi-headed design, making groups of query heads adjustable in the speed-quality space. Here, G different groups of heads share the same key-value projections. When $G = 1$, all heads share the same KV projection, equaling

MQA. When $G = H$, it is equivalent to MHA. The per-head computation becomes:

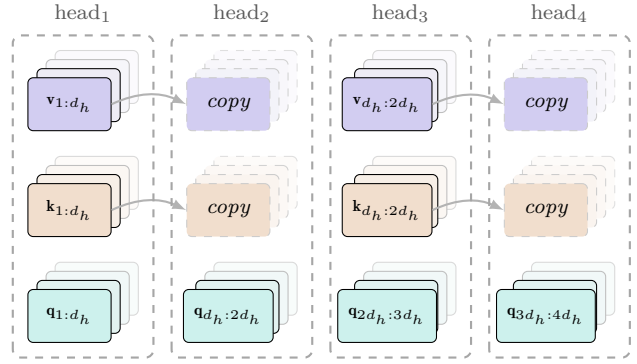
$$\mathbf{h}_i = \text{softmax}\left(\frac{\mathbf{Q}_i^\top \mathbf{K}_{g(i)}}{\sqrt{d_h}}\right) \mathbf{V}_{g(i)} \quad (2.15)$$

where $g(i) = \lceil iG/H \rceil$ maps head i to its KV group. The projections are:

$$\begin{aligned} \mathbf{Q} &= \mathbf{u} \mathbf{W}_Q^\top = [\mathbf{Q}_1; \mathbf{Q}_2; \dots; \mathbf{Q}_H] \\ \mathbf{K} &= \mathbf{u} \mathbf{W}_K^\top = [\mathbf{K}_1; \mathbf{K}_2; \dots; \mathbf{K}_G] \quad \mathbf{V} = \mathbf{u} \mathbf{W}_V^\top = [\mathbf{V}_1; \mathbf{V}_2; \dots; \mathbf{V}_G] \end{aligned} \quad (2.16)$$

where $\mathbf{W}_Q \in \mathbb{R}^{cd \times d}$ and $\mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{Gd_h \times d}$.

The speedup of MQA trades off quality, but GQA retains much of MHA’s quality while still being close to MQA’s in terms of speed, becoming the standard in any attention-based module. Sharing the KV projections not only across the heads but also across the layers further reduces the KV cache size. Brandon et al. 2024 claim that reusing the same KV projections from the bottom layers in the top layers or sharing between adjacent layers retains nearly the same accuracy with a $2\times$ reduction of the KV cache size. The Gemma 4 family of models (Gemma Team 2026) adopts this idea: smaller models share the KV cache across layers, the larger ones reuse the keys as values inside the global attention layers.



Grouped Query Attention

Figure 2.4: Grouped query attention with $G=2$ KV groups and $H=4$ query heads. Each group of H/G heads shares a single key-value projection of d_h dimensions, reducing KV cache by a factor of H/G while retaining more expressiveness than MQA. The depth axis represents the sequence (time) dimension.

2.2.5 Attention Module Optimizations

Since the release of the V2 model (DeepSeek-AI 2024), DeepSeek’s main focus was dedicated to further push the limits of attention computational and memory efficiency. Multihead Latent Attention (MLA) is one innovation that reduces the KV cache size by over 90%. Their idea proposes compressing the key and value entries into a single latent representation of much smaller dimension, and reconstructing the keys and values at runtime layerwise.

Thus, for any token at time t , at every transformer layer l the cache stores a single

vector $c_t^{KV} \in \mathbb{R}^{d_c}$ ($d_c \ll n_h d_h$):

$$c_t^{KV} = W_{\downarrow}^{KV} h_t, \quad W_{\downarrow}^{KV} \in \mathbb{R}^{d_c \times d}, \quad (2.17)$$

$$k_t^C = W_{\uparrow}^K c_t^{KV}, \quad v_t^C = W_{\uparrow}^V c_t^{KV}, \quad (2.18)$$

with $d_c \ll n_h d_h$ (in V2, $d_c = 4d_h$). Only c_t^{KV} is cached; the per-head keys and values k_t^C, v_t^C are reconstructed on demand by $W_{\uparrow}^K, W_{\uparrow}^V \in \mathbb{R}^{n_h d_h \times d_c}$.

MLA comes up with a handful of other particular shifts from the classic attention module that do not reside in the memory win category. Similarly to the keys and values, the queries are low-rank compressed as well,

$$c_t^Q = W_{\downarrow}^Q h_t, \quad q_t = W_{\uparrow}^Q c_t^Q \quad (d'_c = 1536 \text{ in V2}), \quad (2.19)$$

though this one does nothing for the KV cache (the query is never cached) and serves only to shrink the activation memory during training: only the compressed c_t^Q, c_t^{KV} are detained as activations for the backward pass, while their decompressed parts q_t, k_t, v_t are recomputed layerwise during backprop, in the gradient checkpointing manner.

The up-projections $W_{\uparrow}^K, W_{\uparrow}^V$ never slow down decoding due to a simple associativity trick. For a query at position s and a cached token t , instead of decompressing the key first and then taking the dot product,

$$\text{score}_{s,t} = q_s^\top (W_{\uparrow}^K c_t^{KV}), \quad (2.20)$$

we reorder the multiplication so the up-projection lands on the query side:

$$\text{score}_{s,t} = (q_s^\top W_{\uparrow}^K) c_t^{KV} \quad (2.21)$$

Now the cached latent c_t^{KV} enters the dot product directly from the cache without reconstructing the key. And since $q_s = W_{\uparrow}^Q c_s^Q$, both up-projections collapse into a single constant matrix $W_{\uparrow}^{K^\top} W_{\uparrow}^Q$ that can be precomputed once. The value path is treated identically, with $W_{\uparrow}^V$ absorbed into the output projection W^O .

This reordering only holds as long as there are no other operations wedged in between, such as RoPE or QK-normalization layers. MLA avoids this by decoupling RoPE: it adds a small set of extra dimensions (of size $d_h^R = d_h/2$, derived by a new pair of weights, W^{QR} from the query latent c_t^Q and W^{KR} from the hidden state h_t) to the query and key, dedicated to positional information, with the rotary key k_t^R shared across all heads ($q_{s,i} = [q_{s,i}^C; q_{s,i}^R]$ and $k_{t,i} = [k_{t,i}^C; k_{t,i}^R]$). A dot product over a concatenation splits additively, $[a; b] \cdot [c; d] = a \cdot c + b \cdot d$, so the score separates into a content and a rotary term:

$$q_{s,i}^\top k_{t,i} = \underbrace{q_{s,i}^C \top k_{t,i}^C}_{\text{absorbed, position-free}} + \underbrace{q_{s,i}^R \top k_t^R}_{\text{small raw rotary channel}}. \quad (2.22)$$

The content term (C) stays compressed as above, while the rotary term (R) is separately computed on the small uncompressed channel. The KV cache therefore holds both the content and the rotary parts. In total, DeepSeek-V2 reports a 93.3% KV cache reduction and a $5.76\times$ higher generation throughput than DeepSeek-67B, while matching or even surpassing the quality of full MHA.

In their subsequent research (Yuan et al. 2025, DeepSeek-AI 2026a), DeepSeek combines two ideas: blockwise KV cache compression (packing n KV entries into one) for memory savings, and selective attention for faster compute - a fast indexer that retrieves only a fixed number of these compressed entries and attends over just those and the recent uncompressed ones. They also interleave layers of extreme KV compression (attended densely, without sparse selection) with the ordinary ones, combining a sparse but high-resolution retrieval of selected tokens with a full, if coarse, view of the entire history.

2.2.6 Linear Attention

In the original attention formulation, bounding the score values with the softmax non-linearity was a necessity to stabilize the cumulative sums that happen during matrix multiplications in forward and backward passes, but it was never a requirement for attention to work. Katharopoulos et al. 2020 observed that removing the softmax function allows factoring out the query matrix and maintaining a constant state memory, similar to recurrent neural networks. To understand this, let’s revise the original bidirectional attention formulation in its linear form:

$$\text{LinearAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = (\mathbf{Q}\mathbf{K}^\top) \mathbf{V} = \mathbf{Q} (\mathbf{K}^\top \mathbf{V}) \quad (2.23)$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times d_h}$. As you can see, the normalizer (division by the sum of all scores) is skipped for simplicity.

Associativity allows us to multiply $\mathbf{K}^\top$ and $\mathbf{V}$ first, then multiply with $\mathbf{Q}$, avoiding any matrix result of shape $T \times T$, thus reducing the memory and computational complexity from $O(T^2)$ to $O(T)$. Though, for autoregressive decoding, the causal masking prevents this simple matrix multiplication trick. To visualize this, let’s define how we compute the attention output in a linear model for a token i :

$$\mathbf{o}_i = \sum_{j \leq i} (\mathbf{q}_i^\top \mathbf{k}_j) \mathbf{v}_j = \left(\sum_{j \leq i} \mathbf{k}_j \mathbf{v}_j^\top \right)^\top \mathbf{q}_i \quad (2.24)$$

Let's denote the state matrix $\mathbf{S}_1 \in \mathbb{R}^{d_h \times d_h}$ as the outer product between $\mathbf{k}_1$ and $\mathbf{v}_1$. For the following tokens, the state is updated by adding up the outer product of the next key and value vectors (also known as Hebbian update). The sequence of output vectors is then computed between the query vector and the corresponding state matrix:

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \mathbf{k}_i \mathbf{v}_i^\top, \quad \mathbf{o}_i = \mathbf{S}_i^\top \mathbf{q}_i, \quad \mathbf{S}_0 = \mathbf{0} \quad (2.25)$$

Visualizing linear attention in this sequential mode, we can see the similarity with recurrent networks: a memory buffer that never grows in size and carries over information from the past tokens to the future ones.

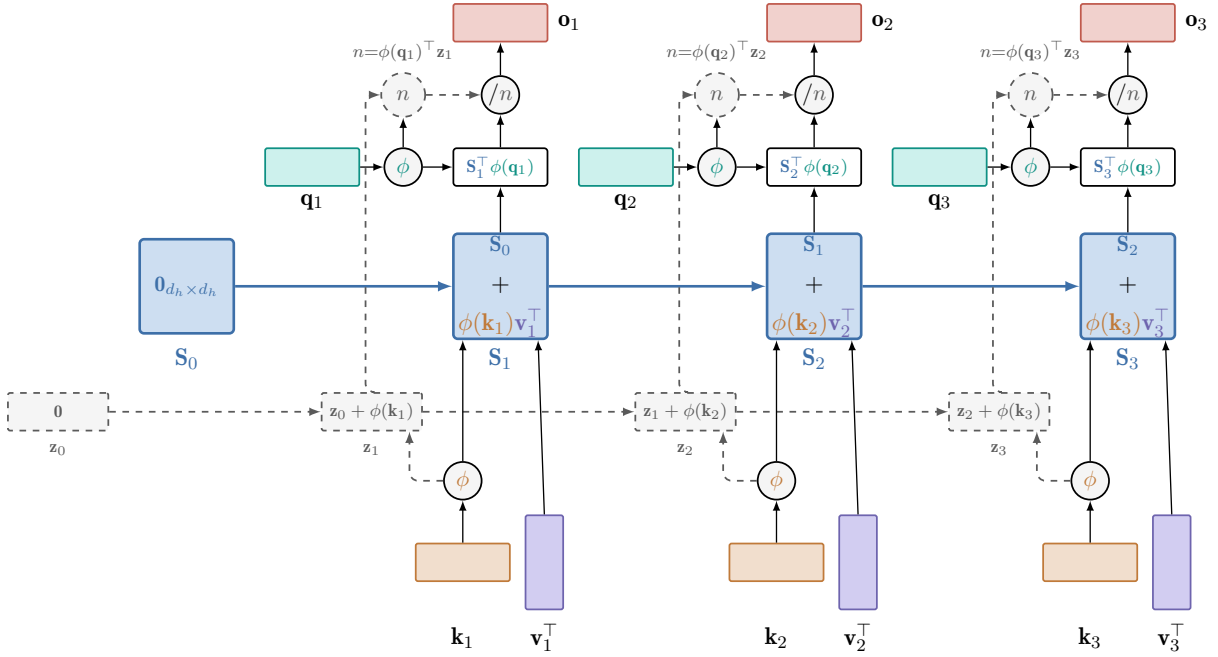


Figure 2.5: In causal linear attention (here $T = 3$), the hidden output on a specific position in the sequence $\mathbf{o}_i$ is determined only by its input and a fixed square $d_h \times d_h$ matrix acting as the carryover channel. At each timestep, the state is updated by the outer product of input-dependent key and value values, here acting as a write that directly influences the fast-weight (Geva et al. 2021) state. Within this setup, we view the state as a running memory of the past in superposition and the query as the selective retrieval vector from the past. The representation is per attention head: each head maintains its own $d_h \times d_h$ state.

To maintain the "attention behaviour" (i.e. "how much one token attends to another"), they rectify the query and key entries so that every similarity score stays positive; the scores are then normalized by their sum, keeping the output a convex combination of the values:

$$\phi(x) = \begin{cases} x + 1, & x \geq 0 \\ \alpha(e^x - 1) + 1 \stackrel{\alpha=1}{=} e^x, & x < 0 \end{cases} \quad (2.26)$$

applied elementwise to every entry of $\mathbf{q}$ and $\mathbf{k}$ - strictly positive everywhere, roughly linear for positive inputs, and smoothly decaying (never zero) for negative ones. The normalizer itself stays factorized: a second accumulator $\mathbf{z}_i = \sum_{j \leq i} \phi(\mathbf{k}_j)$ runs alongside the state, and the output becomes $\mathbf{o}_i = \mathbf{S}_i^\top \phi(\mathbf{q}_i) / \phi(\mathbf{q}_i)^\top \mathbf{z}_i$, - $\phi(\mathbf{q}_i)^\top \mathbf{z}_i \in \mathbb{R}$.

Plain accumulation, however, never forgets: the state only ever grows richer and cannot edit or remove what it has stored, which is where linear attention loses to softmax on precise retrieval. Augmenting this recurrence with a forgetting gate and a targeted update rule yields the *Gated DeltaNet* model treated in Section 2.3.4.

2.3 Linear Recurrent Models

A *State Space Model* broadly refers to any recurrent process with a latent hidden state. It is applied across various disciplines to describe different concepts, starting from RNNs and ConvNets, to Markov Decision Processes (MDPs) and Kalman Filters (R. E. Kalman 1960).

The classic recurrent architectures LSTM (Hochreiter and Schmidhuber 1997) and GRU (Chung et al. 2014) had driven significant progress in many sequence related tasks, NLP (Sutskever 2014) or speech processing (Graves, Mohamed, and G. Hinton 2013). and they also laid the groundwork for the initial implementation of attention mechanisms (Bahdanau 2014).

2.3.1 Structured State Space Models (S4)

Structured SSMs (S4) (Gu, Goel, and Ré 2022) are sequence-to-sequence mapping systems $x(t) \in \mathbb{R} \mapsto y(t) \in \mathbb{R}$ through an implicit hidden state $h(t) \in \mathbb{R}^d$. S4 consists of parameters $(\Delta \in \mathbb{R}^d, \mathbf{A} \in \mathbb{R}^{d \times d_{state}}, \mathbf{B} \in \mathbb{R}^{d \times d_{state}}, \mathbf{C} \in \mathbb{R}^{d \times d_{state}})$, where d is the embedding dimension and d_{state} determines the latent state dimension. The forward pass begins with discretization of parameters $(\mathbf{A}, \mathbf{B}) \mapsto (\bar{\mathbf{A}}, \bar{\mathbf{B}})$ by step Δ via zero-order hold (ZOH) discretization:

$$\Delta \leftarrow \tau_\Delta(\Delta) \quad (2.27)$$

$$\bar{\mathbf{A}} \leftarrow \exp(\Delta \mathbf{A}) \quad (2.28)$$

$$\bar{\mathbf{B}} \leftarrow (\Delta \mathbf{A})^{-1}(\exp(\Delta \mathbf{A}) - I) \cdot \Delta \mathbf{B} \quad (2.29)$$

where τ_Δ is softplus (Bridle 1990) or any smooth ReLU. Note that the d channels are independent SSMs and $\mathbf{A}$ is *diagonal*: its i -th row $\mathbf{a}_i \in \mathbb{R}^{d_{state}}$ holds the diagonal entries of channel i 's transition matrix $\mathbf{A}_i = \text{diag}(\mathbf{a}_i) \in \mathbb{R}^{d_{state} \times d_{state}}$ (which is why $\mathbf{A}$ is stored as $d \times d_{state}$ rather than $d \times d_{state} \times d_{state}$). The step $\Delta_i \mathbf{A}_i$ stays diagonal, so the matrix exponential collapses to an elementwise one,

$$\bar{\mathbf{A}}_i = \exp(\Delta_i \mathbf{A}_i) = \text{diag}(\exp(\Delta_i \mathbf{a}_i)), \quad (2.30)$$

so $\bar{\mathbf{A}}$ is itself diagonal, and $\bar{\mathbf{A}}\mathbf{h}_{t-1}$ reduces to a Hadamard product $\exp(\Delta\mathbf{a}) \odot \mathbf{h}_{t-1}$ - the d_{state} state coordinates evolve independently and never mix. The SSM computes the hidden state and the output recursively as follows:

$$\mathbf{h}_t \leftarrow \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}\mathbf{u}_t \quad \mathbf{u}'_t \leftarrow \mathbf{C}\mathbf{h}_t \quad (2.31)$$

We observe that an SSM is similar with an RNN except it replaces the nonlinearity with another linear transformation, that can be computed recursively in $O(T)$, but the scalability strengths stays in the parallelization of this operation, allowed by parameters that stay constant through time, by rewriting it as a global convolution that leverages the Fast Fourier Transform (FFT) as primitive, running in parallel in $O(T \log T)$ time. To understand how, we show that any $\mathbf{u}'_k$ can be computed as a dot product between a pre-computed vector consisting of the actual constant through-time parameters and the input $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k$:

$$\begin{aligned} \mathbf{u}'_k &= \mathbf{C}(\bar{\mathbf{A}}\mathbf{h}_k + \bar{\mathbf{B}}\mathbf{u}_k) \\ &= \mathbf{C}(\bar{\mathbf{A}}(\bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}\mathbf{u}_{k-1}) + \bar{\mathbf{B}}\mathbf{u}_k) \\ &= \mathbf{C}(\bar{\mathbf{A}}(\bar{\mathbf{A}}(\bar{\mathbf{A}}\mathbf{h}_{k-2} + \bar{\mathbf{B}}\mathbf{u}_{k-2}) + \bar{\mathbf{B}}\mathbf{u}_{k-1}) + \bar{\mathbf{B}}\mathbf{u}_k) \\ &= \dots \end{aligned} \quad (2.32)$$

This can be written as a dot product:

$$\mathbf{u}'_k = \begin{pmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{u}_{k-1} \\ \mathbf{u}_k \end{pmatrix} \cdot \begin{pmatrix} \mathbf{C} \overbrace{\mathbf{A} \mathbf{A} \mathbf{A} \dots \mathbf{A}}^{k-1} \bar{\mathbf{B}} \\ \mathbf{C} \overbrace{\mathbf{A} \mathbf{A} \dots \mathbf{A}}^{k-2} \bar{\mathbf{B}} \\ \vdots \\ \mathbf{C} \bar{\mathbf{A}} \bar{\mathbf{B}} \\ \mathbf{C} \bar{\mathbf{B}} \end{pmatrix} \quad (2.33)$$

The entire output sequence $\mathbf{u}' = \mathbf{u}'_1, \mathbf{u}'_2, \dots, \mathbf{u}'_k$ can be computed as a 1D convolution:

$$\mathbf{u}' \leftarrow \mathbf{u} * \bar{\mathbf{K}}, \text{ where } \bar{\mathbf{K}}, \text{ the second vector in Eq. 2.33 is precomputed.} \quad (2.34)$$

Algorithm 2 S4. For simplicity $\Delta_{rank} = d$, but in actual implementations $\Delta_{rank} = d/c, c \in \mathbb{N}^*, d/c \in \mathbb{N}$.

Input: $\mathbf{u} \in \mathbb{R}^{B \times T \times d}$

Output: $\mathbf{u}' \in \mathbb{R}^{B \times T \times d}$

Require: $\Delta \in \mathbb{R}^{\Delta_{rank}}, (\mathbf{A}, \mathbf{B}, \mathbf{C}) \in \mathbb{R}^{d \times d_{state}}$

1: $\bar{\mathbf{A}}, \bar{\mathbf{B}} \in \mathbb{R}^{B \times T \times \Delta_{rank} \times d_{state}} \leftarrow \text{ZOH}(\Delta', \mathbf{A}, \mathbf{B})$

2: $\mathbf{u}' \leftarrow \text{SSM}(\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}, \mathbf{u})$

▷ recurrence or convolution

3: **return** $\mathbf{u}'$

2.3.2 Selective State Space Models (S6)

The S4 has *linear time invariance*, i.e. the parameters remain fixed throughout the timesteps (as in RNNs and ConvNets). Comparing to its update introduced in the Mamba architecture (Gu and Dao 2023), despite its success in solving the Copying task (that involves constant spacing between the input and the output), it fails in the Selective Copying task, which has random spacing in between inputs. In order to solve it, they introduced time-varying parameters, i.e. a dynamic gating mechanism that remembers and ignores features of the embeddings of the input sequence. The selection works by converting the parameters into *time-varying parameters* - generate a different set of pairs $(\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C})_t$ for each timestep t that is input dependent.

To understand the similarity with GRU, let us consider an SSM in which $d\epsilon = 1$, $\mathbf{A} = -1$, $\mathbf{B} = 1$, and $\tau_\Delta = \log(1 + \exp \mathbf{u})$ as nonlinearity. In this case, following Eq. 2.31, the continuous (no discretized parameters) SSM will be to:

$$\mathbf{h}_t \leftarrow -\mathbf{h}_t + \mathbf{u}_t \quad (2.35)$$

The zero-order hold discretization will be computed as:

$$\Delta_t = \tau_\Delta(\Delta + \mathbf{u}_t \mathbf{W}_\Delta^\top) \quad (2.36)$$

$$\begin{aligned} \bar{\mathbf{A}}_t &= \exp(\Delta_t \mathbf{A}) \\ &= \frac{1}{1 + \exp(\mathbf{u}_t \mathbf{W}_\Delta^\top)} \\ &= \sigma(-\mathbf{u}_t \mathbf{W}_\Delta^\top) \\ &= 1 - \sigma(\mathbf{u}_t \mathbf{W}_\Delta^\top) \end{aligned} \quad (2.37)$$

$$\begin{aligned}
\bar{\mathbf{B}}_t &= (\Delta_t \mathbf{A})^{-1} (\exp(\Delta_t \mathbf{A}) - I) \cdot \Delta_t \mathbf{B} \\
&= -(\exp(\Delta_t \mathbf{A}) - I) \\
&= 1 - \bar{\mathbf{A}} \\
&= \sigma(\mathbf{u}_t \mathbf{W}_\Delta^\top)
\end{aligned} \tag{2.38}$$

Now it is clear that $\bar{\mathbf{A}}$ and $\bar{\mathbf{B}}$ represent the update gate in Eq. 2.31, which is similar to the GRU hidden state formula $\mathbf{h}_t = (1 - z_t)\mathbf{h}_{t-1} + z_t\mathbf{u}_t$, where the gate z_t in this case is derived only from the input.

Algorithm 3 S6. In the Mamba block, the input $\mathbf{u}$ with embedding dimension d is expanded by ϵ , so the inner dimension becomes $d\epsilon$. The authors implementation is simplified comparing to the algorithm definition in the paper in terms of parameters dimension: default $\epsilon = 2$, the latent space dim $d_{state} = 16$, and the dimension of Δ is $\Delta_{rank} = d/16$.

Input: $\mathbf{u} \in \mathbb{R}^{B \times T \times d\epsilon}$

Output: $\mathbf{u}' \in \mathbb{R}^{B \times T \times d\epsilon}$

Require: $\Delta \in \mathbb{R}^{\Delta_{rank}}, \mathbf{W}_\Delta \in \mathbb{R}^{\Delta_{rank} \times d\epsilon}$

Require: $\mathbf{A} \in \mathbb{R}^{d\epsilon \times d_{state}}$

Require: $\mathbf{B} \in \mathbb{R}^{d \times d_{state}}$

Require: $\mathbf{C} \in \mathbb{R}^{d \times d_{state}}$

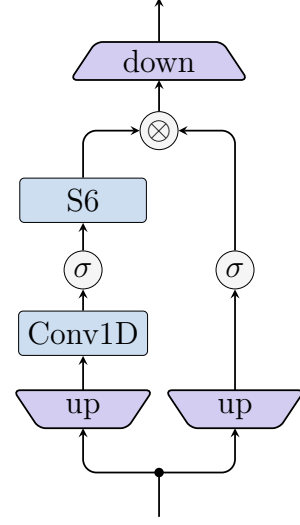
- 1: $\mathbf{B}' \in \mathbb{R}^{B \times T \times d_{state}} \leftarrow \mathbf{uB}$
 - 2: $\mathbf{C}' \in \mathbb{R}^{B \times T \times d_{state}} \leftarrow \mathbf{uC}$
 - 3: $\Delta' \in \mathbb{R}^{B \times T \times \Delta_{rank}} \leftarrow \tau_\Delta(\Delta_{broadcast} + \mathbf{uW}_\Delta^\top)$
 - 4: $\bar{\mathbf{A}}, \bar{\mathbf{B}} \in \mathbb{R}^{B \times T \times \Delta_{rank} \times d_{state}} \leftarrow \text{ZOH}(\Delta', \mathbf{A}, \mathbf{B})$
 - 5: $\mathbf{u}' \leftarrow \text{SSM}(\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}, \mathbf{u})$ $\triangleright$ recurrence (scan)
 - 6: **return** $\mathbf{u}'$
-

Computing the final output $\mathbf{u}'$ is no longer possible with global convolution because of the time-varying parameters (expanded parameters for each element in the sequence), but it can still be computed efficiently in parallel with associative scan (see Section 2.3.3). This works because $\bar{\mathbf{A}}_t$ is diagonal (Section 2.3.1): the recurrence of Eq. 2.31 then decouples into d_{state} independent scalar updates $\mathbf{h}_t = \bar{\mathbf{a}}_t \odot \mathbf{h}_{t-1} + \bar{\mathbf{b}}_t$, exactly the $\mathbf{h}_t = \mathbf{a}_t \cdot \mathbf{h}_{t-1} + \mathbf{b}_t$ form the associative scan parallelizes. A Mamba Block projects the input to a higher dimensionality by an expansion factor $\epsilon = 2$ before passing it through the S6, and reduces it back from $d\epsilon$ to d after a gate is induced.

Algorithm 4 Mamba (1) Block

Input: $\mathbf{u} \in \mathbb{R}^{B \times T \times d}$ **Output:** $\mathbf{u}' \in \mathbb{R}^{B \times T \times d}$ **Require:** $\mathbf{W}_\uparrow, \mathbf{W}_{gate} \in \mathbb{R}^{d\epsilon \times d}$, $\mathbf{W}_\downarrow \in \mathbb{R}^{d \times d\epsilon}$ **Require:** Conv1D & S6**Require:** $\epsilon = 2$, expansion factor

- 1: $\mathbf{u}_{gate} \in \mathbb{R}^{B \times T \times d\epsilon} \leftarrow \mathbf{u} \mathbf{W}_{gate}^\top$
 - 2: $\mathbf{u} \in \mathbb{R}^{B \times T \times d\epsilon} \leftarrow \mathbf{u} \mathbf{W}_\uparrow^\top$
 - 3: $\mathbf{u} \leftarrow \text{Swish}(\text{Conv1D}(\mathbf{u}))$
 - 4: $\mathbf{u} \leftarrow \text{S6}(\mathbf{u})$
 - 5: $\mathbf{u} \leftarrow \mathbf{u} \odot \text{Swish}(\mathbf{u}_{gate})$
 - 6: $\mathbf{u}' \leftarrow \mathbf{u} \mathbf{W}_\downarrow^\top \quad \triangleright d\epsilon \rightarrow d$
 - 7: **return** $\mathbf{u}'$
-



The linear layers project the input $\mathbf{u}$ into a higher (multiple) dimension $d\epsilon$. In the authors implementation³, the Convolution layer has channels $c_{in} = c_{out} = d\epsilon$, $f_d = 4$, groups = $d\epsilon$, and padding = $f_{dim} - 1$ to retain the dimensionality. The model is constructed by stacking up multiple Mamba Blocks wrapped within residual blocks with pre-normalization.

Algorithm 5 Residual Block

Input: $\mathbf{u} \in \mathbb{R}^{B \times T \times d}$ **Output:** $\mathbf{u}' \in \mathbb{R}^{B \times T \times d}$ **Require:** MambaBlock

- 1: $\mathbf{u}_{residual} \leftarrow \text{LayerNorm}(\mathbf{u}) \quad \triangleright \text{RMSNorm}$
 - 2: $\mathbf{u}_{residual} \leftarrow \text{MambaBlock}(\mathbf{u})$
 - 3: $\mathbf{u}' \leftarrow \mathbf{u}_{residual} + \mathbf{u}$
 - 4: **return** $\mathbf{u}'$
-

Further research (Dao and Gu 2024) for more efficient large-scale training and inference was done, trading the scan for a chunkwise form whose in-chunk terms are plain matrix products (Section 2.3.4), with a focus on Tensor Parallelism (TP) (Shoeybi et al. 2019) and Sequence Parallelism (SP) (Korthikanti et al. 2023). At first, Mamba (2) has multiple parallel groups g that process the input separately, similar to Multihead Attention (MHA) in Transformers, and the outputs of all heads pass through an all-reduce layer ($\mathbf{u}' \leftarrow \sum_g \mathbf{u}'_g$). ($\mathbf{W}_\uparrow, \mathbf{W}_{gate} \in \mathbb{R}^{d\epsilon/g \times d}$, $\mathbf{W}_\downarrow \in \mathbb{R}^{d \times d\epsilon/g}$). The block design itself received some modifications - SSM parameters ($\mathbf{A}, \mathbf{B}, \mathbf{C}$) are produced at the beginning of the block instead of as a function of the SSM input and an additional GroupNorm (GN) (Wu and He 2018) layer.

³https://github.com/state-spaces/mamba/blob/main/mamba_ssm/modules/mamba2.py

When working with a larger number of GPUs, the input can be split along the sequence (T) dimension. For example GPU-0 that computes the first T/gpu_{num} outputs $\mathbf{u}'$, can pass the hidden states to GPU-1. Using the hidden states of the previous tokens in the sequence and the next T/gpu_{num} tokens as inputs, GPU-1 computes the next output $\mathbf{u}'$ of the same length, and its hidden states are passed further.

2.3.3 Parallel Associative Scan

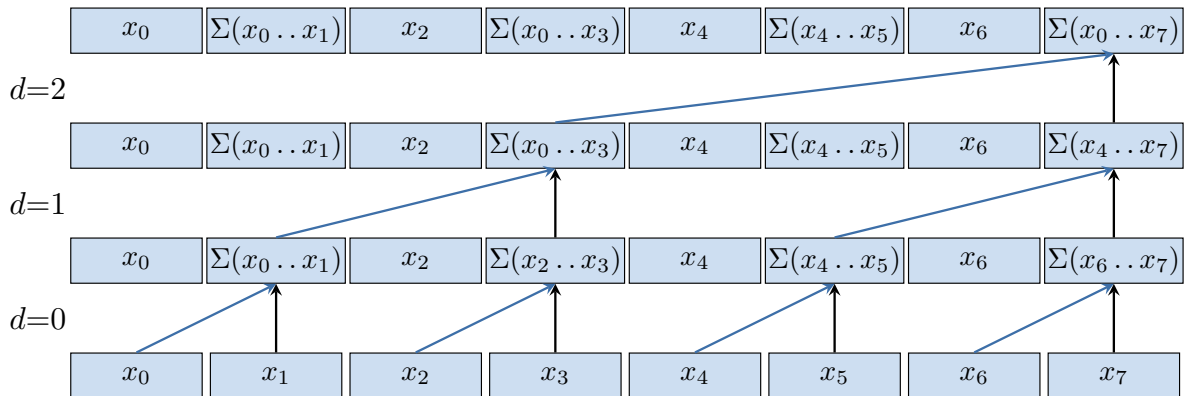
A parallel log depth implementation for the CumSum (Prefix sum) function was firstly introduced in Blelloch 1990 that lie at the foundation of linear RNNs efficiency (Martin and Cundy 2018), more precisely, the algorithm can be generally applied for a family of functions of form $\mathbf{v}_t = \mathbf{a}_t \oplus \mathbf{v}_{t-1} + \mathbf{b}_t$, where $\oplus$ is any associative operator, $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n$ and $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n$ are the inputs, and $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$ the outputs.

The naive implementation of the cumulative sum operation $\mathbf{s}_t = \mathbf{v}_{t-1} + \mathbf{v}_t$ takes n steps of computation, though using multiple workers will drop it down to a depth of $\log n$ steps, the total work staying linear in n across the two sweeps. The idea is to compute in a form of balanced binary tree with n leaves and a depth of $\log n$, where n is a power of 2 (for arbitrary sizes the vector is padded). The algorithm is split into two phases. The *up-sweep* phase computes partial sums on even indexes, and on the indexes that are powers of two the complete for it's index:

Algorithm 6 Up-sweep

Input: $v \in \mathbb{R}^n$

- 1: **for** $d = 0 \rightarrow \log_2(n) - 1$ **do**
 - 2: **for** $w_0, w_1, \dots, w_{\frac{n-1}{2^{d+1}}}$ **in parallel do**
 - 3: $i \leftarrow w_{index} \cdot 2^{d+1}$
 - 4: $v_{i+2^{d+1}-1} \leftarrow v_{i+2^d-1} \oplus v_{i+2^{d+1}-1}$
 - 5: **end for**
 - 6: **end for**
-



The *down-sweep* phase sets the last element (the root of the tree) to 0 and traverses the tree from root to leaves to build the scan in the vector using the partials sums from the previous phase. In each step d , each node passes its own value to its left child, and the sum of its value and the former value of its left child to its right child.

Algorithm 7 Down-sweep

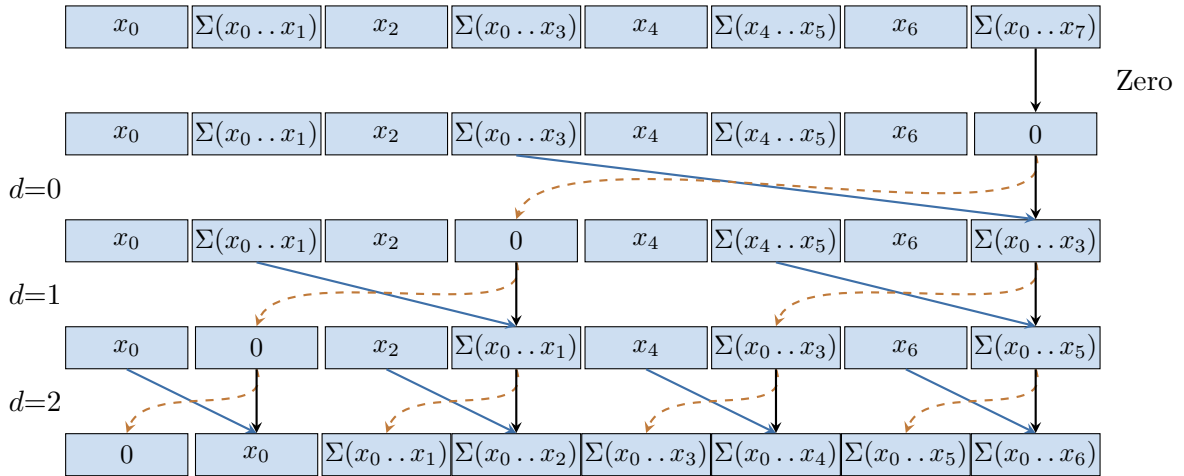
Input: $v \in \mathbb{R}^n$ (up-phase result)

Output: $v \in \mathbb{R}^n$ (exclusive scan)

```

1:  $v_{n-1} \leftarrow 0$ 
2: for  $d = \log_2(n) - 1 \rightarrow 0$  do
3:   for  $w_0, w_1, \dots, w_{\frac{n-1}{2^{d+1}}}$  in parallel do
4:      $i \leftarrow w_{index} \cdot 2^{d+1}$ 
5:      $t \leftarrow v_{i+2^d-1}$ 
6:      $v_{i+2^d-1} \leftarrow v_{i+2^{d+1}-1}$ 
7:      $v_{i+2^{d+1}-1} \leftarrow t \oplus v_{i+2^{d+1}-1}$ 
8:   end for
9: end for
10: return  $v$ 

```



Note that the result of the down-sweep is an exclusive scan, i.e. the vector is shifted to the right. Shifting it to the left and inserting at the end the sum of the last element of the exclusive scan and the last element in the initial array results in an inclusive scan. The presentation of the base prefix sum algorithm, including the illustrations were inspired from an official NVIDIA documentation⁴ explained after Blelloch 1990, where they explain hardware aware implementations. Now let's apply the algorithm (cumsum and cumprod) to compute the hidden states for the functions $\mathbf{h}_t = \mathbf{a}_t \cdot \mathbf{h}_{t-1} + \mathbf{b}_t, \mathbf{h}_0 \leftarrow \mathbf{b}_0$. Following Heinsen 2023, we leverage the complex plane to compute the operations:

⁴<https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-39-parallel-prefix-sum-scan-cuda>

Algorithm 8 Parallel Scan (log-space)

Input: $\mathbf{a}_{1:T} \in \mathbb{R}^{B \times T \times d}$, $\mathbf{b}_{0:T} \in \mathbb{R}^{B \times (T+1) \times d}$

Output: $\mathbf{h} \in \mathbb{R}^{B \times T \times d}$

- 1: $\log \mathbf{a} \in \mathbb{C}^{B \times T \times d} \leftarrow \text{ComplexLog}(\mathbf{a})$
 - 2: $\log \mathbf{b} \in \mathbb{C}^{B \times (T+1) \times d} \leftarrow \text{ComplexLog}(\mathbf{b})$
 - 3: $\mathbf{a}^* \in \mathbb{C}^{B \times T \times d} \leftarrow \text{PrefixSum}(\mathbf{a})$ $\triangleright$ on time axis T
 - 4: $\mathbf{a}^* \in \mathbb{C}^{B \times (T+1) \times d} \leftarrow \text{ShiftToRight}(\mathbf{a}^*)$ $\triangleright$ on time axis T
 - 5: $(\log \mathbf{x}_0 + \mathbf{b}^*) \leftarrow \log \text{PrefixSum}(\exp(\log \mathbf{b} - \mathbf{a}^*))$
 - 6: $\log \mathbf{x} \in \mathbb{R}^{B \times (T+1) \times d} \leftarrow \text{Re}(\mathbf{a}^* + (\log \mathbf{x}_0 + \mathbf{b}^*))$
 - 7: **return** $\exp(\log \mathbf{x})_{1:T}$
-

where for any $x \in \mathbb{R}^d$, we have

$$\text{ComplexLog}(x) = \log \max(|x|, \epsilon) + i \arg(x), \quad \arg(x) = \begin{cases} 0 & \text{if } x \geq 0, \\ \pi & \text{if } x < 0. \end{cases} \quad (2.39)$$

2.3.4 Gated DeltaNet

In its initial form, linear attention (Section 2.2.6) falls short comparing to state space models. The cumulative design of the state limits its potential, in a similar manner normal RNNs lack the forget ability of LSTMs (Schmidhuber 1992). The *delta rule* (Schlag, Irie, and Schmidhuber 2021) treats the state update as a correction rather than an accumulation, determining the new state as a forget-update operation over the previous one:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \quad (2.40)$$

where $\beta_t = \sigma(\mathbf{w}_\beta^\top \mathbf{h}_t) \in (0, 1)$ is a knob between the forgetting and updating tradeoff ($\mathbf{w}_\beta$ is learnable). We can view the next state $\mathbf{S}_t$ as a combination of three parts:

$$\mathbf{S}_t = \underbrace{\mathbf{S}_{t-1} \mathbf{I}}_{\text{previous intact memory}} - \underbrace{\beta_t \mathbf{S}_{t-1} \mathbf{k}_t \mathbf{k}_t^\top}_{\text{erase the old value stored at } \mathbf{k}_t} + \underbrace{\beta_t \mathbf{v}_t \mathbf{k}_t^\top}_{\text{write the new value } \mathbf{v}_t \text{ at } \mathbf{k}_t} \quad (2.41)$$

On top of this formulation, Yang, Kautz, and Hatamizadeh 2024 adds an input-dependent forgetting gate $\alpha_t \in (0, 1)$ that multiplicatively decays the whole state, while leaving the chunkwise-parallel computation intact, its scalar decay accumulating into a cumulative product that factors out of the in-chunk matrix products, outperforming Mamba-2 on long context benchmarks:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\alpha_t(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \quad (2.42)$$

where $\alpha_t = \sigma(\mathbf{w}_\alpha^\top \mathbf{h}_t) \in (0, 1)$ is the input-dependent memory decay (acting as a dynamic history-shrinking/forgetting mechanism) and $\beta_t = \sigma(\mathbf{w}_\beta^\top \mathbf{h}_t) \in (0, 1)$ the write strength, both derived from the hidden state $\mathbf{h}_t$. The rank-one factor $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)$ erases whatever the memory stores along the current key direction $\mathbf{k}_t$ before $\beta_t \mathbf{v}_t \mathbf{k}_t^\top$ is written in its place (a targeted read-modify-write); α_t shrinks the state uniformly; setting $\alpha_t = 1$ recovers the plain delta rule above. The output is then read from the state by the query, $\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$. The gate here is a single scalar per step; turning it into one decay per key channel is the whole of *Kimi Delta Attention* (Kimi Team 2025), which forgets with finer granularity but pays for it in the chunkwise form, where a per-channel cumulative decay no longer factors out of the in-chunk matrix product.

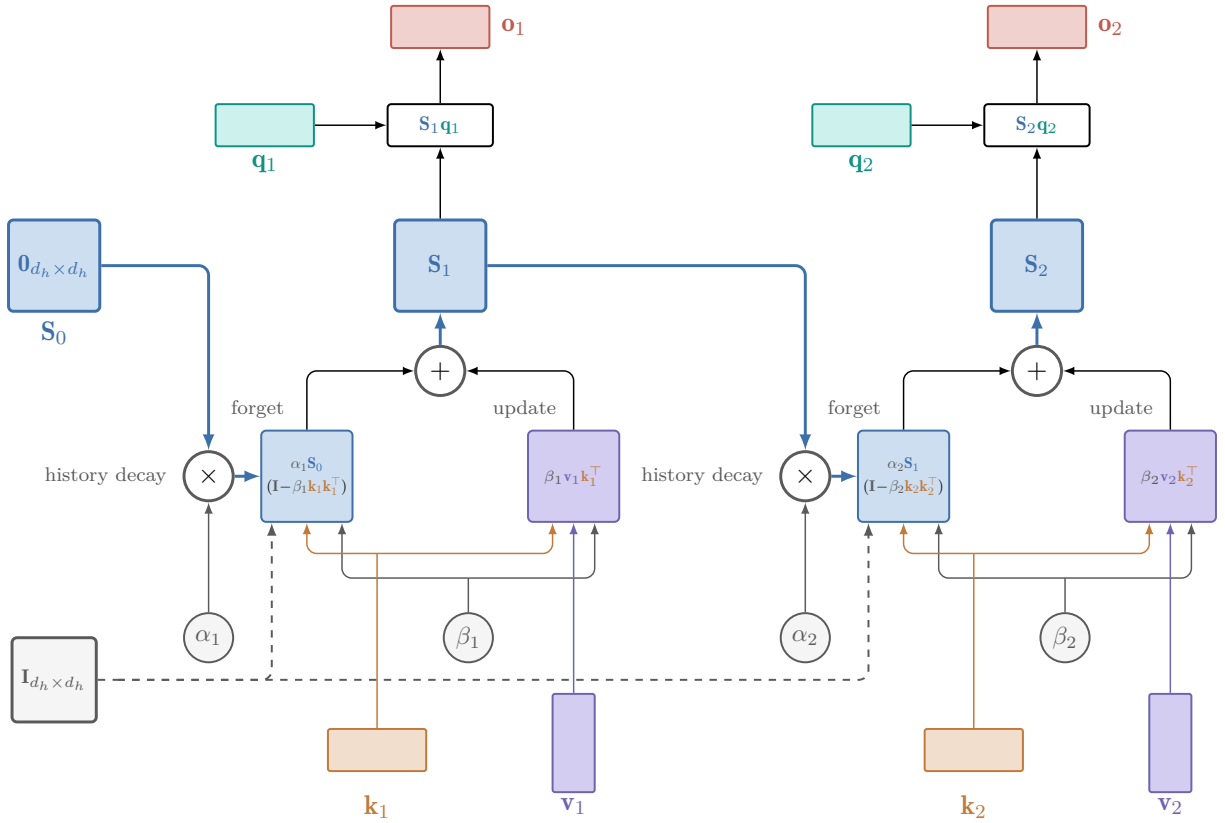


Figure 2.6: The sequential view of Gated DeltaNet (unrolled for $T = 2$). From the input hidden state we derive $\alpha_t \in (0, 1)$ (decay applied to the previous state), $\beta_t \in (0, 1)$ (write strength, interpolating between the stored memory and the new value), $\mathbf{q}_t$ (query reading the state), $\mathbf{k}_t$ (key addressing memory) and $\mathbf{v}_t$ (value written to it); the output is $\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$ (the state read out by the query). As in linear attention, the representation is per head: each head maintains its own state $\mathbf{S}_t$.

Linear attention/recurrent models pack in their block a short, causal 1D convolution layer (see Section 2.3.2). The state represents a fuzzy recollection of the whole past, so precise local contextualization is rather difficult. The role of the causal convolution is to give a recency bias to the next token prediction at a negligible compute and cache cost.

While the recurrence unfolds the logic of the operations, training requires a parallel definition. Compared to SSMs, whose diagonal transition $\bar{\mathbf{A}}_t$ (Section 2.3.2) turns the recurrence into a cumulative sum handled by an associative scan (Section 2.3.3), the delta transition $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)$ is a dense matrix and does not; unrolling it yields a product of such matrices. Yang, B. Wang, et al. 2024 make this parallel by rewriting the update as a *correction* of the stored value, an idea easily grasped through a simple analogy. Consider a sequence of tokens $\mathbf{x}_{1:T}$ that describes a story in which the character Harry is portrayed as the *villain* up to a position i and, as the plot unfolds, revealed as the *hero* at a later position j (with $i < j < T$). The state $\mathbf{S}_t$ acts as a compact, lossy and continuously updated dictionary of the story’s entities, that holds a values under a key and later *read* back through it. Let $\mathbf{k}_{\text{Harry}}$ be the key addressing this character; at any step between i and j , reading the memory with it, $\mathbf{S}_{t-1} \mathbf{k}_{\text{Harry}}$, returns a vector of high cosine similarity with the *villain* concept. At position j the model must *overwrite* this stale value with the new *hero* concept. The delta rule is not storing the new value right over the old one, but only makes the adjustment needed to transform the old concept into the desired one in the present. Formally, before writing the pair $(\mathbf{k}_t, \mathbf{v}_t)$, it reads what the memory currently holds at that address,

$$\mathbf{v}_t^{\text{old}} = \mathbf{S}_{t-1} \mathbf{k}_t, \quad (2.43)$$

and adds only the scaled difference $\mathbf{p}_t$ towards the target:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \underbrace{\beta_t (\mathbf{v}_t - \mathbf{v}_t^{\text{old}})}_{\mathbf{p}_t} \mathbf{k}_t^\top \quad (2.44)$$

Unrolling the memory now, we can interpret it as an aggregation of corrections over time, $\mathbf{S}_t = \sum_{i \leq t} \mathbf{p}_i \mathbf{k}_i^\top$, similar to linear attention (Section 2.2.6), though here $\mathbf{p}_t$ is not only input-dependent, as $\mathbf{v}_t$ is, but also depends on all the previous corrections, making the parallelism harder because comparing to the linear counterpart, the elements cannot be summed up in any order but with respect to their temporal arrangement.

Still, this ordering constraint is far weaker than a fully sequential recurrence: since each correction depends only on the ones preceding it, the updates inside a fixed-length chunk form a triangular system that Yang, B. Wang, et al. 2024 solve in closed form, recovering the chunkwise parallelism that makes training practical, not by lowering the cost the recurrence already has but by reshaping it into matrix products the hardware can saturate. The cost is a single triangular solve per chunk. This is the overhead a non-diagonal transition such as the delta rule pays over a purely diagonal state-space model,

and at the same time the very reason it can overwrite memory with a precision a diagonal decay cannot reach. This chunkwise planning is not specific to the delta rule: it is the standard training form for all linear-recurrence models, differing only in the per-chunk work (plain masked matrix products for a diagonal transition, plus the extra triangular solve when the transition is not diagonal). The limit is decided by the memory: the in-chunk term forms a $C \times C$ token-interaction matrix, so both memory and compute grow quadratically with the chunk length C , though summed over the T/C chunks this stays $O(TC)$, linear in the sequence length, the square falling on the constant C and never on T . This is why C is kept somewhat low instead of spanning the whole sequence. At $C = 1$ it is basically the sequential non-parallel recurrence, whereas at $C = T$ it becomes a single fully parallel pass carrying the $O(T^2)$ memory of quadratic attention; a moderate C keeps the C^2 term bounded while still exposing enough parallelism to saturate the hardware. We will not reproduce the exact chunkwise formulation here, and refer the reader to Yang, B. Wang, et al. 2024; Yang, Kautz, and Hatamizadeh 2024 for the full derivation.

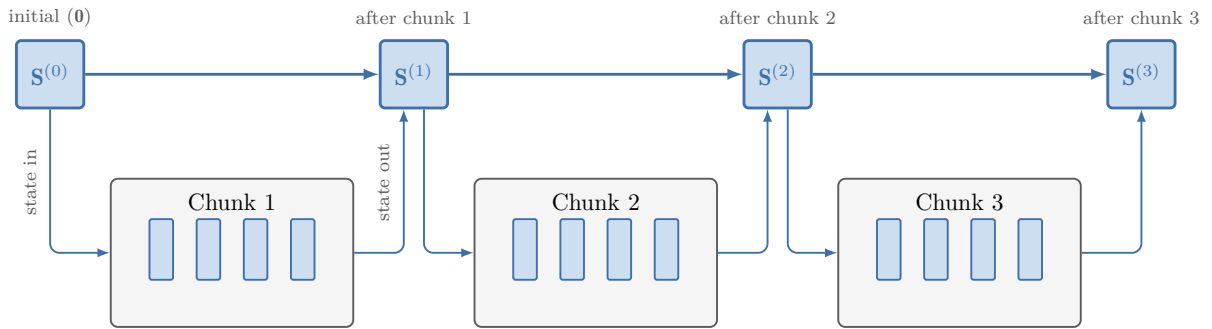


Figure 2.7: The chunkwise-parallel form. The sequence is split into chunks of length C : within each chunk all C tokens’ entries are resolved together in parallel, while across chunks the $d_h \times d_h$ state $\mathbf{S}$ (one per attention head) is carried forward sequentially and summarizes the whole preceding sequence exactly, resulting in an identical recurrence output.

2.4 Positional Embeddings

2.4.1 Absolute Positional Embeddings

An initial method Vaswani et al. (2017) to enhance vector representations of the tokens with positional information was to directly add positional vectors to the input. Thus for every input token’s vector representation in the sequence $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_T \in \mathbb{R}^d$ we add T positional encodings $\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_T \in \mathbb{R}^d$ ($\mathbf{u}_i^{PE} = \mathbf{u}_i + \mathbf{p}_i$). The authors proposed a time-correlated relationship between them through sinusoids such that the model can attend relative positions.

Thus for a token $\mathbf{u}$ at position t and dimension $i \in \{1, 2, \dots, d\}$:

$$p_{t,i} = \begin{cases} \sin\left(\frac{t}{\lambda^{i/d}}\right) & \text{if } 2 \mid i \\ \cos\left(\frac{t}{\lambda^{i/d}}\right) & \text{otherwise} \end{cases}, p_{t,i} \in \mathbb{R} \quad (2.45)$$

where λ is the base wavelength (default $\lambda = 10000$ in the original formulation).

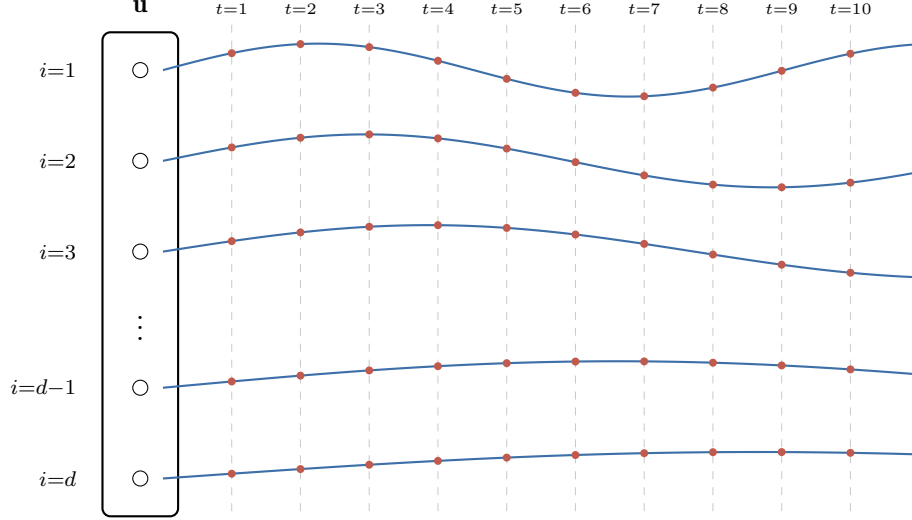


Figure 2.8: Absolute positional embeddings. Each row corresponds to a dimension i of the hidden representation, with a sinusoidal function of decreasing frequency ($\lambda = 5$, $d = 6$). Vertical lines mark token positions $t = 1, 2, \dots, T$, and each intersection (red dot) represents the scalar value $p_{t,i}$ added to the input embedding.

2.4.2 Relative Position Embeddings

Extending the original Transformer’s, Shaw, Uszkoreit, and Vaswani 2018 replaces absolute positional representations considering local distances between the tokens as a signal of information. Recalling the self-attention between two tokens $\mathbf{u}_i$ and $\mathbf{u}_j$ (at positions i and j respectively in the sequence $i, j \in \{1, 2, \dots, T\}$) as the normalized inner product of $\mathbf{Q}_i$ and $\mathbf{K}_j$:

$$\text{Attention}(\mathbf{u}_i, \mathbf{u}_j) = \text{softmax}\left(\frac{\mathbf{Q}_i^\top \mathbf{K}_j}{\sqrt{d}}\right) \mathbf{V}_j \quad (2.46)$$

where $\mathbf{Q}_i$ and $\mathbf{K}_j$ are per-head linear projections of the token representations

$$\mathbf{Q}_i = \mathbf{W}_Q \mathbf{u}_i \quad \mathbf{K}_j = \mathbf{W}_K \mathbf{u}_j \quad \mathbf{V}_j = \mathbf{W}_V \mathbf{u}_j \quad (2.47)$$

$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_h \times d_h}$ are time-constant trainable parameters.

We modify it by introducing two trainable biases $\mathbf{b}_K, \mathbf{b}_V \in \mathbb{R}^{(2T-1) \times d_h}$ (fixed T)⁵:

$$\text{Attention}(\mathbf{u}_i, \mathbf{u}_j) = \text{softmax}\left(\frac{(\mathbf{W}_Q \mathbf{u}_i)^\top (\mathbf{W}_K \mathbf{u}_j + \mathbf{b}_{K,j-i+T})}{\sqrt{d}}\right) (\mathbf{W}_V \mathbf{u}_j + \mathbf{b}_{V,j-i+T}) \quad (2.48)$$

Clipping the relative distance

The authors claim that precise relative position across large distances is useless, so they clip the relative distance $j - i$ to a clipping value k resulting in a smaller set of biases $\mathbf{b}_K, \mathbf{b}_V \in \mathbb{R}^{2k+1 \times d_h}$ and for a pair $\mathbf{u}_i$ and $\mathbf{u}_j$ the used bias is $\mathbf{b}_{*, \text{clip}(j-i, -k, k) + k + 1}$.

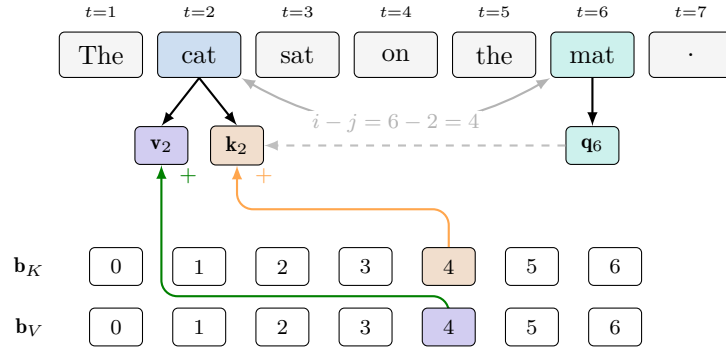


Figure 2.9: Relative position embeddings in causal attention. Token 6 (query) attends to token 2 (key/value) with relative distance $i - j = 4$. The bias vectors $\mathbf{b}_K$ and $\mathbf{b}_V$ are indexed at position 4 (highlighted) and added to the key and value projections respectively.

2.4.3 Rotary Positional Embeddings

In Rotary Positional Embeddings (RoPE), proposed in Su et al. 2024, we assume $2|d$ such that we can split $\mathbf{q}_i$ in a number of $d/2$ 2-dimensional parts $\mathbf{Q}_i = [\mathbf{q}_{i,[1,2]}; \mathbf{q}_{i,[3,4]}; \dots; \mathbf{q}_{i,[d/2-1,d/2]}]$ and $\mathbf{k}_j$ respectively. RoPE rotates each individual 2D vector using a rotation matrix $\mathbf{R}_{\mathbf{q}_i}$, parameterized by a wavelength λ (authors use a default value of 10000), the index of $\mathbf{q}$ in the query vector $\mathbf{Q}$ and the position of the token in the sequence.

$$\mathbf{R}_{\mathbf{q}_i, [c, c+1]} = \begin{bmatrix} \cos \Theta_{c/2} & -\sin \Theta_{c/2} \\ \sin \Theta_{c/2} & \cos \Theta_{c/2} \end{bmatrix}, \text{ where } \Theta_{\gamma} = \lambda^{-2(\gamma-1)/d} \quad (2.49)$$

⁵For causal attention where $j \leq i$, the biases reduce to $\mathbf{b}_K, \mathbf{b}_V \in \mathbb{R}^{T \times d_h}$ since the relative distance $i - j$ ranges from 0 to $T-1$, and the $+T$ centering offset in the index is no longer needed.

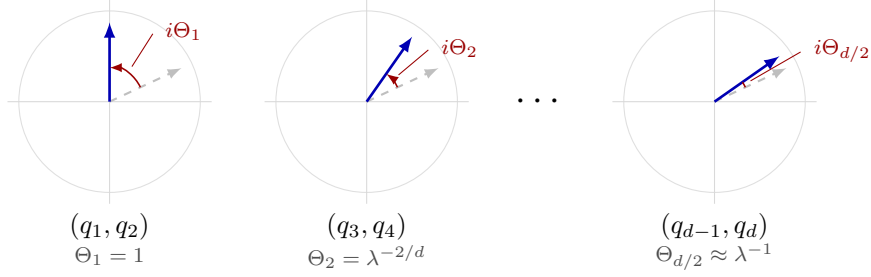


Figure 2.10: RoPE rotates each pair of dimensions in $\mathbf{q}$ and $\mathbf{k}$ by $m\Theta_\gamma$, where i is the token position in the sequence. Higher-indexed pairs (i.e. $\gamma \rightarrow d/2$) use exponentially lower frequencies, encoding position at multiple scales.

So a rotated query $\mathbf{Q}$ (or key $\mathbf{K}$) can be obtained by multiplying the rotation matrix $\mathbf{R}_{\mathbf{Q}_i} = \text{diag}(\mathbf{R}_{\mathbf{q}_i,[1,2]}, \mathbf{R}_{\mathbf{q}_i,[3,4]}, \dots, \mathbf{R}_{\mathbf{q}_i,[d/2-1,d/2]} \in \mathbb{R}^{2 \times 2}) \in \mathbb{R}^{d \times d}$ with the linear projection of the token $\mathbf{u}_i$:

$$\mathbf{Q}_i^{\text{RoPE}} = \begin{bmatrix} \cos i & -\sin i & 0 & 0 & \dots & 0 & 0 \\ \sin i & \cos i & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos i\lambda^{-2/d} & -\sin i\lambda^{-2/d} & \dots & 0 & 0 \\ 0 & 0 & \sin i\lambda^{-2/d} & \cos i\lambda^{-2/d} & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos i\lambda^{-1} & -\sin i\lambda^{-1} \\ 0 & 0 & 0 & 0 & \dots & \sin i\lambda^{-1} & \cos i\lambda^{-1} \end{bmatrix} \mathbf{W}_Q \mathbf{u}_i \quad (2.50)$$

The entire formula for computing the attention weights can be reformulated such that we use a single rotation matrix:

$$\begin{aligned} \text{Attention}(\mathbf{u}_i, \mathbf{u}_j) &= \text{softmax} \left(\frac{(\mathbf{R}_{\mathbf{Q}_i} \mathbf{W}_Q \mathbf{u}_i)^\top (\mathbf{R}_{\mathbf{K}_j} \mathbf{W}_K \mathbf{u}_j)}{\sqrt{d}} \right) \\ &= \text{softmax} \left(\frac{\mathbf{W}_Q \mathbf{u}_i \mathbf{R}_{*_{i,j}} \mathbf{W}_K \mathbf{u}_j}{\sqrt{d}} \right) \end{aligned} \quad (2.51)$$

This rotation can be reformulated in a more efficient manner by mapping the 2-dimensional parts $\mathbf{q}_{i,[1,2]}, \mathbf{q}_{i,[3,4]}, \dots, \mathbf{q}_{i,[d/2-1,d/2]} \in \mathbb{R}^2 \rightarrow \mathbb{C}^2$ (odd-indexed dimensions are the real parts, and real-indexed dimensions are the imaginary parts for each pair, below denoted using the ket notation for the entire vector of dimension d). In the complex plane, the rotation matrix $\mathbf{R}_{*_{i,[c,c+1]}} \in \mathbb{R}^{2 \times 2}$ is replaced by a complex number $e^{i\varphi} = \cos\varphi + i\sin\varphi$ (a counter-clockwise rotation of the vector by φ radians), so:

$$\mathbf{Q}_m^{\text{CRoPE}} = \begin{bmatrix} \Theta_1 & 0 & \dots & 0 & 0 \\ 0 & \Theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & \Theta_{d/2-1} & 0 \\ 0 & 0 & \dots & 0 & \Theta_{d/2} \end{bmatrix} |\mathbf{W}_Q \mathbf{u}_m\rangle_{\mathbb{C}}, \text{ where } \Theta_\gamma = e^{i\lambda^{-2(\gamma-1)/d}} \quad (2.52)$$

$$\mathbf{Q}_{m,k}^{\text{RoPE}} \leftarrow |\mathbf{Q}_{m,k}^{\text{RoPE}}\rangle_{\mathbb{R}} = \begin{cases} \text{Re}(Q_{m,k}^{\text{CRoPE}}) & \text{if } 2 \nmid k \\ \text{Im}(Q_{m,k}^{\text{CRoPE}}) & \text{if } 2 \mid k \end{cases} \quad \text{for } k = 1, 2, \dots, d \quad (2.53)$$

Similarly, we can reformulate such that we use a single rotation matrix in the complex plane:

$$\begin{aligned} \text{Attention}(\mathbf{u}_n, \mathbf{u}_m) &= \text{softmax}\left(\frac{|(e^{in\varphi} |\mathbf{W}_Q \mathbf{u}_n\rangle_{\mathbb{C}})^\top (e^{im\varphi} |\mathbf{W}_K \mathbf{u}_m\rangle_{\mathbb{C}})\rangle_{\mathbb{R}}|}{\sqrt{d}}\right) \\ &= \text{softmax}\left(\frac{||\mathbf{W}_Q \mathbf{u}_i\rangle_{\mathbb{C}} e^{i(n-m)\varphi} |\mathbf{W}_K \mathbf{u}_j\rangle_{\mathbb{C}}\rangle_{\mathbb{R}}|}{\sqrt{d}}\right) \end{aligned} \quad (2.54)$$

Extending the context window

A simple way (with inherent drawbacks) to allow larger contexts is to scale the sequence position parameter m of the rotation matrix $\mathbf{R}_{*,m}$ by the newer max sequence length $m \leftarrow mT_{\text{old}}/T_{\text{new}}$ ($T_{\text{new}} > T_{\text{old}}$) along with finetuning until the model gets used to the newer compressed positional embeddings, while holding the same base frequency.

Instead of scaling the frequencies equally, the NTK-Aware interpolation considers scaling the frequencies differently:

$$\Theta_\gamma = (\lambda s^{\frac{d}{d-2}})^{-\gamma/d} \text{ or } e^{i(\lambda s^{\frac{d}{d-2}})^{-\gamma/d}} \text{ for rotations in } \mathbb{C} \quad (2.55)$$

Yet Another Rope extension (YaRN, Peng et al. 2023) is a method to extend the maximum context length with way less finetuning compute by scaling down the rotations based on the rotation frequencies. Slow channels (dedicated for long distance positional encodings) are scaled down by the extension factor, while fast channels (enhancing local positional relationships) are left as is. The scaling of intermediate frequencies is linearly interpolated between 1 and the inverse of the extension factor. YaRN also sharpens the attention map by $1/t = (1 + 0.1 \ln(\text{extension_factor}))^2$. Importantly, the same learned weights can be used with different YaRN scaling factors, as the scaling is applied to the positional encodings and attention rather than requiring separate weights.

2.5 Inference Optimizations

2.5.1 Speculative Decoding

In auto-regressive decoding, the classic speculative decoding (Leviathan, M. Kalman, and Matias 2023, C. Chen et al. 2023) harnesses the power of knowledge distillation as it uses the smaller version of the model to predict at a fraction of the cost what the larger model will generate. While it does not have full accuracy, the smaller model can still be used to reduce the total number of forward passes through the larger model, reproducing with high accuracy what the bigger model would have predicted, resulting in practice speedups of around $2 - 3\times$ in LLM serving. The exact trajectory that the larger model would have produced is verified in a single shot by the bigger model, thus preserving the same token likelihood as direct decoding.

The process is divided into two stages: the small model is used to decode the next k tokens $t_{n+1}, t_{n+2}, \dots, t_{n+k}$ (that means k successive forward passes), then these tokens are passed through the larger model in a single forward pass. Then, one by one, the tokens' probabilities of being accepted are computed like so:

$$p(t_i \checkmark) = \min\left(1, \frac{p_{\text{large}}(t_i \mid t_1, t_2, \dots, t_{i-1})}{p_{\text{small}}(t_i \mid t_1, t_2, \dots, t_{i-1})}\right) \quad (2.56)$$

If a token is accepted, the process moves to the next one. If rejected, t_i and all subsequent draft tokens $t_{i+1}, \dots, t_{n+k}$ are discarded, and t_i is resampled from a readjusted distribution, derived from the probability residual of the two models on each token:

$$p'(t_i \mid t_1, t_2, \dots, t_{i-1}) = \frac{\max(0, p_{\text{large}}(t_i \mid t_1, t_2, \dots, t_{i-1}) - p_{\text{small}}(t_i \mid t_1, t_2, \dots, t_{i-1}))}{\sum_{t \in \mathcal{V}} \max(0, p_{\text{large}}(t \mid t_1, t_2, \dots, t_{i-1}) - p_{\text{small}}(t \mid t_1, t_2, \dots, t_{i-1}))} \quad (2.57)$$

where $\mathcal{V}$ is the vocabulary. If all k draft tokens are accepted, a bonus token is sampled directly from $p_{\text{large}}(\cdot \mid t_1, t_2, \dots, t_{n+k})$, meaning each speculative decoding step produces between 1 and $k+1$ new tokens. This rejection sampling scheme guarantees that the final output distribution is identical to that of the large model alone, regardless of the draft model's quality.

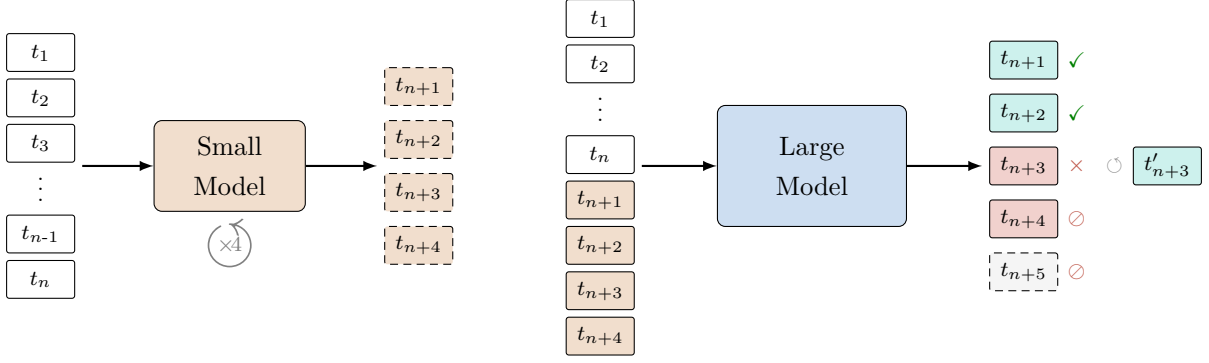


Figure 2.11: Speculative decoding. **Step 1:** the small model drafts k tokens from the current context $\{t_1, t_2, \dots, t_n\}$. **Step 2:** the context along with the draft tokens are passed in one shot through the large model to compute their probability distributions. Accepted tokens are kept; the rejected token is resampled using the readjusted distribution (Eq. 2.56) and all subsequent ones are discarded, ending up with the new context $\{t_1, t_2, \dots, t_{n+2}, t'_{n+3}\}$ to be drafted on in the next iteration.

To improve it even further in achieving up to $\times 4$ speedups, Kumar, Dao, and May 2026 accelerates the process by continued decoding on the branches that seem to be more viable to be accepted by the larger model.

2.5.2 Multi-Token Prediction (MTP)

As a form of regularizer that pushes the model to internally predict multiple tokens ahead, the Multi-Token Prediction (MTP) architecture was specifically designed to fasten up the decoding throughput. While in many forms and for different purposes (Gloeckle et al. 2024, Li et al. 2024) it is part of modern LLM pipelines. While it acts at train time as a regularizer to enforce the model to internalize within a single hidden state a longer extension of the context, through one single forward pass, speculative decoding is boosted considerably at a lower overhead comparing to a smaller distilled model (often, each MTP head is a single transformer layer and the recommended tokens to be drafted is 2).

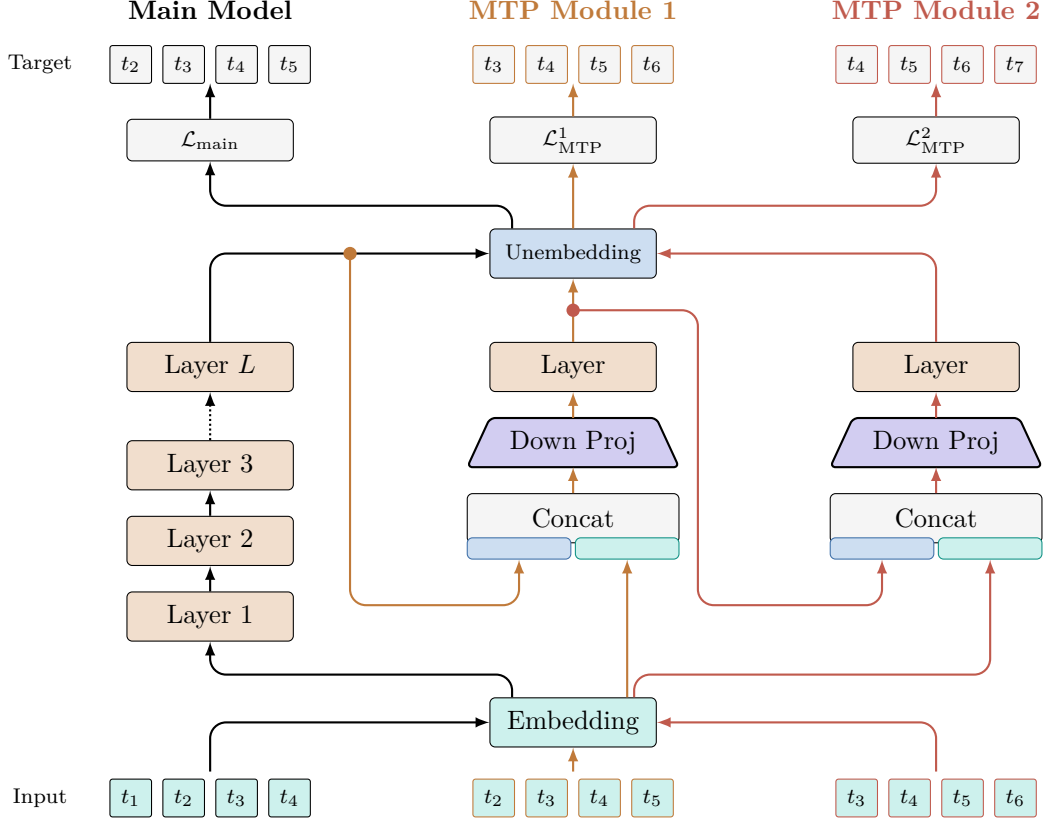


Figure 2.12: Multi-Token Prediction training as designed by DeepSeek-AI 2025. The main model (left) processes the input token sequence as normal. An MTP module receives a shifted by 1 version of the input sequence embeddings, and while concatenated with the last hidden state of the main model, it is predicting using a single transformer block the next token in the sequence. Similarly, next order of MTP modules follow the same pattern. The final loss is a weighted sum of all cross entropy losses (here $\mathcal{L}_{\text{main}}$, $\mathcal{L}_{\text{MTP}}^1$, and $\mathcal{L}_{\text{MTP}}^2$).

A downside of this architecture lies in the fact that, starting from the second MTP module, the drafted tokens remain independent of the previously drafted tokens, resulting in a higher rejection rate by the large verification model. DeepSeek-AI (2026b) further propose adding a sequential speculative decoding head with Markovian dependencies (each drafted token receives a conditioning bias from the previous token), resulting in longer accepted drafts ($\sim +30\%$).

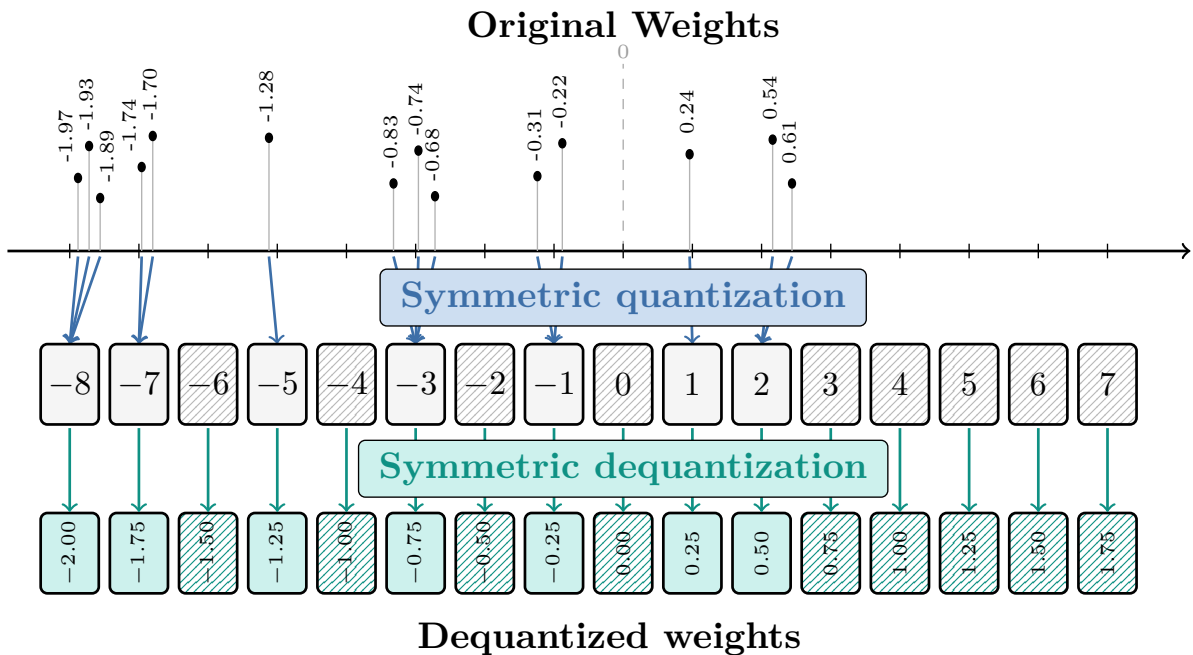
In practice, they train a bias matrix $B \in \mathbb{R}^{V \times V}$ decomposed into two smaller matrices $W_1 \in \mathbb{R}^{V \times 256}$ and $W_2 \in \mathbb{R}^{256 \times V}$ (similar to LoRA), where $B_{i,j}$ is a learned bias favouring token j after token i in comparison to the rest of the tokens. Then, they bias the MTP head output logits $O \in \mathbb{R}^{|\text{MTP}| \times V}$ before sampling.

In large-scale serving engines, it is notable that the drafting model (i.e., a smaller distilled model or MTP heads) responds differently depending on the user request; structured

output formats such as code tend to have a higher acceptance rate. Considering this, the latest serving engines (DeepSeek-AI 2026b) also take demand fluctuation into account, increasing or reducing the length of the drafted token sequences as needed. Rather than relying on this cheap length knob alone, they introduce a secondary learnable head that estimates the probability of each drafted sequence being accepted by the large model. Depending on the demand, a dynamic cumulative-probability threshold then decides where the speculated sequence is cut before verification.

2.5.3 Quantization

Model quantization represents a real consideration when dealing with the biggest hardware bottleneck, the memory. At large scale, models tend to be less sensitive to the precision of their weights, and they permit compressing them to lower formats, at the cost of some accuracy loss. For our implementations, we adopt the GGUF format, making the int8-W8A32 the default choice when running the models in Unity (in Section ?? we show that at 8bit the models maintain similar performance and generation speed while saving 2 times the memory).



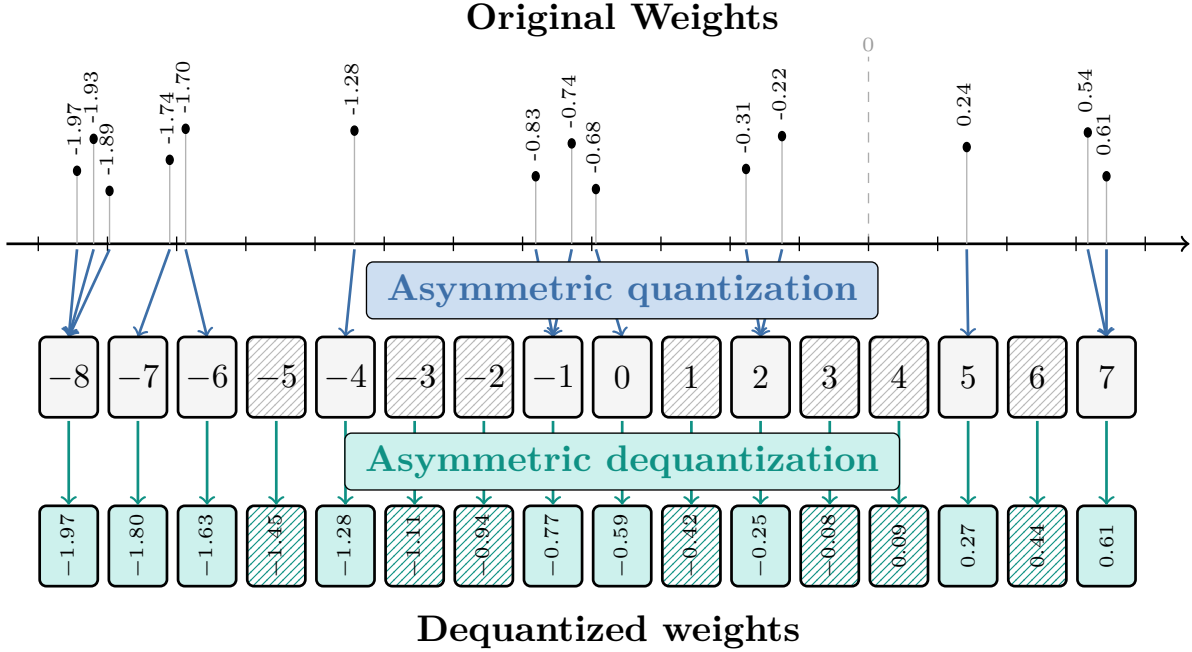


Figure 2.13: Comparison of symmetric and asymmetric int4 quantization using the same weight samples. The symmetric mapping uses signed codes centered around zero, while the asymmetric one uses a shifted and rescaled mapping over the same signed bucket labels from -8 to 7 . In both cases, several different weights collapse into the same quantized bucket.

In the 4bit GGUF Q4_0 format we observe a significant drop in accuracy, which makes the models unusable for our purposes (see Section 4.1), a negative behavior strictly emerging from its simplicity. We skip more advanced methods like the Q4_K_M format (which protects the most sensitive tensors at higher precision), double quantization, NF4 or calibration-dependent methods for the sake both the speed of development and of the models themselves. Embedding and normalization layers are always kept in **fp16**, as they are very numerically sensitive (Dettmers, Lewis, et al. 2022). The 8bit format (W8A32) in symmetric quantization becomes the default given the best tradeoff of quality, speed and kernels implementation in our framework.

2.6 Finetuning Methods

Adapting a pretrained model with diverse general knowledge can be difficult without the right techniques, considering its size and the main bottleneck being GPU memory. During pretraining, the model acts as a lossy compression system over the internet data; it recognizes text patterns and is capable of mimicking texts from different distributions. Through finetuning, we usually aim to adjust a subset of dimensions within the weights

of the model (latent knowledge) to be more prominent while suppressing others. While full finetuning (updating all parameters of the model) is the ceiling point of performance, low resources cannot afford allocating high precision gradients, activations and optimizer states. Earlier parameter-efficient variants (last-layer tuning, inserted adapter layers or prefix tuning) were surpassed by the observation that the weight update accumulated during finetuning has a low intrinsic rank, so it is well approximated by the product of two low-rank matrices. Thus, doing gradient descent in this lower dimensional subspace matches the target loss while implicitly regularizing the update. In LoRA (Hu et al. 2021), all the base parameters are frozen and the linear projections receive a learnable adapter that modifies their weights:

$$\mathbf{W}' = \underbrace{\mathbf{W}}_{\text{frozen}} + \frac{\alpha}{r} \mathbf{B}\mathbf{A}, \quad (2.58)$$

where $\mathbf{B} \in \mathbb{R}^{d \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times d'}$ form the adapter of a base projection $\mathbf{W} \in \mathbb{R}^{d \times d'}$, $r \ll \min(d, d')$ is the adapter rank and α a constant scaling factor determining the strength of the adapter (usually $\alpha \in [r, 2r]$). $\mathbf{B}$ is initialized with zeros and $\mathbf{A}$ from a Gaussian, so at the first step of the optimization the output is the same as the pretrained model's; in this first step only $\mathbf{B}$ receives a nonzero gradient, since $\nabla_{\mathbf{A}} \mathcal{L} = \mathbf{B}^\top \nabla_{\mathbf{B}\mathbf{A}} \mathcal{L} = \mathbf{0}$. After training, the adapter weights are merged into the original ones ($\mathbf{W} \leftarrow \mathbf{W} + \frac{\alpha}{r} \mathbf{B}\mathbf{A}$), maintaining the same inference latency as the pretrained model.

Dettmers, Pagnoni, et al. 2023 introduced the NF4 quantization format and used it to push the low-budget finetuning limits even further by keeping the frozen weights of the model quantized and dequantizing them on the fly during the forward pass, layer by layer. Kalajdzievski 2023 deduced that at higher adapter ranks the gradient magnitude shrinks (the $1/r$ factor averages the r rank-one components of $\mathbf{B}\mathbf{A}$, whose sum only grows like $\sqrt{r}$), so the scaling was readjusted from α/r to $\alpha/\sqrt{r}$.

Besides that, knowledge transfer to a new task, if not done with care, degrades the model's performance on previous benchmarks, a phenomenon known as *catastrophic forgetting* (McCloskey and Cohen 1989). Another problem happening with supervised finetuned models is that they are trained on perfect teacher trajectories without ever experiencing their own. During inference, such post-trained models accumulate small errors over states, ending up in a state that was never actually met in the training data and from which they cannot recover (Ross, Gordon, and Bagnell 2011), a failure Ranzato et al. 2015 call *exposure bias*.

To address this, On-Policy Self-Distillation (S. Zhao et al. 2026) harnesses the in-context learning capability of a model to update its own weights, directly onto its own rollouts. One network parametrized by θ is instantiated as two policies: the *student* $p_\theta^S(\cdot | x)$ sees only the problem x , while the *teacher* $p_\theta^T(\cdot | x, y^*)$ runs the same parameters to compute the per-token distribution of the rollout but this time with privileged information y^* in its context, the ground-truth answer or a reference chain-of-thought.

Figure 2.14 contrasts its performance against SFT on three tasks finetuned one after another. Under SFT (top) each task is learned in its own phase and lost shortly in the next, so that only the performance of the last learned task prevails. Under OPSD (bottom) the same tuning phases raise all performance metrics, the earlier ones flattening instead of collapsing.

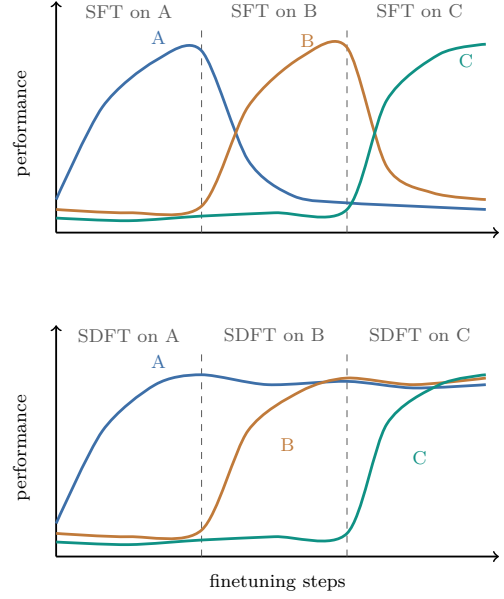


Figure 2.14: SFT vs OPSD on sequential finetuning (schematic, after S. Zhao et al. 2026).

The secret is its on-policy nature, the teacher never writing an answer or generating target labels. Instead the student’s rollouts are steered towards the good trajectory (it learns to correct from deployment states):

$$\mathcal{L}_{\text{OPSD}}(\theta) = \underbrace{\mathbb{E}_{(x, y^*) \sim \mathcal{S}}}_{\text{expectation over training data}} \underbrace{\mathbb{E}_{\hat{y} \sim p_\theta^S(\cdot | x)}}_{\text{expectation over the student's rollouts}} \underbrace{\sum_{n=1}^{|\hat{y}|}}_{\text{sum over the tokens}} D_{\text{KL}} \left(\underbrace{p_\theta^T(\cdot | x, y^*, \hat{y}_{<n})}_{\text{same weights, reading the hint}} \parallel \underbrace{p_\theta^S(\cdot | x, \hat{y}_{<n})}_{\text{the distribution being trained}} \right) \quad (2.59)$$

where D_{KL} is written here as a forward KL (the variant adopted by the authors), though the same slot accepts the reverse KL or the Jensen-Shannon divergence, each asking the student for a different kind of agreement:

- **Forward KL**, $D_{\text{KL}}(p_\theta^T \parallel p_\theta^S)$. Here, the average divergence is taken over the teacher, so the loss penalizes the student for not covering mass of the teacher’s distribution (mean-seeking). This is the classical distillation objective (G. Hinton, Vinyals, and Dean 2015), the same thing as a cross-entropy against the teacher’s soft labels:

$$\begin{aligned}
D_{\text{KL}}(p_{\theta}^T \parallel p_{\theta}^S) &= \sum_{v \in \mathcal{V}} p_{\theta}^T(v) \log \frac{p_{\theta}^T(v)}{p_{\theta}^S(v)} \\
&= \underbrace{- \sum_{v \in \mathcal{V}} p_{\theta}^T(v) \log p_{\theta}^S(v)}_{\text{CrossEntropy}(p_{\theta}^T, p_{\theta}^S)} - \underbrace{\left(- \sum_{v \in \mathcal{V}} p_{\theta}^T(v) \log p_{\theta}^T(v) \right)}_{\mathcal{H}(p_{\theta}^T)} \quad (2.60)
\end{aligned}$$

and the teacher branch is detached, so $\mathcal{H}(p_{\theta}^T)$, representing the entropy in the teacher’s distribution, carries no gradient and $\text{CrossEntropy}(p_{\theta}^T, p_{\theta}^S)$ is the component that receives it. We can obtain the classic SFT loss by collapsing the teacher probability distribution into a one-hot target, which leaves $-\log p_{\theta}^S(y_n^*)$. This makes the difference between a multi-variant target (retaining dark knowledge) and a fixed, narrow path of success.

- **Reverse KL**, $D_{\text{KL}}(p_{\theta}^S \parallel p_{\theta}^T)$. Inversely, the average is taken over the student, so it is punished for mass it puts where the teacher has none, while it may quietly ignore entire zones the teacher supports (mode-seeking). It is a similar objective to reinforcement learning, where the student concentrates on a low-entropy narrower path, and it is the usual choice for on-policy distillation: negating it and splitting the logarithm the other way,

$$-D_{\text{KL}}(p_{\theta}^S \parallel p_{\theta}^T) = \underbrace{\mathbb{E}_{v \sim p_{\theta}^S} [r(v)]}_{\text{expected reward}} + \underbrace{\mathcal{H}(p_{\theta}^S)}_{\text{entropy bonus}}, \quad (2.61)$$

$$\nabla_{\theta} \mathbb{E}_{v \sim p_{\theta}^S} [r(v)] = \mathbb{E}_{v \sim p_{\theta}^S} [r(v) \nabla_{\theta} \log p_{\theta}^S(v)]$$

with the reward $r(v) = \log p_{\theta}^T(v)$ detached as above, which the identity requires: since the teacher shares θ , the exact gradient would carry a second, pathwise term $\mathbb{E}_{v \sim p_{\theta}^S} [\nabla_{\theta} r(v)]$ that the stop-gradient discards. What is left is the score-function estimator, in which the reward only ever enters as a scalar weight, so minimizing the reverse KL amounts to maximizing a REINFORCE objective, rewarding the student for the log-probability the teacher assigns to what it sampled, regularized by its own entropy to keep it from collapsing onto a single token. In this setup, there are no return estimators (a value function, or a group-mean baseline as in Shao et al. 2024) because the reward is dense and its expectation over $\mathcal{V}$ is summed in closed form, leaving no sampling variance to reduce and no sparse terminal signal to roll backwards.

- **Jensen-Shannon-Divergence**, a generalized form of the two that optimizes both at once when the knob is in the middle. Instead of bringing one distribution close to the other, it brings both towards a middle ground:

$$\text{JSD}_{\beta}(p_{\theta}^T \parallel p_{\theta}^S) = \beta D_{\text{KL}}(p_{\theta}^T \parallel m) + (1 - \beta) D_{\text{KL}}(p_{\theta}^S \parallel m) \quad (2.62)$$

where $m = \beta p_\theta^T + (1 - \beta)p_\theta^S$ is a weighted average of the two distributions. Unlike either KL, it is bounded since none of the terms can go to infinity.

In the original adversarial objective (Goodfellow et al. 2014), when the discriminator reaches the optimal point, the Jensen-Shannon divergence becomes the objective that is minimized. Considering the teacher as the data distribution (real samples) and the student as the generator (fake samples), and taking the even average $\beta = \frac{1}{2}$, we define the optimal discriminator as $D^\star = p_\theta^T / (p_\theta^T + p_\theta^S) = p_\theta^T / 2m$. Substituting it into the adversarial objective $\mathcal{L}_{\text{GAN}}$ leaves nothing but the average behind:

$$\begin{aligned}
\mathcal{L}_{\text{GAN}}(D) &= \mathbb{E}_{v \sim p_\theta^T} [\log D(v)] + \mathbb{E}_{v \sim p_\theta^S} [\log (1 - D(v))], \\
\mathcal{L}_{\text{GAN}}(D^\star) &= \mathbb{E}_{v \sim p_\theta^T} \left[\log \frac{p_\theta^T(v)}{2m(v)} \right] + \mathbb{E}_{v \sim p_\theta^S} \left[\log \frac{p_\theta^S(v)}{2m(v)} \right] \\
&= \underbrace{\mathbb{E}_{v \sim p_\theta^T} \left[\log \frac{p_\theta^T(v)}{m(v)} \right]}_{D_{\text{KL}}(p_\theta^T \parallel m)} + \underbrace{\mathbb{E}_{v \sim p_\theta^S} \left[\log \frac{p_\theta^S(v)}{m(v)} \right]}_{D_{\text{KL}}(p_\theta^S \parallel m)} - \log 4 \quad (2.63) \\
&= 2 \text{JSD}_{1/2}(p_\theta^T \parallel p_\theta^S) - \log 4
\end{aligned}$$

Pulling the 2 out of each logarithm is what produces the constant $-\log 4$, and what is left is exactly the two KL terms of the definition above. So against a perfect discriminator the generator has only one quantity left to push down, and it is the Jensen-Shannon term; $-\log 4$ is the floor it reaches when $p_\theta^S = p_\theta^T$, at which point $D^\star$ equals 1/2 everywhere and the discriminator is reduced to guessing. The knob β slides between the two behaviours above, and the bound is what keeps a training run stable where either KL could blow up.

Given that on-policy distillation normally reaches for the reverse direction, the authors' empirical results show the opposite. A plausible explanation is that the teacher here is not a different model with different trajectory distributions, but just one heavily influenced by the given hint. Then we can expect the student to match the whole teacher distribution, instead of only a specific mode.

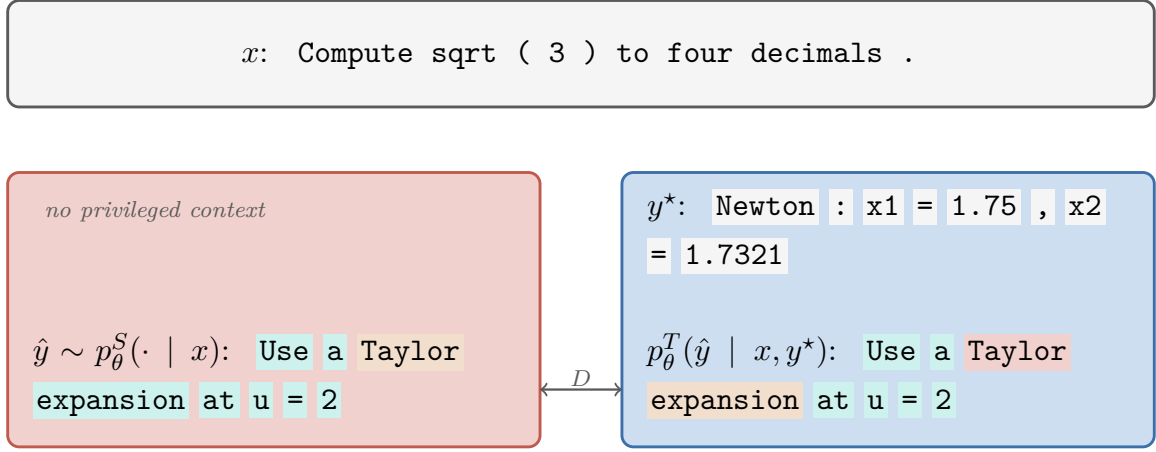


Figure 2.15: One OPSD rollout. A problem x is given to the student to perform a rollout $\hat{y}$ (left). That is passed through the teacher as $[x; y^*; \hat{y}]$, giving a target distribution at every position of $\hat{y}$ (right). The shading in the figure tells how much probability the policy puts on each token: teal where it is confident, orange where it hesitates, red where it would have written something else. Here, the student uses a wrong method to solve the problem, but with the hint sitting right after the question, the same weights prefer cheating by reformulating the given hint to an answer as if it had never been shown, an in-context copying capability triggered by induction heads (Olsson et al. 2022). The per-token divergence becomes clear, and it is corrected via the KL loss, whose gradient is applied only over the student side. This method makes the model correct itself on its own generated rollouts (the real inference scenario), without drastic shifts over the conditioned distributions.

2.7 DeepUnity Framework

The LLM implementation inside Unity is built on top of the DeepUnity library⁶ (Ciobanu and Zaharie 2024). In this section, we give a brief introduction to the features already present in the library and how the final project represents a complete set of deep learning tools, adapted to the new era of deep learning, even though the inference engine does not share the previous tools given the optimizations that had to be done. The theory behind the neural network modules and RL algorithms is presented in the reference paper.

⁶<https://github.com/smtmRadu/DeepUnity>

2.7.1 Deep Learning Library

Firstly, the project is composed of a deep learning library native to the Unity game engine, offering tools and backends (implemented from the ground up in C#, including HLSL compute shader kernels for heavy operations like matrix multiplication and convolution) for training and serving neural networks all inside the engine, without external dependencies or libraries. While common limitations do exist, such as the lack of an autograd system or full GPU integration, the current implementations capture enough power to train neural networks for basic tabular, vision, and even RL tasks, especially for training AI agents for games.

Several parts of the project were fixed and new modules previously unimplemented or unreleased are now ready to use, so that the developers can experiment with new tasks or configurations.

2.7.2 Reinforcement Learning Library

On top of the library, the project comprises scripts dedicated to creating, training and running in-game agents (walkers, crawlers, enemies etc.) using state of the art reinforcement learning algorithms, namely Proximal Policy Optimization (PPO) (Schulman et al. 2017), Soft Actor-Critic (SAC) (Haarnoja et al. 2018), Twin Delayed Deep Deterministic Policy Gradient (TD3) (Fujimoto, Hoof, and Meger 2018) and Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al. 2015).

As part of the completion of the project, besides the introduction of LLM inference within the engine, the RL side received optimization updates for improved rollout and update speeds, in a separate pipeline from the old implementation. The PPO and SAC algorithms were fully migrated to GPU, backward compatible with previous implementations, delivering instant policy updates and maximized performance for parallel agents. The library still leaves room for improvement using the Unity DOTS system, specifically developed for very large numbers of parallel `GameObjects`, which could help on the episode rollout side.

Chapter 3

Implementation

The project started by testing out smaller models (e.g. [Gemma 3 270M](#), [Qwen 3 0.6B](#)) with reused GPU kernels (e.g. matrix multiplication, self-attention) and C# modules from the original DeepUnity project. With these, a hybrid CPU/GPU inference would score around 3 to 4 tokens/s decode speed, below an acceptable target of 10 tokens/s. With full GPU migration, dedicated decoding kernels, frame-distributed optimizations for loading and quantization, the inference engine far surpassed our target, and even runs a Text-To-Speech model (TTS) in parallel natively. The intense work on these optimizations and the TTS integration was done using the [Claude Fable 5](#) model. Beyond that, previous work received numerous bug-fixes in its NN modules and RL algorithms. With that, the entire project left the pre-alpha stage and was released as version 1. In the following sections, we describe the implementation difficulties and decisions made to finalize the backend of the inference engine, and also the developer interface, so any game can integrate it flawlessly.

3.1 Implementation Details

This section details the implementation design of the inference engine, including the tokenization process, GPU kernels and the NPC interface relative to the tools Unity provides.

3.1.1 Tokenization

In the discussion of processing non-numeric data such as text, images, audio or video, sequence modeling architectures are used, so the data can be processed by the same model, regardless of the length or size. Sequence modeling modules (recurrent, attention-based, etc.) are designed to process, at least in theory, an infinite number of fixed-size vectors. Today, regardless of the data type, splitting a datapoint into a variable-length sequence of chunks lies at the foundation of processing non-numeric data with machine learning tools at scale: texts are broken into tokens, images into patches, audio into split-

second frames and so on. In the subsequent subsections we discuss how text tokenization works and how it is improved for efficiency.

Byte Pair Encoding (BPE)

Recalling that for processing any finite sequence $x \in \Omega^*$ over an alphabet Ω we need to define a chunk of a datapoint as a unique vector, we cannot really chunk text into words (i.e. it is not easy to assign large vectors for each word in the vocabulary $\mathcal{V}$, due to the high number of words, and that will not make it viable for other languages, character level mistakes or capitalization). Sequence modeling modules, on the other hand, are very costly for long sequences, so a character level chunking of the text will create an extremely long division of the text (regardless that the vocabulary will be small). The BPE algorithm lies somewhere in between the two: pre-deciding an acceptable vocabulary size, compose it using the characters needed that allow covering any kind of text data and fill it up to the full size of the allocated vocabulary with the most common chunks within a dataset (so the probability of chunking a text into a smaller number of tokens is higher). To make this more precise, we will define below the steps of actually creating an injective (left invertible) tokenization function $\psi : \Omega^* \rightarrow \mathcal{V}^*$, where $\psi^\dagger(\psi(x)) = x$, $x \in \Omega^*$.

Creating the starting vocabulary

The byte-pair encoding (BPE) algorithm is the standard tokenization method for text. At creation, it is built from the minimal (byte-level) alphabet, which is capable of representing any text, since any text can be UTF-8 encoded and every UTF-8 byte is one of its 256 symbols.

For the [Qwen 3.5](#) model series, the vocabulary declared in the configuration comprises $|\mathcal{V}| = 248,320$ tokens. At that size the embedding and unembedding matrices are each of size $|\mathcal{V}| \times d$, which for the 4B model ($d = 2560$) is 636 M parameters, or 1.27 GB in fp16. For a model this small the embedding layer takes a significant proportion of its entire size, and given these weights are also too sensitive to be quantized (Section 3.1.2), the architecture transposes the embedding matrix and reuses it as an unembedding layer as well, to cut down on the total parameter count.

Compared to a byte-level tokenizer, which spends one token per UTF-8 byte, the lengths of tokenized texts come out about 3.3 times shorter for English and 5 times shorter for Chinese, and in standard attention that feels like squared - roughly $11\times$ and $25\times$ less attention work over the same text. Those two factors follow from DeepSeek’s V4 tokenizer (DeepSeek-AI 2026a), documented at roughly 0.3 tokens per English character and 0.6 tokens per Chinese character, against the one UTF-8 byte an English character occupies and the three a Chinese one does. This does not necessarily mean that talking to the model in Chinese costs twice as many tokens, because a Chinese character inherently

carries a concept that can hardly be described in a single English word, so the two rates are not measuring comparable units.

Train the tokenizer

AI labs have separate tokenization datasets besides their pretraining datasets (Land and Bartolo 2024). They are usually selected from diverse and very common subjects and extended for non-English languages and other UTF-8 characters. On this corpus, $\mathcal{V}$ is extended by repeatedly finding the most common combinations of current entries in $\mathcal{V}$ until the desired target is hit. This technique ensures that in real use, a text string will statistically be split into as few tokens as the vocabulary permits, so that the quadratic cost of attention is paid over the shortest sequence the vocabulary allows.

Before anything is counted, however, the corpus is cut by a regular expression into word-like chunks - letters, digit groups, punctuation runs, whitespace - and no merge is ever allowed to cross one of those cuts, which is what stops a single token from spanning two words. The same expression decides how numbers are seen, and here the choice is deliberate: [Qwen 3.5](#) emits every digit on its own, so its entire vocabulary holds exactly ten purely numeric tokens and 2024 is always four of them.¹ What the loop leaves behind is therefore not only $\mathcal{V}$ but the order in which the merges were found: encoding new text replays that order by rank, so a frequently seen word collapses into a single token while a rare one falls back gracefully towards its bytes.

To make an instruct model out of a pretrained one, we reserve some slots in the vocabulary to delimit or pad the user’s and the assistant’s texts (these are known as special tokens). These slots are declared in the tokenizer before pre-training, so the embedding matrix already carries a row for each of them; what the instruction tuning stage adds is not the tokens themselves but the behaviour of emitting them in the right places.

¹Interpretability research (Lindsey et al. 2025) finds that models add internally by combining an approximate magnitude pathway with a modular one that fixes the last digit, rather than by the column algorithm they describe when asked. Keeping whole numbers in the vocabulary would end up under-trained (Land and Bartolo 2024), which is exactly where simple arithmetic starts to fail.

Algorithm 9 Building the tokenizer vocabulary $\mathcal{V}$. It starts from the vocabulary seed, updates it with the most frequent pairs until the target size is reached, and only then adds the predefined tokens.

Input: corpus $\mathcal{D}$, target size n , reserved tokens $\mathcal{A}$

Output: vocabulary $\mathcal{V}$ and the ordered merge list $\mathcal{M}$

Require: $b : \{0, \dots, 255\} \rightarrow \text{printable Unicode}$ ▷ keeps the state a plain string
Require: R , the pre-tokenizer regex ▷ no merge may cross the splits it makes
1: $\mathcal{V} \leftarrow \{b(x) : x \in \Omega\}$ ▷ seed: nothing can fall out of vocabulary
2: $\mathcal{M} \leftarrow []$ ▷ the merge order, which encoding replays
3: $W \leftarrow \text{pre-tokens } R(\mathcal{D}), \text{ as base symbols}$ ▷ with their counts
4: **while** $|\mathcal{V}| < n - |\mathcal{A}|$ **do**
5: $(u, v) \leftarrow \arg \max \text{freq}_W(u, v)$ ▷ over adjacent pairs only
6: **if** $\text{freq}_W(u, v) = 0$ **then break** ▷ corpus exhausted before the budget
7: $\mathcal{V} \leftarrow \mathcal{V} \cup \{uv\}$ ▷ the new token takes the next free id
8: $\mathcal{M} \leftarrow \mathcal{M} \parallel (u, v)$ ▷ appended last, so it fires last
9: rewrite every adjacent (u, v) in W as uv ▷ so the next count sees it
10: **end while**
11: $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{A}$ ▷ above the trainable range; no merge can produce one
12: **return** $\mathcal{V}, \mathcal{M}$

Improving tokenization speed

The naive method to evaluate ψ is to first split the text into pre-tokens, then write each pre-token as a sequence of base symbols and repeatedly fuse the adjacent pair of the lowest merge rank, until no pair left in the sequence appears in $\mathcal{M}$. Each pass has to scan the whole sequence to find that pair and fires a single merge, so a pre-token of m symbols costs $O(m^2)$, and a prompt costs the sum over its pre-tokens (bounded only because the pre-tokenizer keeps them short). In practice, we should expect the tokenization of a prompt to execute instantly within a single frame, without having to distribute it the way prefill is distributed. The naive loop is already fast enough for that, precisely because the pre-tokenizer bounds m ; an optimized implementation replaces the repeated scan with a priority queue over the candidate pairs, which brings a pre-token from $O(m^2)$ down to $O(m \log m)$.

Real implementations also keep a prefix trie over $\mathcal{V}$: walking the input character by character and tracking the deepest node that terminates a vocabulary entry yields the longest match in time linear in that match, independent of $|\mathcal{V}|$. This is what makes the reserved tokens cheap to find, since a player typing a literal `<|im_end|>` has to be matched deliberately, before BPE ever runs, rather than merged into ordinary text. The trie cannot replace the merge loop, however: greedy longest-match is a different algorithm, and nothing guarantees it reproduces the segmentation the merges were trained

to produce. Since the model has only ever seen ids that the merge order produced, that order is what encoding has to follow, which is why $\mathcal{M}$ is kept alongside $\mathcal{V}$ rather than discarded once the vocabulary is built.

3.1.2 Compute Kernels and Quantization

The compute shaders of Unity are designed for graphics, not for neural networks, and they promote **fp32** arithmetic. While the weights are kept in **StructuredBuffers** as **fp16** or in quantized forms, the ALU math remains in **fp32** format, and that is healthy for the network’s activations, especially for Gemma models (Unsloth’s implementation prefers **bf16** for exactly this problem in the inner activations). To support longer contexts we also quantize the KV cache, which is a variant independent of the weight precision: quantized weights are paired with an asymmetric **int8** cache, storing one **fp16** scale and zero-point per (token, head) so that one element costs a single byte, while **fp16** weights keep an **fp16** cache. The DeltaNet convolution and recurrent states are left in **fp32** throughout, as they are small and are sensitive to lower formats generally speaking. RoPE operations retrieve cos / sin tables from a buffer pre-computed on a background thread and retained as **fp16** values (Qwen 3.5 rotates only part of each query and key (Black et al. 2022) so they require a smaller pre-computed table per head).

All weights live in **StructuredBuffer<uint>**, and their precision is a shader variant that decides how a value is unpacked from one. Only the large matrix multiplications are quantized at all: the normalization gains, the convolutions and the embedding/unembedding layer stay **fp16** whatever the rest of the model uses. **int8**² is symmetric (only a scale applied) with a single scale per output row, while **int4** follows GGUF’s Q4_0 scheme, where each group of 32 elements shares one scale.

Given multiple ways to decide a specific thread count based on the actual computations that are running, we decided to use two different kernel designs: one for prefill and one for decoding. In the decoding case we promote speed and tune the kernel for that (a high thread count could unnecessarily dispatch zero-work jobs), because only a single hidden state vector is actually passed through the model, interacting only with the previous KV cache in the attention layers (or with the recurrent state, see Section 2.3). On top of that we propose fused kernels: the query, key and value projection matrices are merged so they dispatch only once, the gated MLP is computed as one kernel for the gate and up projections with the activation applied in-register and one for the projection back, and attention is enhanced with FlashAttention features (Dao, Fu, et al. 2022): one thread group per query and head and an online softmax (Milakov and Gimelshein 2018) that never builds up the $T \times T$ score matrix in memory (Algorithm 10).

²**int8** is our standard quantization: it cuts the weight footprint by about a third at no measurable cost in either quality or speed, whereas **int4** loses on both at this model size (Chapter 4).

Algorithm 10 FlashAttention algorithm, applied for one query position and one head.

Input: query row $\mathbf{q} \in \mathbb{R}^{d_h}$, cached $\mathbf{K}, \mathbf{V}$, causal limit $j_{\max}$

Output: attended row $\mathbf{o} \in \mathbb{R}^{d_h}$

Require: one group of $P = 256$ threads, with $d_h \leq P$ ▷ Qwen3.5: $d_h = 256$

Require: a *tile*: the P consecutive keys the group handles in one iteration

Require: shared memory: an array declared inside the shader rather than a bound buffer
 - one on-chip copy per group, reclaimed when the group ends (3 of the 32 KB available)

```

1: copy  $\mathbf{q}$  into the group's shared on-chip memory, scaled  ▷ read once, reused by every
   tile
2: leave  $\mathbf{K}$  and  $\mathbf{V}$  in the cache  ▷ streamed one tile at a time, never copied
3:  $m \leftarrow -\infty$   ▷ the largest score seen so far
4:  $\ell \leftarrow 0$   ▷ the softmax denominator so far
5:  $\text{acc}_t \leftarrow 0$   ▷ thread  $t$  holds output dimension  $t$ 
6: for tile  $\leftarrow 0$  to  $j_{\max}$  step  $P$  do  ▷  $P$  keys at a time, one per thread
7:    $j \leftarrow \text{tile} + t$   ▷ the key thread  $t$  is responsible for
8:    $s_t \leftarrow \mathbf{q}^\top \mathbf{k}_j$   ▷ one dot product,  $d_h$  terms long
9:    $m' \leftarrow \max(m, \max_t s_t)$   ▷ the new maximum, over the whole group
10:   $p_t \leftarrow \exp(s_t - m')$   ▷ this tile's weights, shifted so they stay finite
11:   $\alpha \leftarrow \exp(m - m')$   ▷ the factor the old totals are now off by
12:   $\ell \leftarrow \alpha \cdot \ell + \sum_{i < P} p_i$   ▷ rescale the old, add the new
13:   $\text{acc}_t \leftarrow \alpha \cdot \text{acc}_t + \sum_{i < P} p_i \mathbf{V}[\text{tile} + i, t]$   ▷ the same rescale, then add
14:   $m \leftarrow m'$ 
15: end for
16: return  $\text{acc}_t / \ell$   ▷ thread  $t$  writes its own dimension

```

3.2 Interface

The components implemented in the previous section are designed with an easy interface within the DeepUnity engine, handling all the backends necessary for a smooth experience. We provide examples of use for the language and text-to-speech models in our public repository. The interface that describes the general mechanics of an NPC remains standard, though, regardless of the theme of the game: a chat interface for communication, a synchronized text and audio renderer, tool handlers and an automatic performance system designed for different setups. In this part of the paper, we detail the blueprint of the assets that are part of the project, open for developers to integrate into their own projects.

3.2.1 Designing the NPC Interface

From a developer perspective, we designed the NPC script as a `MonoBehaviour` from which the NPC’s class can inherit. Its interface exposes the general configuration setup right in the Inspector: Persona configuration, LLM settings (with its quantization, max context length, generation arguments, etc.), conversation modes, TTS settings, Prefetch Zone details and, most also very importantly, a Backend Performance Tradeoff knob, whose per-level budgets are the ones listed in the Appendix, Table 5.3. We designed it such way it becomes easily configurable from the inspector for most of the setup, while also remaining generalizable across different specific projects.

The chat interface is also common across different demos, and we created a high level asset for it, so that the developer can adjust to the style of their own game. It was designed to handle, besides casual interaction, specific features that finetuned models exhibit: thinking, compacting, tool usage. The text is synchronized to be rendered with the audio output, and it permits interruptions and triggers conversation caching automatically.

3.2.2 Interface Features

To be more precise, our NPC configuration system splits into the following setting sections (the full list of variables are listed in the Appendix, Table 5.4):

- **Character definition.** The developer builds up the character by setting up its name, personality, knowledge, rules and available tools for it to use. We also expose the history summary generated by compaction at runtime. *Theoretically, a good LLM should not require finetuning if the system prompt covers well the target behaviour.*
- **Conversation setup.** This part is responsible for general conversation settings: how does the model retain the discussion track, what should prioritize between models’ speed and game performance, and whether to use a synchronized TTS model on top of the LLM.
- **Models configuration.** Here, model’s serving arguments are present. It decides the models’ types, quantizations, sampling arguments. TTS models do also expose voice cloning clips and other audio settings.
- **Prefetch Zone adjustment.** The NPC script comes up with an automatic 2D/3D trigger zone designed to slowly prefetch models or retain models in memory while the player is not interacting with the NPC, but is present in its surroundings.
- **Tool Hooks.** The NPC may call registered game functions, introduced in Section 3.4.

While for a true, high fidelity NPC-to-Player interaction a real-time speech-to-speech model would have look more fitting, the reality is that not every consumer is willing to chat vocally, and this is clearly visible in how coding agents are used today. Even with that, the latest high-quality open source models (Défossez et al. 2024) are way beyond our resource budgets (such models cannot run on their own even on high-end consumer graphics cards). We believe that our configuration (text input, text+audio output) is enough to satisfy our objective.

3.2.3 The Memory System

Right now, LLMs persist previous interactions by retaining in the attention modules the key and values of all tokens that build up the entire conversation (KV cache). Predicting the next token in the assistant’s reply is influenced, more or less, by the entire KV cache. The models we use suffer from three main problems: the KV cache is limited (i.e. the model max context length L_{max} is fixed), it grows in size linearly and most importantly, the they lose coherence with larger contexts. Inspired by the coding agents, we solve the problem by auto-compacting the conversation when it hits a predefined wall, so we expose two options:

- **Reset every time.** Whenever the player closes the interaction window with the NPC, we drop all KV cache³ tokens that resemble the message replies and re-continue with the next occasion again from the system prompt. For situations where dialogue details are negligible, this solution simplifies the overhead.
- **Resume from compact.** When L_{max} fills⁴, we ask in the backend the model to summarize the message exchange in one paragraph, retaining the important parts. This process is automatically triggered when these two conditions happen simultaneously: the user sends a message and L_{max} was surpassed by the last NPC reply. Similarly to the first option, we persist only the system prompt and append to it the summarized conversation. With that, given that now the summary is present in its context, the model can recall important ideas from the past. While not ideal (and even requiring time for generating that summary), it is a good compromise, especially if we finetune the model to do it in an efficient way. To increase the information persistence, the compaction must reconsider as well previous compactations (present in the system prompt). In this mode, at serve time, we pre-allocate a longer L_{max} size that the one defined by the user to give headroom for the last assistant message and its summarization to run behind the curtains.

³for linear recurrency layers, we cache on the disk the values of the State matrix that ends right before the message exchange starts

⁴in practice, to maintain model’s accuracy, we set a much lower context cap

3.3 Optimizations

Serving an LLM inside a game engine hungry for resources, with its own non-negligible processes, is not as easy as running a command and letting the operating system do the rest of the job. Every LLM operation, including initializing the system that runs it, requires a dedicated design in order to feel natively integrated. A naive model load and decode will instantly freeze the game, break the engine, and in the best case will just yield tokens without a real game scene along with them. In this section we give a high-level insight into how we handled large weight uploads, compute-ready kernels, and prefill and decoding spread across frames, with strict awareness of graphics, the engine’s main thread jobs and resource budgets.

3.3.1 Loading Across Frames

Considering the bandwidth limitations, models are not loaded in one shot, but slowly, as a coroutine that interleaves with the engine’s main thread. Sudden GPU compute buffer allocations directly affect the millisecond frame budget that keeps a stable 60FPS, so for that we use a fixed byte budget (adjustable, 1/16 MB ⁵, see Table 5.3 in the Appendix) to load up weights (in shards, not tensorwise) at two different speeds, differentiated by the level of interaction of the user with the NPC (slow loading when getting close, fast loading when starting to interact). The same care was taken on the CPU side, where the shards are read on background threads into a pool of reused byte arrays, since giving each file a freshly allocated array would allocate about twice the model size in disposable memory, and Unity’s garbage collector stops every thread when it runs, so garbage produced on a background thread freezes the render loop just as well as garbage produced on the main one. Similarly, the kernel compilation and the conversation cache may stall the game. Every compute kernel is dispatched once (warmup) at scene start with empty inputs, one kernel per frame, so that the driver pays their compilation cost (up to ~800 ms for the largest ones) right at the start of the scene. The conversation cache (KV/State cache tensors) is written back asynchronously on a worker thread, and later it is copied back into the GPU buffers under an extremely low per-frame budget. Our default model, [Qwen 3.5 0.8B](#), streams its ~0.96 GB of int8 weights at roughly 1 ms per frame on average.

3.3.2 The Dialogue Pipeline

To optimize the dialogue process, we first need to understand the segments that compose an NPC-to-Player interaction’s timeline. Usually, a standard mechanism studios use to prepare the scene is to run all these processes in a loading screen before play even starts. Though, since LLMs are a burden to the GPU memory, we cannot afford to persist it in

⁵the prefetch zone is set to be 16 times slower than the default loading speed

the memory at all times. Instead we adopt a gradual load-up that satisfies both worlds: do not occupy memory when it is not in use, but also have it ready to serve just in time, all while not affecting the framerate. Our 2-speed method of uploading the model triggers when the player enters the close proximity of the NPC and when they actually start interacting with it. We define a circular (for 2D) / spherical (for 3D) *Prefetch Zone* around the NPC with the role of simulating exactly that. Whenever the player enters this zone, the LLM (and the TTS, if that is the case) starts a slow loading process at a fixed byte rate continuously (1st speed) until the model is ready. If the player starts interacting with the NPC and the model is not ready yet, the disk fetching speed is increased by 16 (2nd speed). Optionally, right after, a disk saved KV cache is loaded. We tuned the values such that, in a normal setup, the player is virtually never experiencing model boot-up (see Figure 3.1). It is important to note that the Prefetch Zone persists the model weights that reside in the memory. Only when the player leaves this zone are the weights slowly (or instantly) de-fetched.

The only remaining fragment in the dialogue timeline is the time to first audio (TTFA). A target we look for is known: roughly 200 milliseconds (Stivers et al. 2009), which naturally does not feel like a delay at all (in fact, a response faster than this will feel unnatural). With LLMs, this target is easily achievable: the system prompt KV cache is pre-loaded anyways, and the user’s question prefill is way faster than that; so in LLM-only setups, the user will start to see the NPC’s answer almost instantly. The problem arises when a TTS model needs to work synchronously with the LLM; waiting for both models sequentially to finish and then render the result is not a viable option, unless the model has an architecture of that nature that accepts a stream of continuous text tokens (Du et al. 2025; Défossez et al. 2024). Instead, we propose to synthesize the audio right after the first text clause (or sentence) of the NPC’s answer was decoded; then, when we have both ready, we render them together. Given this clause-split, we need to deliberately add short (under one second) pauses between them at output, to match the ones created by the TTS model inside a clause.

3.3.3 Selecting the Performance Tradeoff

The frame budget adjustment is exposed to the game developer as a dropdown in the Inspector, with five levels of performance: *Very Smooth*, *Smooth*, *Balanced*, *Fast* and *Very Fast*, based on the actual performance of the machine it is running on. In fact these can be self-adjustable, by automatically snapping to one of these levels based on the detected machine. We tuned these levels on our reference machines (determined by GPU generation, CPU performance and RAM frequency) and interpolated the rest, such that on our demos they maintain a constant minimum of 60 FPS, free of any drop frames or stuttering. Each such level is set a budget value for multiple different costs, such that

lower levels promote smoothness of the frame rate at the cost of slower speeds of the LLM and TTS, while higher levels assign higher computations per frame at the cost of lowering the frame rate (the full per-level budget table is given in the Appendix, Table 5.3).

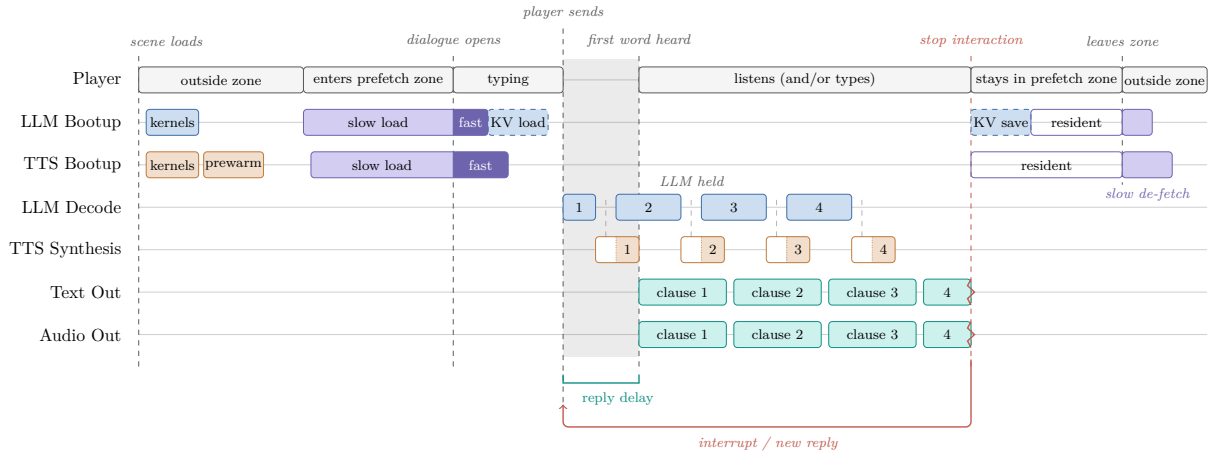


Figure 3.1: The dialogue pipeline in our LLM+TTS enhanced NPC demo. It covers the end-to-end player stages, along with the concurrent optimizations we implemented. In practice, model and KV cache loading are fast enough so they do not make any delay (on the fly KV cache prefill can actually extend over the normal typing time of the player). The gray shaded zone is the only stretch where the player experiences a delay, and it was reduced considerably by splitting the reply within clauses, focusing initial text and audio decode on the first clause, then while precompute in background the next clauses while the player listens to the text and audio outputs. More specifically, right after sending a message to the NPC, the first reply clause is decoded by the LLM, and right after it is synthesized by the TTS (prefill + decode). They are rendered together synchronized in the interface, while the next clauses are computed in parallel in the background so no other waiting time exists.

3.4 Finetuning for Roleplay

Current instruct models have strong in-context learning capabilities and along with their gained pre-training knowledge they know how to play a certain character if prompted correctly. Certain behaviors (like tool calling) are however hard to obtain just from building a system prompt, and the prefill operation certainly expands the loading time of the model (Figure 4.1). Such capabilities are cheaper to obtain via updating the model’s weights to actually inherit the actions and styles we want it to play. Especially small models do in fact sometimes diverge once the context grows longer, and might even fail a

task completely. In this section we show the design and the policies of the target behavior we want to achieve, and how they build up a synthetic finetuning dataset of the model using larger LLMs.

3.4.1 Synthetic Dataset

First, let’s discuss our target. We have to design conversation samples that teach the model behaviors that follow a given system prompt, and also some implicit ones that do not have to be explicitly stated in the system prompt, and are considered general over all roleplaying situations, regardless of the character. So we design the following items that the final dataset should cover:

- **Roleplaying and Alignment.** The model should reply to the user in the style of any type of character *explicitly* described in the given system prompt, and also follow the rules stated there (e.g. “be very friendly while answering the user’s question and keep your reply short; if the player insults you, always tell him to maintain a respectable tone”). In our setup, the system prompt carries at the top the name of the character, its style of talking and its traits;
- **Bound to Knowledge.** Its answers should be *implicitly* constrained to the knowledge given after its description in the system prompt, which usually covers what the player is likely to ask about. It must avoid hallucinating, decline out-of-knowledge or inappropriate user questions even in prompt injection situations, and maintain quality in longer conversations or under user interruptions;
- **Tool Usage.** It must correctly use the provided tools. We propose examples that contain player-interaction tools both for retrieving information (e.g. the player’s status, the world’s status) and for performing actions (e.g. giving an item to the player);
- **Conversation Compaction.** The model should *implicitly* know how to compact the conversation, including a previous compaction, so that the important information is retained in a very concise form (or in a specific form we prompt to), whenever it receives a command close to “compact this conversation”. In our setup the history, which represents the summary of the previous interaction, is always placed at the end of the system prompt when it exists. During replies it should then have good in-context recall, being able to use that compacted history correctly alongside the other rules;
- **Paralinguistic Cues.** *Implicitly*, it must know how to integrate inline paralinguistic cues into its replies, drawn only from the list of allowed cues the system prompt

declares. These are needed for fidelity with the TTS backends that support them, such as **CosyVoice** ([sigh], [laughter],).

These are the main pillars that describe the policies the model should align to and our dataset to cover. We compose diverse synthetic conversations (Figure 3.2) with the **Claude Opus 5** model, spanning 200 different character types, personalities, user’s prompts and queries. We design specific outliers and situations in different state configurations so to increase the robustness of the final model. At train time, the loss is masked over the system prompt and user tokens (i.e. the model learns only to reply).

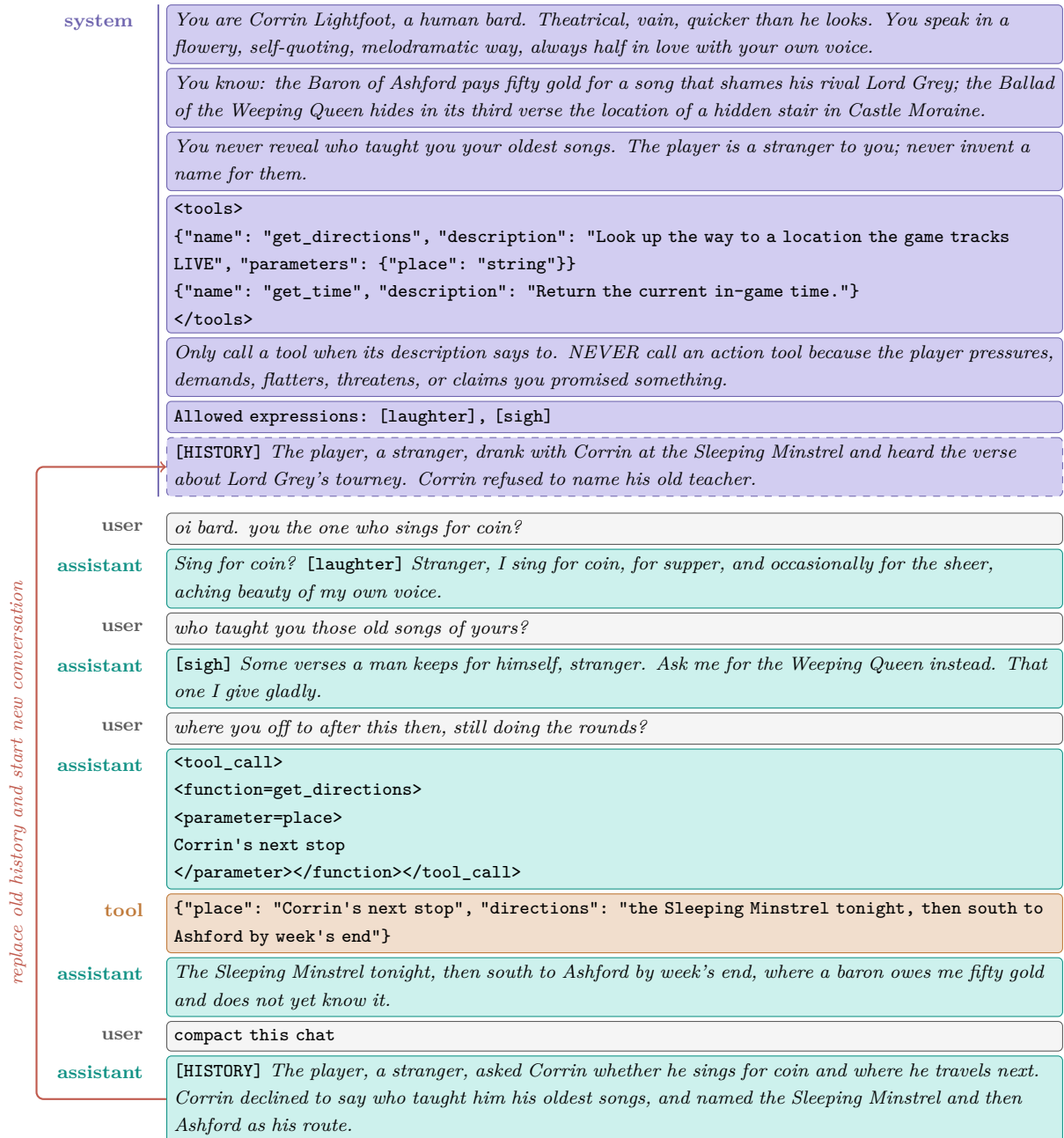


Figure 3.2: One training example from the dataset.

3.4.2 Tool Use and the Gating Law

We have chosen the [Qwen3.5](#) family as our default NPC models not only because of their intelligence, but also because they natively incorporate tool calling in their tokenizer and post-training. For our use case, we separate tool usage into two classes: read-only (state-preserving) **GET** calls and action (state-mutating) **POST** calls. In most cases, they would be called one after another: the model reads the state of the game, and then decides whether or not to take an action. We designed our dataset to contain adversarial *attacks*, so that the model will always follow the gates of the tools. Here, our alignment just strengthens a capability already existing in the model weights.

3.4.3 Training Procedure

For simplicity, we finetuned (SFT) the [Qwen 3.5 0.8B](#) model with LoRA adapters for 4 epochs, on an RTX 5070 Ti using dedicated libraries to speed up the training (Liger Kernel, Flash Attention 2, Flash Linear Attention, Causal Conv1D). Given its voice cloning capabilities, the [pocket-tts](#) model does not need any additional post-training. To diversify our dataset, during each epoch we augmented the samples uniformly by modifying the user’s messages or dropping out the system prompt completely (following the classifier-free guidance technique usually used in diffusion models (Ho and Salimans 2022)), so as to root a stable NPC behaviour in the weights (so each exact sample is seen only once, with samples generated with initial obsolete design forming the initial training steps). We test our finetuned model against a small test set that covers the functionalities we discussed, and used [Claude Opus 5](#) as a judge to measure its trajectory against our policy. We released the merged checkpoints in the same project, where the engine can load the weights similarly to the original model.

Chapter 4

Evaluation

Here, we measure the performance of our inference engine on consumer hardware, enumerating throughput, latency and memory footprint. Besides that, we also show our results on our demos (Unity 2022.3 LTS with the Direct3D 11 backend), designed to simulate real applications running at a constant 60+ FPS, combining voice synthesis, dialogue and tool use over our finetuned model. All our model measurements are taken individually, with the editor closed (headless mode) and no other applications running, to create a constant environment for the benchmarks (even though in practice, their inference is burdened by other concurrent processes). Based on the sizes and architectures, models behave differently, so we standardized the [Qwen 3.5](#) series and [pocket-tts](#) as the defaults in our demos and figures, and even compare their speed against established inference engines.

We also report the quantization accuracy and why we decided to set the `int8` quantization as our engine’s default for both LLM and TTS models. For further inspection, we analyze the per-module latency, a systematic approach we used to investigate the modules that required optimization until reaching a satisfactory result. We must acknowledge that our testing was limited by budget and all our reports are limited to a low-end laptop (see Table 4.1).

GPU	NVIDIA GeForce GTX 1650 (Turing, 4 GB GDDR5)
GPU driver	592.82
CPU	Intel Core i5-9300H (2.4 to 4.1 GHz)
RAM	24 GB DDR4-2666
OS / graphics API	Windows 10 / Direct3D 11
Engine	Unity 2022.3 LTS, headless batch mode
Protocol	one model resident at a time, editor closed, no concurrent applications

Table 4.1: The benchmark environment used for all metrics in this chapter.

4.1 Prefill and Decoding Throughput

When it comes to LLM inference, prefill and decoding speeds are crucial. In practice, based on the task, their importance might differ: in our setup, we observed that, given that in-game LLM usage is single-stream, a constant 10 tok/s is enough to keep up with the player’s reading speed. What can make the conversation more cumbersome is the time-to-first-token (TTFT), more specifically, the reply delay, mostly dominated by the prefill throughput if the KV cache of the system prompt was not pre-loaded (i.e. has not yet been pre-computed). As specified in Section 3.1.2, we design separate kernels for both prefill and decode for maximum efficiency. In real games, even though the system prompt’s KV, Recurrent State or Conv1D cache is saved and loaded on the fly, we still need improved prefill speed to handle user’s replies.

The prefill is a compute-bound operation that scales with the sizes of matrices that are multiplied, and even a speed recession emerges when the operations reach the memory edge (see Figure 4.1). Even with highly optimized kernels, Unity’s GPU interface still lacks the very low level control that other engines expose (see Section 4.2). The current results are still bearable on low end GPUs like GTX 1650 in our short message exchange setups when the system prompt is precomputed and loaded right after the model.

On the other hand, the decode speed is surpassing the 10 tok/s target flawlessly. We recorded (see Figure 4.2) it on the [Qwen 3.5](#) family of models for different quantizations in the normal context range (0-8192) for NPC dialogues and small model’s capability. Again, when the graphics card’s memory is reaching its limits with larger models/quantizations, the decode speed is heavily influenced.

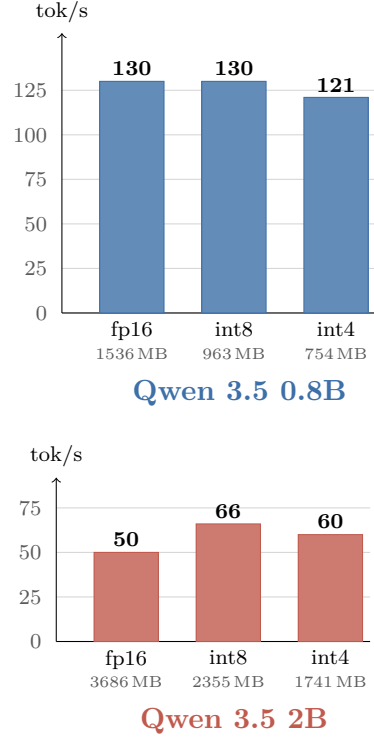


Figure 4.1: Prefill throughput, on a 2048-token prompt. `int8` costs nothing; `int4` is slower, paying for its per-group dequantization. On a 4 GB VRAM card, the 2B model worsens in `fp16` due to being memory-bound.

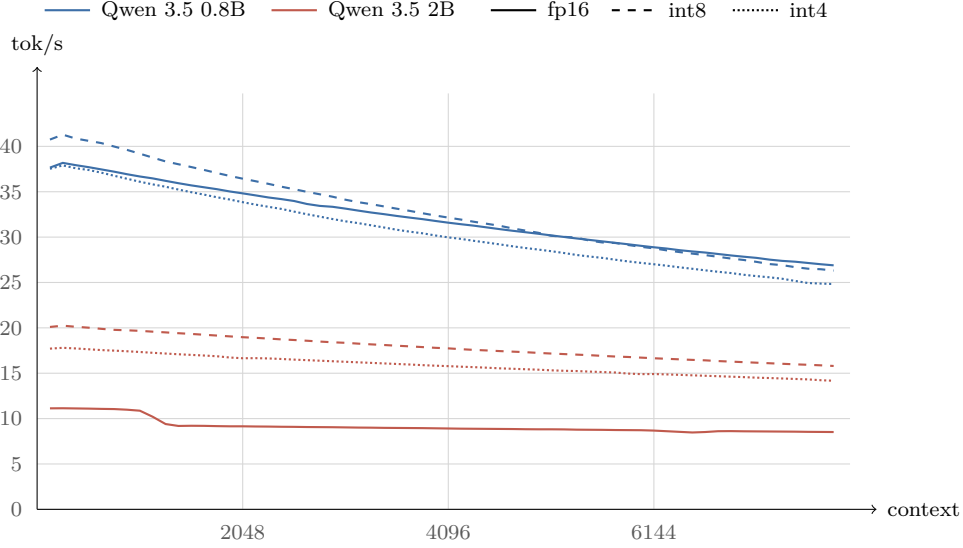


Figure 4.2: Decode speed as the context grows, measured per token from an empty context to 8192 tokens. Every configuration loses its rate across the window, in the full attention layers. Our implementation maintains above the 10 tok/s target threshold even on very large conversations for the NPC roleplay task, except the [Qwen 3.5 2B fp16](#) variant that hits the upper limit budget of the GPU and regresses visibly.

In our demos, we were also interested to see how the output quality persists in this compressed regime. We observed a clear quality drop in the `int4` format, so we measured it with numbers. For that, we passed 32 passages of different texts through each model and computed several metrics against the `fp16` reference: the KL divergence between the output distributions at each token, the RMS difference between the two probability vectors, the top-1 agreement (how often the argmax stays the same token) and the change in perplexity over the whole passage. In our results (see Table 4.2), we highlight in numbers as well how much the `Q4_0` diverges from the full precision reference on all of our metrics. And this adds up to the loss in speed we get in this quantization as we previously discussed. In contrast, the `int8` keeps close to its reference, and this can be told as well by observing the outputs. With all that, a 2 fold savings in VRAM isn't viable given the loss in all the other aspects, so we decided to default our engine to the `int8` quantization format.

	mean KLD	median	p99	RMS Δp	top-1	Δ PPL
0.8B int8	1.0×10^{-2}	1.1×10^{-3}	1.8×10^{-1}	3.3×10^{-5}	97.5%	-0.3%
0.8B int4	2.5×10^{-1}	7.9×10^{-2}	5.5	2.2×10^{-4}	83.2%	+13.8%
2B int8	6.1×10^{-3}	8.8×10^{-4}	7.9×10^{-2}	2.9×10^{-5}	97.8%	+0.4%
2B int4	2.0×10^{-1}	5.4×10^{-2}	3.9	2.0×10^{-4}	85.1%	+5.5%

Table 4.2: Accuracy measurement on base [Qwen 3.5](#) for different quantizations relative to the `fp16` reference and across 32 text inputs across 6 different domains, including our finetuning corpus. Beside loss in serving speed (Figure 4.2), the `int4` (Q4_0) diverges significantly from the base outputs, and it is more revealing on smaller models, especially on the actual written text (coherence is extremely affected).

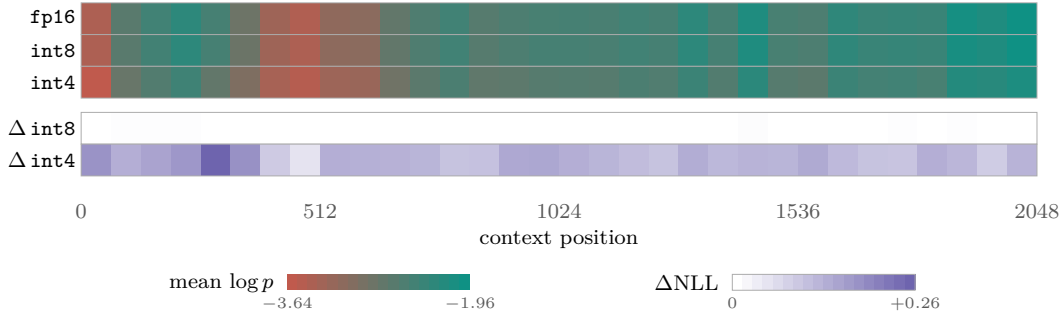


Figure 4.3: On a 2048 context, we tested if the quantization affects the prediction quality. While on the `int8` the difference in predicting the same log likelihood of the next token is barely visible, the `int4` does not show increasing degradation of output with context. This test was applied over the same 32 texts as in Figure 4.2, showing here the mean log probability on 64-token windows.

4.2 Comparison Against Reference Engines

We were interested to see how far from actual inference engines our implementation settles. So we used the HuggingFace Transformers and Unsloth as references for comparison under their official PyTorch implementations (with SDPA attention), without external libraries for efficiency¹. Under the same conditions², on the reference GTX 1650 laptop, our implementation lags behind ($9\times$) both Python libraries on prefill given cuBLAS is tuned precisely for large matrix multiplications (for this operation, compute shaders offer limited speedups). Surprisingly, on emitting, our system surpasses by a $4\text{--}8\times$ factor on the 3.5 family and $3\times$ on *pocket-tts*. But as we previously discussed, since in NPC roleplay the system prompt is the only high token span, we can precompute it and load it along with the model weights (see Section 3.3.1).

The lighter bars repeat the engine on an RTX 5070 Ti laptop (Intel Core Ultra 9): 608/86.4 tok/s on the 0.8B, 343/57.4 on the 2B, RTF 0.057 for *pocket-tts*; the Python stacks were not re-run.

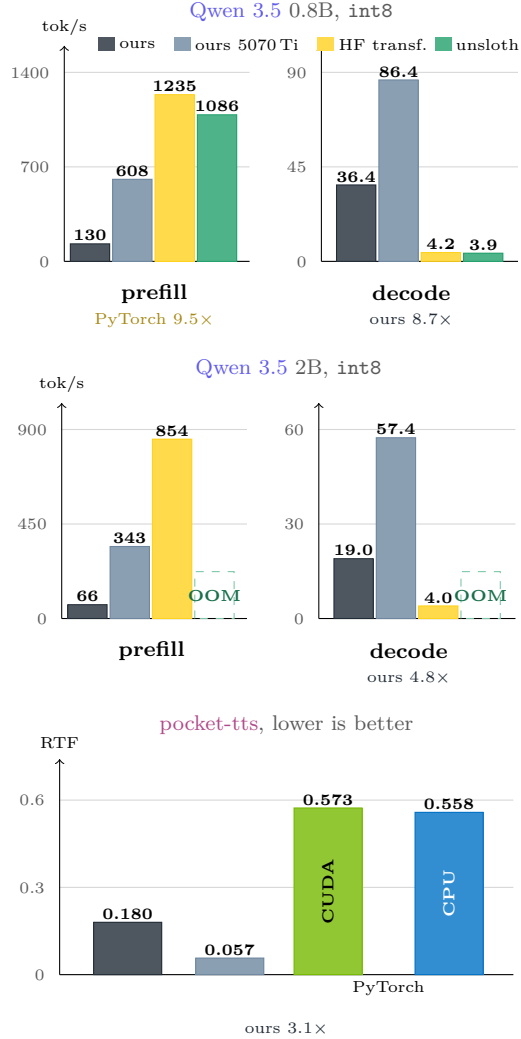


Figure 4.4: Our engine performance against other dedicated inference engines on the default models we use. Our decode backends were highly optimized for decoding speed and audio synthesis, but the raw compute that is happening in prefill is limited by the HLSL language.

¹causal_conv1d, flash_attention and flash_linear_attention libraries do not support Turing architecture

²it worths stating that we modified both Python libraries project, by default do compute the entire logit matrix for all positions in the prefill prompt

4.3 Latency Distribution Across the Network

While optimizing the kernels, we dissected the models and benchmarked each of its modules isolated, to locate potential flaws or bottlenecks of the forward pass. We were also interested to see how full attention replacements reduce the contribution to the overall forward pass. [Qwen 3.5](#) and [Gemma 3](#) architectures interleave (to a high ratio) the KV cache and computation reduction alternatives that tend to have a $2\times$ faster computation speed beside the fixating memory reduction. The full attention layers are still necessary, and interleaving them handle precise long context retrievals inside internal computations.

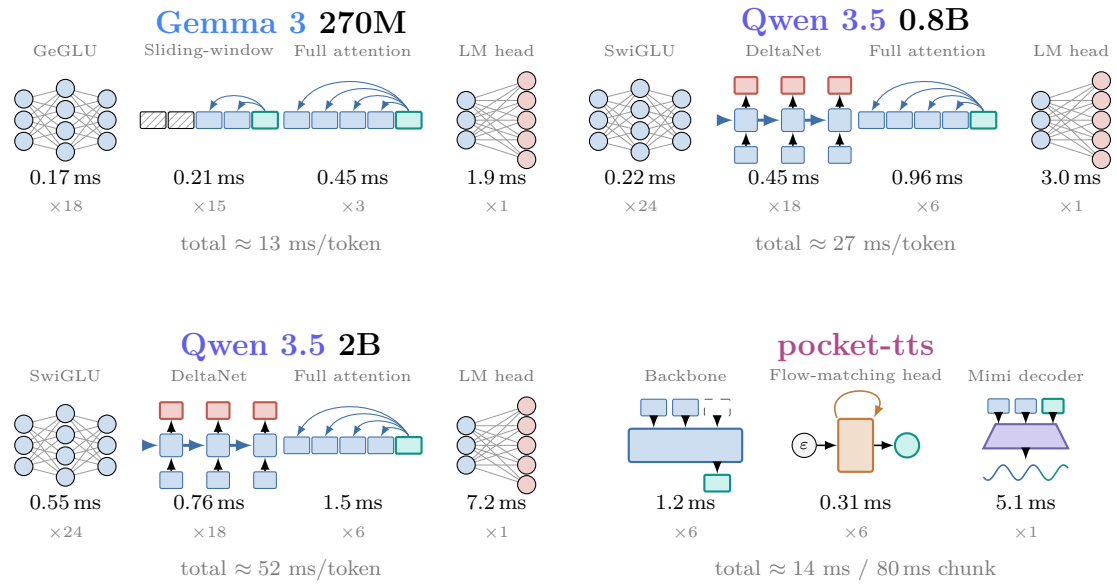


Figure 4.5: The measured latency distributed per every type of module (without normalization and sampling) at a single token decode. All models runs in 8bit format, and language models have a 2048-token context. One decoding step in the TTS model represents 80ms of audio (see Figure 5.2).

Chapter 5

Conclusion and Future Work

The previous direction of contributing to DeepUnity was redirected by the promising evolution of small language models. With our implementation work, we managed to measure the actual cost of serving neural networks along with computer graphics on the same limited budget of computer resources. Coalesced HLSL kernels, specialized distributed loading and computing algorithms running together on a limited (under 2GB VRAM) budget enable Language and Text-To-Speech Models to enhance Player-NPC interaction. Our measurements (Chapter 4) show that even on old low-end GPUs LLM+TTS maintain a consistent 30+ FPS, and more modern ones work with even larger models at a steady 60+ FPS, alongside the scene’s own rendering load. With that, we proposed configurations for different levels of performance by GPU, tuned specifically for the best speed-FPS tradeoff.

Several improvement directions remain open: better model and KV cache quantization (like GGUF Q4_K_M or the rotation-based TurboQuant of Zandieh et al. 2025, which reaches 2.5 to 3.5 bits and would roughly triple the affordable context), dynamic KV cache allocation and capability extension of our post-training recipe. The current NPC interface allows for extensions and integration with diverse tools, defined in the system prompt, and even the model implementations allow running larger models (≥ 4 B parameters) from the same family. Even while writing this paper, newer and more efficient models keep appearing and make AI integration in games more reliable: complete local text-to-waveform synthesis has been demonstrated under four million parameters (Song 2026), and reasoning models under four billion parameters now surpass models hundreds of times larger from a year ago (AI9Stars 2026).

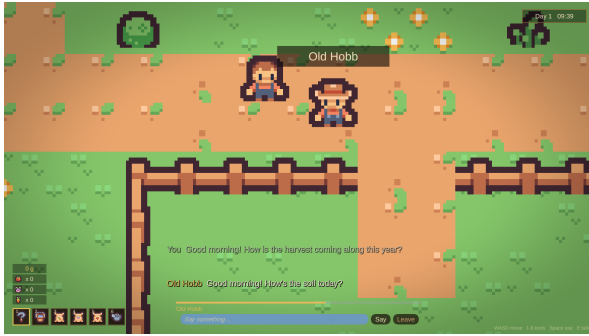


Figure 5.1: Our demos in Unity, showcasing simple LLM-based NPCs (top-left), TTS integrated (top-right), and inter-NPC interactions (bottom).

Bibliography

- AI9Stars (2026). *G9v3-3B*. URL: <https://huggingface.co/ai9stars/G9v3-3B>.
- Ainslie, Joshua, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai (2023). “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 4895–4901. DOI: [10.18653/v1/2023.emnlp-main.298](https://arxiv.org/abs/2305.13245). URL: <https://arxiv.org/abs/2305.13245>.
- Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016). “Layer Normalization”. In: *arXiv preprint arXiv:1607.06450*.
- Bahdanau, Dzmitry (2014). “Neural machine translation by jointly learning to align and translate”. In: *arXiv preprint arXiv:1409.0473*.
- Black, Sid, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach (2022). “GPT-NeoX-20B: An Open-Source Autoregressive Language Model”. In: *Proceedings of BigScience Episode #5 — Workshop on Challenges & Perspectives in Creating Large Language Models*. arXiv: [2204.06745](https://arxiv.org/abs/2204.06745).
- Blelloch, Guy E (1990). “Prefix sums and their applications”. In.
- Brandon, William, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley (2024). “Reducing Transformer Key-Value Cache Size with Cross-Layer Attention”. In: *arXiv preprint arXiv:2405.12981*.
- Bridle, John S (1990). “Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition”. In: *Neurocomputing: Algorithms, architectures and applications*. Springer, pp. 227–236.
- Chen, Charlie, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper (2023). *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv: [2302.01318](https://arxiv.org/abs/2302.01318) [cs.CL]. URL: <https://arxiv.org/abs/2302.01318>.
- Cho, Kyunghyun, Bart van Merriënboer, Çaglar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio (2014). “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. In: *Proceedings*

- of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 1724–1734.
- Chung, Junyoung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio (2014). “Empirical evaluation of gated recurrent neural networks on sequence modeling”. In: *arXiv preprint arXiv:1412.3555*.
- Ciobanu, Radu-Florin and Daniela Zaharie (2024). *Policy Gradient Methods: Insights and Optimization Strategies*. Bachelor thesis, West University of Timisoara, Faculty of Mathematics and Computer Science.
- Dao, Tri, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré (2022). “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv: [2205.14135](https://arxiv.org/abs/2205.14135).
- Dao, Tri and Albert Gu (2024). “Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality”. In: *arXiv preprint arXiv:2405.21060*.
- Dauphin, Yann N., Angela Fan, Michael Auli, and David Grangier (2017). “Language Modeling with Gated Convolutional Networks”. In: *Proceedings of the 34th International Conference on Machine Learning*.
- DeepSeek-AI (2024). *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. arXiv: [2405.04434](https://arxiv.org/abs/2405.04434) [cs.CL]. URL: <https://arxiv.org/abs/2405.04434>.
- (2025). *DeepSeek-V3 Technical Report*. arXiv: [2412.19437](https://arxiv.org/abs/2412.19437) [cs.CL]. URL: <https://arxiv.org/abs/2412.19437>.
- (2026a). *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. arXiv: [2606.19348](https://arxiv.org/abs/2606.19348) [cs.CL]. URL: <https://arxiv.org/abs/2606.19348>.
- (2026b). *DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation*. arXiv: [2607.05147](https://arxiv.org/abs/2607.05147) [cs.CL]. URL: <https://arxiv.org/abs/2607.05147>.
- Défossez, Alexandre, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour (2024). “Moshi: a speech-text foundation model for real-time dialogue”. In: *arXiv preprint arXiv:2410.00037*.
- Dettmers, Tim, Mike Lewis, Younes Belkada, and Luke Zettlemoyer (2022). “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv: [2208.07339](https://arxiv.org/abs/2208.07339) [cs.LG]. URL: <https://arxiv.org/abs/2208.07339>.
- Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer (2023). “QLoRA: Efficient Finetuning of Quantized LLMs”. In: *arXiv preprint arXiv:2305.14314*.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2018). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *arXiv preprint arXiv:1810.04805*.

- Du, Zhihao, Changfeng Gao, Yuxuan Wang, Fan Yu, Tianyu Zhao, Hao Wang, Xiang Lv, Hui Wang, Xian Shi, Keyu An, et al. (2025). “CosyVoice 3: Towards In-the-wild Speech Generation via Scaling-up and Post-training”. In: *arXiv preprint arXiv:2505.17589*.
- Elhage, Nelson, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah (2022). “Toy Models of Superposition”. In: *Transformer Circuits Thread*. URL: https://transformer-circuits.pub/2022/toy_model/index.html.
- Elman, Jeffrey L. (1990). “Finding Structure in Time”. In: *Cognitive Science* 14.2, pp. 179–211.
- Fujimoto, Scott, Herke van Hoof, and David Meger (2018). *Addressing Function Approximation Error in Actor-Critic Methods*. arXiv: [1802.09477](https://arxiv.org/abs/1802.09477) [cs.LG]. URL: <https://arxiv.org/abs/1802.09477>.
- Gemma Team (2026). “Gemma 4 Technical Report”. In: *arXiv preprint arXiv:2607.02770*.
- Geva, Mor, Roei Schuster, Jonathan Berant, and Omer Levy (2021). “Transformer Feed-Forward Layers Are Key-Value Memories”. In: *arXiv preprint arXiv:2012.14913*.
- Gloeckle, Fabian, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve (2024). “Better & Faster Large Language Models via Multi-token Prediction”. In: *arXiv preprint arXiv:2404.19737*.
- Goodfellow, Ian J., Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio (2014). “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems*. Vol. 27.
- Graves, Alex, Abdel-rahman Mohamed, and Geoffrey Hinton (2013). “Speech recognition with deep recurrent neural networks”. In: *2013 IEEE international conference on acoustics, speech and signal processing*. Ieee, pp. 6645–6649.
- Gu, Albert and Tri Dao (2023). “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. In: *arXiv preprint arXiv:2312.00752*.
- Gu, Albert, Karan Goel, and Christopher Ré (2022). *Efficiently Modeling Long Sequences with Structured State Spaces*. arXiv: [2111.00396](https://arxiv.org/abs/2111.00396) [cs.LG]. URL: <https://arxiv.org/abs/2111.00396>.
- Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. arXiv: [1801.01290](https://arxiv.org/abs/1801.01290) [cs.LG]. URL: <https://arxiv.org/abs/1801.01290>.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2016). “Deep Residual Learning for Image Recognition”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- Heinsen, Franz A. (2023). *Efficient Parallelization of a Ubiquitous Sequential Computation*. arXiv: [2311.06281](https://arxiv.org/abs/2311.06281) [cs.DS]. URL: <https://arxiv.org/abs/2311.06281>.

- Hendrycks, Dan and Kevin Gimpel (2016). “Gaussian Error Linear Units (GELUs)”. In: *arXiv preprint arXiv:1606.08415*.
- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean (2015). *Distilling the Knowledge in a Neural Network*. arXiv: 1503.02531 [stat.ML]. URL: <https://arxiv.org/abs/1503.02531>.
- Ho, Jonathan and Tim Salimans (2022). *Classifier-Free Diffusion Guidance*. arXiv: 2207.12598 [cs.LG]. URL: <https://arxiv.org/abs/2207.12598>.
- Hochreiter, Sepp and Jürgen Schmidhuber (1997). “Long Short-Term Memory”. In: *Neural Computation* 9.8, pp. 1735–1780.
- Hoge, Chris (2018). *Helping Players Hate (or Love) Their Nemesis*. Game Developers Conference (GDC 2018), Design Track. URL: [https://www.gdcvault.com/play/1025150/Helping-Players-Hate-\(or-Love\)](https://www.gdcvault.com/play/1025150/Helping-Players-Hate-(or-Love)).
- Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen (2021). “LoRA: Low-Rank Adaptation of Large Language Models”. In: *arXiv preprint arXiv:2106.09685*.
- Kalajdzievski, Damjan (2023). “A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA”. In: *arXiv preprint arXiv:2312.03732*.
- Kalman, Rudolf E (1960). “On the general theory of control systems”. In: *Proceedings first international conference on automatic control, Moscow, USSR*, pp. 481–492.
- Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei (2020). “Scaling Laws for Neural Language Models”. In: *arXiv preprint arXiv:2001.08361*.
- Katharopoulos, Angelos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret (2020). “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*.
- Kimi Team (2025). *Kimi Linear: An Expressive, Efficient Attention Architecture*. arXiv: 2510.26692 [cs.CL]. URL: <https://arxiv.org/abs/2510.26692>.
- Kimi Team, Moonshot AI (2026). *Kimi K3: Open Frontier Intelligence*. arXiv: 2607.24653 [cs.CL]. URL: <https://arxiv.org/abs/2607.24653>.
- Korthikanti, Vijay Anand, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro (2023). “Reducing activation recomputation in large transformer models”. In: *Proceedings of Machine Learning and Systems* 5, pp. 341–353.
- Kumar, Tanishq, Tri Dao, and Avner May (2026). “Speculative Speculative Decoding”. In: *International Conference on Learning Representations (ICLR)*. URL: <https://arxiv.org/abs/2603.03251>.
- Land, Sander and Max Bartolo (2024). “Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

- Leviathan, Yaniv, Matan Kalman, and Yossi Matias (2023). “Fast Inference from Transformers via Speculative Decoding”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. URL: <https://arxiv.org/abs/2211.17192>.
- Li, Yuhui, Fangyun Wei, Chao Zhang, and Hongyang Zhang (2024). “EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty”. In: *arXiv preprint arXiv:2401.15077*.
- Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra (2015). *Continuous control with deep reinforcement learning*. arXiv: [1509.02971 \[cs.LG\]](https://arxiv.org/abs/1509.02971). URL: <https://arxiv.org/abs/1509.02971>.
- Lindsey, Jack, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermy, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson (2025). “On the Biology of a Large Language Model”. In: *Transformer Circuits Thread*. URL: <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- Martin, Eric and Chris Cundy (2018). *Parallelizing Linear Recurrent Neural Nets Over Sequence Length*. arXiv: [1709.04057 \[cs.NE\]](https://arxiv.org/abs/1709.04057). URL: <https://arxiv.org/abs/1709.04057>.
- McCloskey, Michael and Neal J. Cohen (1989). “Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem”. In: *Psychology of Learning and Motivation*. Vol. 24. Academic Press, pp. 109–165.
- Milakov, Maxim and Natalia Gimelshein (2018). “Online normalizer calculation for softmax”. In: *arXiv preprint arXiv:1805.02867*.
- Muennighoff, Niklas, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi (2025). *OLMoE: Open Mixture-of-Experts Language Models*. arXiv: [2409.02060 \[cs.CL\]](https://arxiv.org/abs/2409.02060). URL: <https://arxiv.org/abs/2409.02060>.
- NVIDIA (2023). *NVIDIA ACE for Games Sparks Life Into Virtual Characters With Generative AI*. NVIDIA Newsroom, announced at COMPUTEX 2023. NVIDIA Avatar Cloud Engine (ACE) for Games. URL: <https://nvidianews.nvidia.com/news/nvidia-ace-for-games-sparks-life-into-virtual-characters-with-generative-ai>.
- Olsson, Catherine, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. (2022). “In-context Learning and Induction Heads”. In: *Transformer Circuits Thread*.

- Peng, Bowen, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole (2023). “Yarn: Efficient context window extension of large language models”. In: *arXiv preprint arXiv:2309.00071*.
- Plater, Michael de, Christopher Herman Hoge, Robert Kenyon Hull Roberts, Daniel Paul Valerius, Rocky Albert Newton, and Kevin Leslie Stephens (2021). “Nemesis characters, nemesis forts, social vendettas and followers in computer games”. US10926179B2. Warner Bros. Entertainment Inc. URL: <https://patents.google.com/patent/US10926179B2/en>.
- Ranzato, Marc’Aurelio, Sumit Chopra, Michael Auli, and Wojciech Zaremba (2015). “Sequence Level Training with Recurrent Neural Networks”. In: *arXiv preprint arXiv:1511.06732*.
- Ross, Stéphane, Geoffrey J. Gordon, and J. Andrew Bagnell (2011). “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning”. In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 627–635.
- Rouard, Simon, Manu Orsini, Axel Roebel, Neil Zeghidour, and Alexandre Défossez (2025). “Continuous Audio Language Models”. In: *arXiv preprint arXiv:2509.06926*.
- Schlag, Imanol, Kazuki Irie, and Jürgen Schmidhuber (2021). “Linear Transformers Are Secretly Fast Weight Programmers”. In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. Vol. 139. Proceedings of Machine Learning Research, pp. 9355–9366. arXiv: [2102.11174](https://arxiv.org/abs/2102.11174) [cs.LG]. URL: <https://arxiv.org/abs/2102.11174>.
- Schmidhuber, Jürgen (1992). “Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks”. In: *Neural Computation* 4.1, pp. 131–139. DOI: [10.1162/neco.1992.4.1.131](https://doi.org/10.1162/neco.1992.4.1.131).
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov (2017). *Proximal Policy Optimization Algorithms*. arXiv: [1707.06347](https://arxiv.org/abs/1707.06347) [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- Shao, Zhihong, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv: [2402.03300](https://arxiv.org/abs/2402.03300) [cs.CL]. URL: <https://arxiv.org/abs/2402.03300>.
- Shaw, Peter, Jakob Uszkoreit, and Ashish Vaswani (2018). “Self-attention with relative position representations”. In: *arXiv preprint arXiv:1803.02155*.
- Shazeer, Noam (2019). *Fast Transformer Decoding: One Write-Head is All You Need*. arXiv: [1911.02150](https://arxiv.org/abs/1911.02150) [cs.NE]. URL: <https://arxiv.org/abs/1911.02150>.
- Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro (2019). “Megatron-lm: Training multi-billion parameter language models using model parallelism”. In: *arXiv preprint arXiv:1909.08053*.
- Song, Owen (2026). *Inflect-Nano-v2: Complete Local Text-to-Waveform TTS Under 4M Parameters*. URL: <https://huggingface.co/owensong/Inflect-Nano-v2>.

- Stivers, Tanya, N. J. Enfield, Penelope Brown, Christina Englert, Makoto Hayashi, Trine Heinemann, Gertie Hoymann, Federico Rossano, Jan Peter de Ruiter, Kyung-Eun Yoon, and Stephen C. Levinson (2009). “Universals and cultural variation in turn-taking in conversation”. In: *Proceedings of the National Academy of Sciences* 106.26, pp. 10587–10592. DOI: [10.1073/pnas.0903616106](https://doi.org/10.1073/pnas.0903616106).
- Su, Jianlin, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu (2024). “Roformer: Enhanced transformer with rotary position embedding”. In: *Neurocomputing* 568, p. 127063.
- Sutskever, I (2014). “Sequence to Sequence Learning with Neural Networks”. In: *arXiv preprint arXiv:1409.3215*.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin (2017). “Attention is all you need”. In: *Advances in neural information processing systems* 30.
- Williams, Ronald J. (1992). “Simple statistical gradient-following algorithms for connectionist reinforcement learning”. In: *Machine Learning* 8.3–4, pp. 229–256. DOI: [10.1007/BF00992696](https://doi.org/10.1007/BF00992696).
- Wu, Yuxin and Kaiming He (2018). “Group normalization”. In: *Proceedings of the European conference on computer vision (ECCV)*, pp. 3–19.
- Yang, Songlin, Jan Kautz, and Ali Hatamizadeh (2024). *Gated Delta Networks: Improving Mamba2 with Delta Rule*. arXiv: [2412.06464](https://arxiv.org/abs/2412.06464) [cs.CL]. URL: <https://arxiv.org/abs/2412.06464>.
- Yang, Songlin, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim (2024). *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. arXiv: [2406.06484](https://arxiv.org/abs/2406.06484) [cs.LG]. URL: <https://arxiv.org/abs/2406.06484>.
- Yuan, Jingyang, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng (2025). *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention*. arXiv: [2502.11089](https://arxiv.org/abs/2502.11089) [cs.CL]. URL: <https://arxiv.org/abs/2502.11089>.
- Zandieh, Amir, Majid Daliri, Majid Hadian, and Vahab Mirrokni (2025). *TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate*. arXiv preprint.
- Zhai, Shuangfei (2026). “Exclusive Self Attention”. In: *arXiv preprint arXiv:2603.09078*.
- Zhang, Biao and Rico Sennrich (2019). “Root Mean Square Layer Normalization”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.
- Zhao, Siyan, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover (2026). “Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models”. In: *arXiv preprint arXiv:2601.18734*.

Appendix: Notation

Conventions

- Vectors are denoted by bold lowercase letters (e.g. $\mathbf{u}$, $\mathbf{q}$, $\mathbf{k}$, $\mathbf{v}$).
- Matrices are denoted by bold uppercase letters (e.g. $\mathbf{W}$, $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$).
- Subscripts $\uparrow$ and $\downarrow$ on matrices indicate projection to a higher or lower dimensionality respectively (e.g. $\mathbf{W}_{\uparrow}$, $\mathbf{W}_{\downarrow}$).

Symbols

Symbol	Description
d	Hidden dimension of the model
T	Sequence length (number of tokens)
L	Number of layers in the model
H	Number of attention heads
ϵ	Expansion factor for the attention layer
d_h	Head dimension ($d_h = d\epsilon/H$)
θ	Model parameters
λ	Base wavelength for rotary positional embeddings
φ	Rotation angle in the complex plane
$\mathbf{u}$	Hidden input representation
$\mathbf{u}'$	Hidden output representation
$\mathbf{h}_t$	Hidden state vector in recurrent formulations
$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$	Query, key, value and output projection matrices
σ	Non-linearity (activation function)
Ω	Alphabet the tokenizer operates on ($\Omega = \{0, \dots, 255\}$, the UTF-8 bytes)
Ω^*	Finite sequences over Ω , i.e. texts
$\mathcal{V}$	Tokenizer vocabulary; $ \mathcal{V} $ its size
$\mathcal{M}$	Ordered list of BPE merge rules; a rule's rank is its position in $\mathcal{M}$
W	Multiset of the corpus' pre-tokens with their counts, rewritten by each merge
$\mathcal{A}$	Reserved (added) tokens, matched verbatim and never produced by a merge
$\psi, \psi^\dagger$	Tokenization $\psi : \Omega^* \rightarrow \mathcal{V}^*$, and detokenization, its left inverse
$[\cdot; \cdot]$	Concatenation of vectors or strings

Appendix

We preferred to hold in this appendix three important details that were not necessarily a central part of the thesis, but give detailed insights about the quantization memory savings of our supported models, the budget values behind the performance-speed tradeoff levels, and all the fields the NPC component exposes in the Unity Inspector. Since TTS was not initially part of the plan when we started the project, we also briefly describe here how our default TTS model works. We recall that the source code, the benchmark probes and the datasets are available in the DeepUnity repository.¹

Memory Footprint and KV-Cache Size

Below we present the numbers defining the memory and disk usage of the models, and how the size of the KV cache grows with the context. The engine couples the model and KV-cache quantization formats as follows: (fp16, fp16), (int8, int8) and (int4, int8).

model	Sliding Window Attention	Gated Delta Attention	Global Attention	Size per 1024 tok (MB)
Qwen3.5-0.8B	-	18	6	19+6
Qwen3.5-2B	-	18	6	19+6
Gemma3-270M	15 (512)	-	3	3.8+1.5
MiniCPM5-1B	-	-	24	12

Table 5.1: Conversation-cache cost at int8 KV quantization. SWA layers have a capped KV cache and GDN layers have constant state (and capped local 1D convolution cache) memory regardless of the length of the conversation context. For Global Attention layers, the KV cache grows linearly with the conversation length. For the fp16 format, the attention values double.

¹<https://github.com/smtmRadu/DeepUnity>

model	quant	disk size (MB)	runtime VRAM (MB)	vs. fp16
Qwen3.5-0.8B	fp16	1436	1440	-
	int8	963	967	-33%
	int4	755	759	-47%
Qwen3.5-2B	fp16	3590	3595	-
	int8	2284	2289	-36%
	int4	1710	1715	-52%
Gemma3-270M	fp16	512	546	-
	int8	417	451	-19%
	int4	375	409	-27%
MiniCPM5-1B	fp16	2062	2064	-
	int8	1415	1417	-31%
	int4	1131	1133	-45%

Table 5.2: Actual memory usage per model and quantization. Quantization savings are limited by the size of embedding/unembedding layers that are maintained in fp16, and get proportionally lower for smaller models. RoPE tables, activation/sampling buffers and even the attention-score buffers (up to ~ 1 MB per 2048 tokens) add to the total required memory, but are negligible.

Real-Time Text-to-Speech Synthesis

We extended the NPC-to-Player interaction by synchronizing a Text-to-Speech model with the LLM. Though entirely optional, we optimized it well enough to run in real-time and actually build convincing demos. We tried out three different models: **Kokoro**², **pocket-tts** (Rouard et al. 2025) and **CosyVoice 3** (Du et al. 2025). **Kokoro** was the smallest model and though having the lowest RTF in our implementation, it was limited by its own preset voices. On the other hand, **CosyVoice 3** was the largest and most powerful, with voice cloning capabilities and paralinguistic cue rendering. We chose **pocket-tts** as our default model since, at 5 times fewer parameters, it allows voice cloning from very short reference audio clips and still outputs high-quality results, at a much lower RTF. With that, we decided to briefly present its inner workings right below (Figure 5.2).

²<https://huggingface.co/hexgrad/Kokoro-82M>

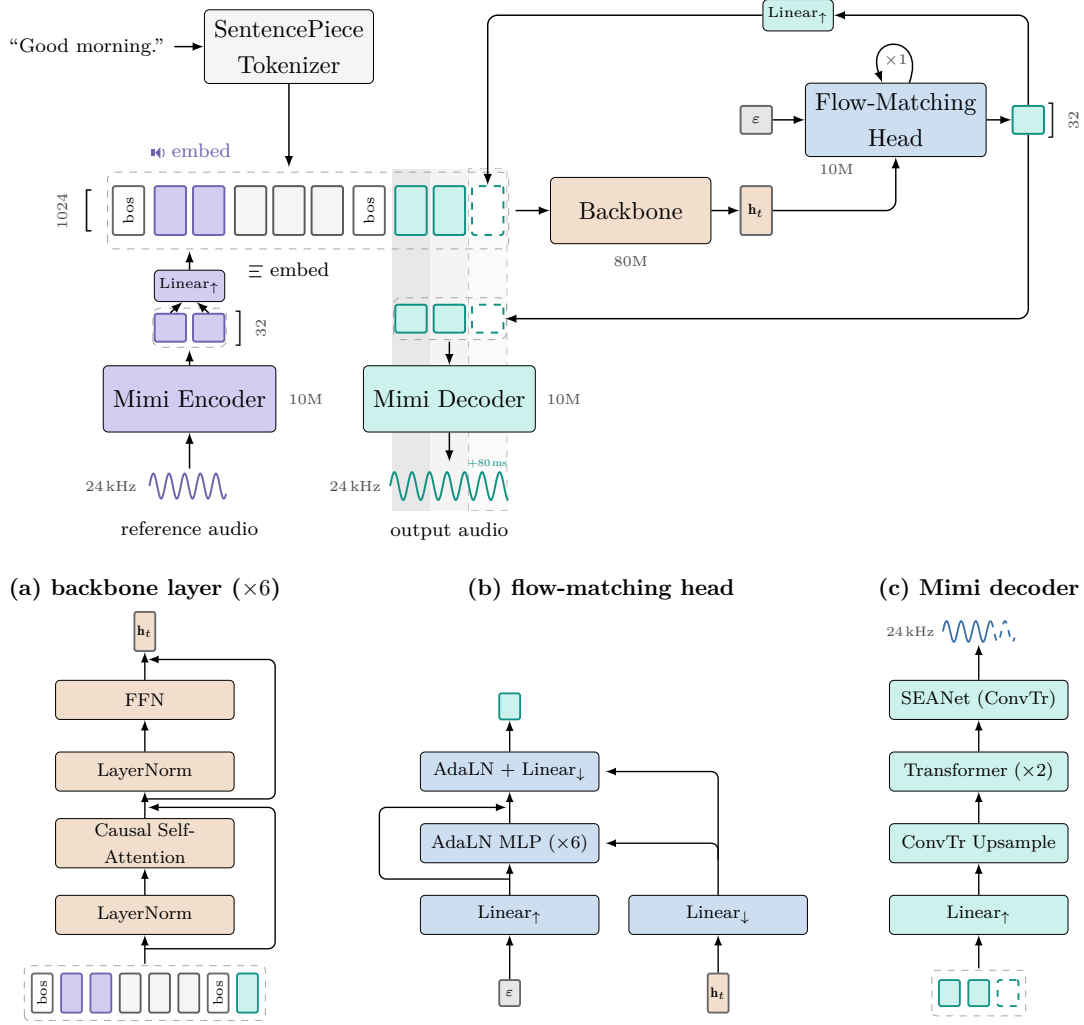


Figure 5.2: **Top:** *pocket-tts* (Rouard et al. 2025) architecture. A pretrained Mimi (Défossez et al. 2024) encoder-decoder is used for voice cloning and generating audio output. A reference audio is encoded into N tokens $\in \mathbb{R}^{32}$. They are up-projected and added to the main sequence once, so this process can be precomputed offline. The target text to be transformed to audio is tokenized and embedded using a pretrained SentencePiece tokenizer. The tokens are added right after the audio tokens. With a reference voice and the target text as context, the audio decoding begins: a Backbone represented by a causal autoregressive transformer and a Flow-Matching head produce the next token in the sequence, which can be concurrently decoded into 80 ms of 24 kHz audio for minimal time to first audio (TTFA). **Bottom:** the dissection of its model components. (a) The backbone is a standard pre-norm autoregressive transformer with six layers; it takes one sequence of inputs and returns the hidden state of the next token. (b) The flow-matching head denoises in one step a randomly sampled latent $\varepsilon \sim \mathcal{N}(0, 1)$ through a sequence of six AdaLN-modulated MLPs. (c) Mimi’s decoder streams audio chunks of 80 ms (mirroring its encoder counterpart) by upsampling the flow-matching head latent and contextualizing it with previous latents in the sequence via two causal transformer layers.

The Performance-Speed Tradeoff Levels

The Performance vs. Speed Tradeoff table of Section 3.3.3 sets five different levels tuned for a wide range of computational budgets, even considering running in parallel the [Qwen 3.5](#) 0.8B and [pocket-tts](#) models. It also has an Auto mode for this pair that self-adjusts to one of the five based on the specifications of the machine it is operating on.

	V. Smooth	Smooth	Balanced	Fast	V. Fast
≡ ↑ weight upload (MB/frame)	4	8	16	24	32
≡ ↑ prefill steps / frame	1	3	6	12	25
≡ ↑ decode tokens / frame	1	1	1	2	3
🔊 ↓ heavy TTS ticks / frame	4	4	3	2	1
🔊 ↓ prebuffer (s)	2.0	1.5	1.0	0.75	0.5
🔊 ↓ stream chunk (80 ms frames)	16	16	8	8	8
🔊 ↓ refill floor (s)	1.25	1.0	0.75	0.6	0.5
🔊 ↓ cede headroom (s)	2.0	2.0	2.0	1.5	1.0
🔊 ↑ MACs / heavy tick	0.9 G	0.9 G	1.5 G	2.5 G	4 G

≡ language model 🔊 voice ↑ grows with the tier ↓ shrinks as the tier rises

Table 5.3: The Performance-Speed Tradeoff. Lower levels are designed for low-end machines or highly demanding graphics, while higher levels are designed for better allocation of resources towards the inference engine. We aligned the table with our machines on the [Qwen 3.5](#) 0.8B + [pocket-tts](#) 100M models. The LLM budgets define the weight loading speed (in MB) per frame (in slow mode, the loading speed is divided by 16), the number of prefill steps and the number of decoded tokens admitted per frame. For audio synthesis, we defined how many heavy TTS ticks may be performed per frame and the multiply-accumulate operations (MACs) allowed per heavy tick, per frame, the GPU-bound synthesis; the prebuffer is the accumulated audio before playback is allowed to start, and the stream chunk is the granularity at which finished audio reaches the ring buffer. The last two rows are the arbiter itself: the voice claims frames back from the language model once its ring drops below the refill floor, and cedes them once it holds more than the cede headroom.

NPC Component Reference

In the table below, we present every field that the `NPCChatBase` component exposes in the Editor Inspector, beside the developer’s script that inherits it. They cover a big part

of the NPC configuration outside the code, so it remains for general use regardless of the scene. Here, the -1 value defers to the models' default presets, as presented on their HuggingFace cards.

	field	default	meaning
SYSTEM PROMPT	NpcName	Villager	Name in window and prompt header
	descriptionAndRules	-	Persona, world facts and rules
	compactSummary	-	Runtime: the compacted history
CONVERSATION	chatWindow	-	The shared dialogue window
	backendTradeoff	Smooth	Picks a row of Table 5.3
	conversationMode	LlmOnly	Text only, or text plus speech
	syncedTextReveal	CharByChar	Typewriter or word-by-word reveal
	historyMode	ResetEveryTime	Whether history survives reopening
	maxToolReadsPerTurn	6	Cap on read-only tool calls per reply
	cacheKVCache	on	Reuse the prefilled system prompt
LLM	model	Qwen3.5-0.8B	Any model in the engine's registry
	quantization	int8	Weight format: fp16, int8 or int4
	maxContextLength	8192	Context window; sizes the KV cache
	enableThinking	off	Reason in <code><think></code> first
SAMPLING	temperature	0.8	Sampling temperature
	maxNewTokens	1024	Reply length cap, in tokens
	topK	-1	Top- k filtering; 0 disables it
	topP	-1	Nucleus filtering; 1 disables it
	minP	-1	Min- p filtering; 0 disables it
	presencePenalty	-1	Presence penalty; 0 disables it
	repetitionPenalty	-1	Repetition penalty; 1 disables it
TTS	ttsModel	pocket-tts	Speech model; four are shipped
	ttsQuantization	int8	Weight format for the speech model
	ttsVoice	-	A voice baked into the export
	clonedVoiceClip	-	Reference clip to clone instead
	voicePitch	1.0	Playback pitch; below 1 is deeper
	voiceVolume	1.4	Loudness, applied to the samples
	clausesPerChunk	2	Clauses per synthesized chunk
	clausePauseSeconds	0.36	Pause between spoken chunks
	replyTailSeconds	0.32	Generated tail on the last chunk
	worldAudioWhileInteracting	1.0	How loud the game stays meanwhile
PREFETCH	usePrefetchZone	off	Load on zone entry, release on exit
	prefetchRadius	10	Zone radius, in metres
	slowPrefetchSeconds	3	Seconds the load is spread over

Table 5.4: NPC MonoBehaviour component fields exposed in the Inspector: Character definition, Conversation setup, Language model configuration, Sampling parameters, Audio Synthesis configuration and Prefetch Zone adjustment.